

การออกแบบและประเมินเชิงสถาปัตยกรรมของระบบซื้อขายพลังงานแสงอาทิตย์แบบ Peer-to-Peer ผ่าน Smart Contract บนเครือข่าย Solana-compatible Consortium ในระบบจำลอง

จันทร์ธวัช กิริยาดี

สาขาวิศวกรรมคอมพิวเตอร์และปัญญาประดิษฐ์ (Computer Engineering and Artificial Intelligence)

มหาวิทยาลัยหอการค้าไทย (University of the Thai Chamber of Commerce)

กรุงเทพมหานคร, ประเทศไทย

2410717302003@live4.utcc.ac.th

บทคัดย่อ (Abstract) — ระบบผลิตไฟฟ้าจากพลังงานแสงอาทิตย์บนหลังคาที่เพิ่มขึ้นทำให้ผู้ใช้ไฟฟ้าจำนวนมากมีพลังงานคงเหลือและต้องการซื้อขายกันเอง แต่การซื้อขายไฟฟ้าระหว่างกันโดยตรงแบบ Peer-to-Peer (P2P) ยังทำได้ยาก เพราะข้อจำกัดของโครงข่ายไฟฟ้าและการขาดกลไกตั้งราคาที่โปร่งใส บทความนี้เสนอการออกแบบและประเมินเชิงสถาปัตยกรรมของระบบซื้อขายพลังงาน P2P บนเครือข่าย consortium blockchain แบบ permissioned ที่กำกับดูแลและมีต้นทุนธุรกรรมคาดการณ์ได้ หัวใจของการออกแบบคือการแยกการตรวจสอบข้อมูลออกจากการชำระธุรกรรมอย่างชัดเจน โดยการตรวจสอบนอกเชนทำผ่าน Aggregator Bridge ที่ยืนยันลายเซ็น Ed25519 ของค่าอ่านมาตรวัด ก่อนจับคู่คำสั่งด้วยกลไก Continuous Double Auction (CDA) ขณะที่การชำระบนเชนบันทึกเฉพาะรายการที่ผ่านการตรวจสอบและบังคับเงื่อนไขความถูกต้องในทุกขั้นตอน คุณค่าของงานจึงไม่ได้อยู่ที่องค์ประกอบใดองค์ประกอบหนึ่ง แต่อยู่ที่การประสานองค์ประกอบทั้งหมดเข้าด้วยกัน การประเมินบนระบบจำลองให้ผลที่สอดคล้องกับแนวคิดนี้ กล่าวคือ เส้นทางรับข้อมูลรองรับมาตรวัด 80 เครื่องที่อัตราเชิงออกแบบ 5.33 รายการต่อวินาทีโดยไม่มีข้อมูลสูญหาย และเมื่อเพิ่มภาระแบบขั้นบันไดถึง 640 เครื่องยังคงสัดส่วนการสูญหายไม่เกิน 0.03% โดยอัตราการรับข้อมูลที่วัดได้เป็น lower bound ของไคลเอนต์ผู้ส่งเดียว เครื่องจับคู่คำสั่งประมวลผลได้ประมาณ 3.1×10^4 คู่คำสั่งต่อวินาที และการชำระบนเชนใช้ 96,707 compute units ต่อคำสั่ง หรือราว 48% ของงบประมาณตั้งต้น ขณะที่ปริมาณงานของเส้นทางชำระจริงบน validator เดียววัดได้ราว 0.5 ธุรกรรมต่อวินาที ซึ่งถูกผูกกับการเขียนบัญชีส่วนกลางโดยการออกแบบ แต่ยังคงรองรับได้ราว 450 การชำระต่อหน้าต่งเคลียร์ตลาด 900 วินาที เกินความต้องการของภาระงานที่ทดสอบ ผลเหล่านี้สนับสนุนการใช้บล็อกเชนเป็นชั้นชำระธุรกรรมแบบบาง ทั้งนี้เป็นการประเมินเชิงสถาปัตยกรรมบนระบบจำลอง ยังไม่ใช้การวัดจากระบบไฟฟ้าจริง

คำสำคัญ (Index Terms) — Consortium Blockchain, Peer-to-Peer, Continuous Double Auction, Proof of Authority, Microservices

I. INTRODUCTION

การขยายตัวของพลังงานหมุนเวียน (Renewable Energy: RE) ทำให้ผู้ใช้ไฟฟ้าจำนวนมากเปลี่ยนบทบาทจากผู้บริโภคเพียงอย่างเดียวไปสู่การเป็นผู้ผลิตและผู้บริโภคพลังงานในเวลาเดียวกัน (Prosumer) การเปลี่ยนแปลงนี้สร้างความท้าทายต่อโครงข่ายไฟฟ้าแบบดั้งเดิม ซึ่งออกแบบมาสำหรับการจ่ายไฟจากศูนย์กลางไปยังผู้ใช้ปลายทางเป็นหลัก งานวิจัยด้านตลาดพลังงานแบบ Peer-to-Peer จึงให้ความสำคัญกับกลไกซื้อขายที่รักษาทั้งข้อจำกัดของโครงข่ายและความน่าเชื่อถือของระบบไฟฟ้า [1], [2], [3]

การเปลี่ยนผ่านไปสู่ระบบ Smart Grid บริบทประเทศไทยตามแผนแม่บทการพัฒนาโครงข่ายไฟฟ้าประเทศไทย มีการใช้โครงสร้างพื้นฐานมาตรวัดขั้นสูง (Advanced Metering Infrastructure: AMI) ทำให้ข้อมูลการผลิตและการใช้พลังงานมีความละเอียดมากขึ้น การบูรณาการทรัพยากรพลังงานแบบกระจายศูนย์ (Distributed Energy Resources: DER) เช่น ระบบผลิตไฟฟ้าพลังงานแสงอาทิตย์บนหลังคา สถานีอัดประจุยานยนต์ไฟฟ้า (EV) และระบบกักเก็บพลังงานด้วยแบตเตอรี่ (Battery Energy Storage System: BESS) ต้องคำนึงถึงข้อกำหนดด้านการเชื่อมต่อ DER, Microgrid controller, Islanding และข้อจำกัดของโครงข่ายแรงดันต่ำ [4], [5], [6]

บทความนี้มีส่วนสนับสนุนหลักสามประการ ประการแรก การออกแบบสถาปัตยกรรมแบบลดการพึ่งพากัน (decoupled) ที่แยกการตรวจสอบข้อมูลนอกเชนผ่าน Aggregator Bridge ได้แก่ การตรวจลายเซ็น Ed25519 ของข้อมูลมาตรวัดตามมาตรฐาน DLMS/COSEM และการประเมินเงื่อนไข Grid stability ร่วมกับการจับคู่คำสั่งด้วย Continuous Double Auction (CDA) ออกจากการชำระธุรกรรมบนเชน (on-chain settlement) อย่างชัดเจน ประการที่สอง

การออกแบบขอบเขตความเชื่อถือ (trust boundary) ของการชำระธุรกรรมบนเครือข่าย Anchor/Solana-compatible แบบ Proof of Authority (PoA) consortium โดยบล็อกเชนตรวจสอบลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขาย การกันส่งข้ามผ่าน order nullifier และสถานะ escrow ขณะที่เงื่อนไขปริมาณพลังงานและ oracle attestation ถูกบังคับใช้ในชั้น off-chain ทำให้บล็อกเชนทำหน้าที่เป็นชั้น settlement และ audit อย่างชัดเจน (ดูหัวข้อ IV) และประการที่สาม การพัฒนาชุดจำลอง AMI ที่อาศัยแบบจำลองโครงข่ายด้วย pandapower และ Smart Meter Simulator เพื่อสร้างข้อมูลมาตรวัดแบบ deterministic พร้อมการวัดอัตราการรับข้อมูลของเส้นทาง telemetry ingest เบื้องต้น จุดต่างหลักจากงานก่อนหน้า (ดูหัวข้อ II) ไม่ได้อยู่ที่องค์ประกอบเดียว (consortium blockchain, CDA หรือ off-chain oracle ซึ่งล้วนมีในงานก่อนหน้า) แต่อยู่ที่การผสมผสานองค์ประกอบเหล่านี้เข้าด้วยกันผ่านขอบเขตความเชื่อถือที่นิยามชัด คือ CDA แบบแบ่งโซนบนเครือข่าย consortium PoA ที่แยกชั้นตรวจสอบ telemetry (Ed25519/DLMS) นอกเชนออกจากชั้น settlement อย่างชัดเจน โดยกลไกการชำระบนเชนที่เป็นแกนของงานนี้คือการส่งมอบพร้อมชำระแบบ atomic (DvP) ที่รวมการตรวจลายเซ็น Ed25519 ของทั้งสองฝ่าย การกันส่งข้ามแบบ partial-fill ผ่าน order nullifier และเพดานการไหลข้ามโซนไว้เป็นชุดเงื่อนไขความถูกต้องเดียวที่บังคับใช้ในธุรกรรมเดียว ซึ่งงานก่อนหน้ามิได้ระบุไว้อย่างเป็นระบบ ทั้งนี้ขอบเขตของงานเป็นการออกแบบและประเมินเชิงสถาปัตยกรรมบนระบบจำลอง ไม่ใช้การวัดจากเครือข่ายไฟฟ้าภาคสนามหรือเครือข่าย Solana production ส่วนสนับสนุนเชิงประจักษ์ของงานจึงเป็นการบ่งชี้คุณลักษณะ (characterization) ของระบบเดียวแบบทำซ้ำได้ ได้แก่ อัตราการรับข้อมูล ต้นทุน compute-unit และปริมาณงานของเส้นทางชำระ มิใช่การเปรียบเทียบเชิงปริมาณแบบ head-to-head กับแพลตฟอร์มอื่น ซึ่งเป็นงานในอนาคต

ในด้านกลไกเชิงเศรษฐศาสตร์ ระบบใช้โทเคนพลังงาน (energy token) ที่ออกจากการผลิตจริงและรับรองด้วย Renewable Energy Certificate (REC) โดยหนึ่งโทเคน REC แทนพลังงานสีเขียวที่รับรองแล้ว 1 MWh ทั้งนี้ในบริบทการกำกับของไทย REC ออกโดยคณะกรรมการกำกับกิจการพลังงาน (Energy Regulatory Commission; ERC) ซึ่งเป็นหน่วยงานรัฐผู้มีอำนาจออกใบรับรองแต่เพียงผู้เดียว โปรแกรมบนเชนจึงตั้งชื่อคำสั่งออกใบรับรองและบัญชีใบรับรองด้วยตัวระบุ *erc* (*issue_erc*, *ErcCertificate*) ตามนั้น ร่วมกับเหรียญ stablecoin ที่ตรึงค่ากับเงินบาท (THBC) สำหรับการตั้งราคาและชำระธุรกรรม และโทเคน GRX สำหรับการ stake และการกำกับดูแล (governance) โดยรายละเอียดแบบจำลองราคาอธิบายไว้ในหัวข้อ V และรายละเอียดของโปรแกรมที่เกี่ยวข้องอธิบายไว้ในหัวข้อ VI.E

บทความนี้จัดเรียงเนื้อหาดังนี้หัวข้อ II ทบทวนงานวิจัยที่เกี่ยวข้องด้านบล็อกเชนและตลาดพลังงานแบบ Peer-to-Peer หัวข้อ III กำหนดแบบจำลองระบบ ความเชื่อถือ และผู้โจมตีหัวข้อ IV อธิบายแบบจำลองการชำระธุรกรรมของระบบหัวข้อ V อธิบายกลไกราคา CDA และการคำนวณ settlement หัวข้อ VI อธิบายสถาปัตยกรรม การเชื่อมต่อ และรายละเอียด implementation หลักหัวข้อ VII ระบุรายละเอียดการทดลองและการะงานหัวข้อ VIII สรุปการประเมินเชิงสถาปัตยกรรม และหัวข้อ IX อภิปรายผล ข้อจำกัด และแนวทางการพัฒนาระบบในอนาคต ด้วยข้อที่ข้อบ่งชี้บทความสรุปไว้ในตารางที่ I

ตารางที่ I
Abbreviations used in this paper.

ตัวย่อ	ความหมาย
AMI	Advanced Metering Infrastructure (โครงสร้างพื้นฐานมาตรวัดขั้นสูง)
BESS	Battery Energy Storage System
BFT	Byzantine Fault Tolerance
CDA	Continuous Double Auction (ตลาดประมูลสองทางแบบต่อเนื่อง)
CPI	Cross-Program Invocation
CU	Compute Unit (หน่วยประมวลผลบนเชนของ Solana)
DAO	Decentralized Autonomous Organization
DER	Distributed Energy Resource (ทรัพยากรพลังงานแบบกระจายศูนย์)
DLMS/COSEM	Device Language Message Specification / COSEM (IEC 62056)
DSO	Distribution System Operator
DvP	Delivery-versus-Payment (การส่งมอบพร้อมชำระเงินแบบ atomic)
GRX / THBC	Governance token / THB-pegged stablecoin (โทเคนกำกับดูแล / stablecoin ตรึงเงินบาท)
IAM	Identity and Access Management
mTLS	Mutual TLS
OPF	Optimal Power Flow
PDA	Program Derived Address
PoA	Proof of Authority
PoH	Proof of History
RBAC	Role-Based Access Control
REC	Renewable Energy Certificate
SPIFFE	Secure Production Identity Framework for Everyone
SPL	Solana Program Library (Token)
SVM	Solana Virtual Machine
VPP	Virtual Power Plant

II. RELATED WORK

เทคโนโลยีบล็อกเชนถือกำเนิดจาก Bitcoin [7] ในฐานะระบบบัญชีแยกประเภทแบบกระจายศูนย์ที่ไม่ต้องอาศัยตัวกลาง และขยายขีดความสามารถสู่การประมวลผล Smart Contract ด้วย Ethereum [8], [9] ภาพรวมของเทคโนโลยีและการจำแนกประเภทเครือข่ายแบบ permissionless และ permissioned ได้รับการสรุปไว้โดย NIST [10] ซึ่งเป็นกรอบอ้างอิงในการเลือกสถาปัตยกรรมเครือข่ายให้เหมาะกับข้อกำหนดด้านการกำกับดูแล

ในบริบทของตลาดพลังงานแบบ Peer-to-Peer มีงานวิจัยจำนวนมากที่ศึกษาตลาดและการออกแบบการประมูล Mengelkamp และคณะ [11] เปรียบเทียบการออกแบบตลาดพลังงานท้องถิ่นและกลยุทธ์การเสนอราคา ขณะที่ Munsing และคณะ [12] เสนอการใช้บล็อกเชนเพื่อกระจายการหาค่าที่เหมาะสมที่สุด (decentralized optimization) ของทรัพยากรพลังงานในเครือข่ายไม่โครกริด งานเหล่านี้ชี้ให้เห็นถึงศักยภาพของบล็อกเชนในการรองรับการซื้อขายพลังงานโดยตรงระหว่างผู้ใช้ แต่ส่วนใหญ่ยังประเมินบนเครือข่ายสาธารณะที่มีข้อจำกัดด้านต้นทุนธุรกรรมและความหน่วงในการยืนยันบล็อก ตัวอย่างที่เป็นรูปธรรมล่าสุดคือแพลตฟอร์มซื้อขายแบบกระจายศูนย์ของ Esmat และคณะ

[13] ที่พัฒนาตลาดและการชำระบนบล็อกเชนสาธารณะ (Ethereum) แต่ไม่ได้แยกชั้นการตรวจสอบข้อมูลนอกเชนจากการชำระบนเชนอย่างชัดเจน

เพื่อตอบโจทย์ด้านการกำกับดูแลและความสามารถในการคาดการณ์ต้นทุนงานวิจัยจำนวนหนึ่งจึงหันมาใช้เครือข่ายแบบ Consortium หรือ permissioned Kang และคณะ [14] ใช้ consortium blockchain สำหรับการซื้อขายไฟฟ้าแบบ Peer-to-Peer ระหว่างยานยนต์ไฟฟ้าแบบ plug-in และ Hyperledger Fabric [15] เป็นตัวอย่างของระบบปฏิบัติการแบบกระจายสำหรับเครือข่าย permissioned ที่กำหนดสิทธิ์ผู้เข้าร่วมได้ชัดเจน ในด้านฉันทามติ ปัญหา Byzantine Generals [16] และอัลกอริทึม Practical Byzantine Fault Tolerance [17] เป็นรากฐานเชิงทฤษฎีของการยืนยันธุรกรรมในเครือข่ายที่มีโหนดจำนวนจำกัด ซึ่งสอดคล้องกับสมมติฐานเครือข่าย permissioned ที่ควบคุมการเข้าร่วม validator แบบ Proof of Authority (PoA) ในงานนี้ (ทั้งนี้ PoA เป็นขั้นกำกับการเข้าร่วม ส่วนฉันทามติระดับ Layer 1 ยังคงอาศัย PoH ร่วมกับ Tower BFT)

การทบทวนเชิงระบบของ Andoni และคณะ [18] สำรวจการประยุกต์บล็อกเชนในภาคพลังงานอย่างครอบคลุมและชี้ความท้าทายหลักด้านความสามารถในการขยายขนาด ต้นทุน และการกำกับดูแล ซึ่งเป็นกรอบอ้างอิงตั้งต้นของงานวิจัยกลุ่มนี้ งานทบทวนวรรณกรรมล่าสุดยังสะท้อนว่าหัวข้อนี้ยังเป็นพื้นที่วิจัยที่เปิดอยู่ โดย Tanis และคณะ [19] ทบทวนโครงสร้างตลาด ขั้นตอนดำเนินงาน และระบบหลายพลังงานของการซื้อขายแบบ Peer-to-Peer อย่างครอบคลุม ขณะที่ Bhavana และคณะ [20] สำรวจการประยุกต์บล็อกเชนในตลาดพลังงานและห่วงโซ่อุปทานไฮโดรเจนสีเขียว และชี้ว่าความสามารถในการขยายขนาด (scalability) ต้นทุน และความสอดคล้องด้านการกำกับดูแลยังเป็นความท้าทายหลัก นอกจากนี้การประเมินเปรียบเทียบกลไกฉันทามติสำหรับการซื้อขายพลังงานแบบ Peer-to-Peer ในไมโครกริด [21] สนับสนุนการเลือกเครือข่าย permissioned ที่ควบคุมสิทธิ์ได้ ซึ่งสอดคล้องกับสมมติฐานการกำกับการเข้าร่วมแบบ PoA ที่ใช้ในงานนี้

ต่างจากงานข้างต้น บทความนี้มุ่งเน้นการออกแบบและประเมินสถาปัตยกรรมของระบบจำลองที่แยกการตรวจสอบนอกเชน (off-chain verification) ผ่าน Aggregator Bridge ออกจากขั้น Settlement บนบล็อกเชนอย่างชัดเจน ขอบเขตของงานนี้จึงเป็นการออกแบบและประเมินสถาปัตยกรรมไม่ใช้การรายงานผลวัดจากเครือข่ายไฟฟ้าภาคสนามหรือ Solana production network โดยข้อมูลพลังงานที่ใช้ในต้นแบบมาจาก AMI Simulator และผลที่บันทึกบนบล็อกเชนเป็น Settlement event ที่ผ่านการตรวจสอบจาก Backend/Aggregator Bridge แล้ว ซึ่งสอดคล้องกับแนวทางเบื้องต้นในการทดสอบระบบควบคุมไมโครกริด [22] ตำแหน่งของงานนี้เทียบกับงานที่ใช้บล็อกเชนสำหรับตลาดพลังงานแบบ Peer-to-Peer สรุปไว้ในตารางที่ II โดยจุดต่างหลักคือการผสาน CDA แบบแบ่งโซนบนเครือข่าย consortium แบบ PoA เข้ากับการแยกชั้นตรวจสอบข้อมูลมาตรวัด (Ed25519/DLMS) นอกเชนออกจากขั้น settlement อย่างชัดเจน ทั้งนี้ตารางที่ II เป็นการเทียบเชิงคุณภาพ (qualitative positioning) เนื่องจากงานเหล่านี้รันบนเครือข่าย ชุดข้อมูล และสมมติฐานที่ต่างกัน จึงไม่มีตัวชี้วัดเชิงปริมาณร่วมที่เทียบกันได้โดยตรง

ตารางที่ II
Positioning of this work vs prior blockchain-based P2P energy-trading studies.

งาน	เครือข่าย	ฉันทามติ	กลไกตลาด	แยก off/on-chain
Mengelkamp [11]	Public (LEM)	Public	Local market / bidding	ไม่แยกชัดเจน
Munsing [12]	Public	Public	Decentralized optimization	ไม่แยกชัดเจน
Kang [14]	Consortium	Consortium	Iterative double auction	บางส่วน
Hyperledger Fabric [15]	Permissioned	Pluggable (Raft/BFT)	แพลตฟอร์ม (ไม่เจาะตลาด)	—
Esmat [13]	Public (Ethereum)	Public	Double auction	บางส่วน
งานนี้ (This work)	Consortium (Solana-compatible)	PoH + Tower BFT (กำกับเข้าร่วมแบบ PoA)	CDA price-time แบ่งโซน	แยกชัดเจน + Ed25519/DLMS telemetry

III. SYSTEM MODEL AND THREAT MODEL

ส่วนนี้กำหนดแบบจำลองระบบ (system model) ผู้มีส่วนร่วม (actors) สมมติฐานความเชื่อถือ (trust assumptions) และแบบจำลองผู้โจมตี (adversary model) ของระบบจำลอง เพื่อให้ขอบเขตด้านความปลอดภัยของสถาปัตยกรรมแบบแยกชั้นชัดเจนก่อนการอภิปรายแบบจำลองการชำระธุรกรรม ในหัวข้อ IV ทั้งนี้ขอบเขตของงานเป็นการออกแบบระบบจำลอง การวิเคราะห์ด้านล่างจึงระบุถึงภัยคุกคามที่กลไกปัจจุบันรองรับและความเชื่อถือที่ยังคงเหลืออยู่ (residual trust) อย่างตรงไปตรงมา

A. System Model and Actors

ระบบประกอบด้วยผู้มีส่วนร่วมหลักดังนี้

- Smart Meter (edge device): อุปกรณ์ปลายทางที่ถือกุญแจคู่ Ed25519 ประจำเครื่อง ลงนามค่าอ่านพลังงานก่อนส่งออก
- Aggregator Bridge: ชั้นตรวจสอบและรวบรวมข้อมูลนอกเซน ตรวจสอบลายเซ็น Ed25519 ของทุกค่าอ่านเทียบกับกุญแจสาธารณะของอุปกรณ์ ถอดรหัส payload ตามมาตรฐาน DLMS/COSEM และประเมินเงื่อนไขปริมาณพลังงานคงเหลือและ Grid stability
- Trading Service: จับคู่คำสั่งซื้อขายด้วย Continuous Double Auction (CDA)
- Chain Bridge: ทำหน้าที่เป็น Reference Monitor และเป็นบริการเดียวที่ถือกุญแจลงนามและเรียก Solana RPC โดยตรง ทุกธุรกรรมที่เขียนลงเงินถูกบีบให้ผ่านเส้นทางลงนามเดียว (single signing path)
- Anchor Programs: ขึ้นบังคับใช้คดีกานเซนที่รับเฉพาะคู่คำสั่งที่ผ่านการตรวจสอบแล้ว และทำหน้าที่เป็นชั้น settlement และ audit
- PoA / Governance Authority: หน่วยงานผู้มีสิทธิ์หลักตามนโยบาย consortium ที่ควบคุมการรับรอง REC สิทธิ์ผู้มีอำนาจ และ allow-list ของ aggregator

B. Trust Assumptions

สมมติฐานความเชื่อถือของระบบถูกบังคับใช้ในสามขอบเขต ได้แก่ ขอบเขตอุปกรณ์-สู่-บริดจ์ (device-to-bridge) ขอบเขตระหว่างบริการ (service-to-service) และขอบเขตบริการ-สู่-เซน (service-to-chain)

ในขอบเขตอุปกรณ์-สู่-บริดจ์ ตัวตนของอุปกรณ์แต่ละเครื่องผูกกับกุญแจสาธารณะ Ed25519 ที่ลงทะเบียนไว้ Aggregator Bridge ตรวจสอบลายเซ็นของค่าอ่านทุกรายการ (ลายเซ็น 64 ไบต์ กุญแจสาธารณะ 32 ไบต์) ทั้งแบบรายจุดและแบบกลุ่ม (batch) โดยใช้นโยบายแบบ fail-closed กล่าวคือเมื่อค้นกุญแจไม่พบหรือแหล่งเก็บกุญแจไม่ตอบสนอง ระบบจะปฏิเสธ (error) มิใช่ยอมรับค่าอ่านโดยปริยาย ส่วน payload แบบ Binary ตามมาตรฐาน DLMS/COSEM ถูกเข้ารหัสด้วย AES-256-GCM ด้วยกุญแจประจำอุปกรณ์ ในโหมด production หากอุปกรณ์ไม่มีกุญแจเข้ารหัส เฟรมนั้นจะถูกข้าม (fail-closed) และเส้นทาง plaintext เปิดใช้ได้เฉพาะในโหมดพัฒนาเท่านั้นผ่านการตั้งค่าที่ระบุชุด

ในขอบเขตระหว่างบริการที่เขียนสู่เซน (โดยเฉพาะเส้นทางสู่ Chain Bridge) การสื่อสารใช้ ConnectRPC บน mutually authenticated TLS (mTLS) โดยผู้ตัวตนของบริการด้วย SPIFFE [23] X.509 identity ที่ตรวจสอบจากใบรับรองฝั่ง client ในขั้นตอน handshake สิทธิ์การเข้าถึงถูกกำหนดจาก SPIFFE URI ของใบรับรองที่ผ่านการตรวจสอบแล้ว แปลงเป็นบทบาทบริการ (service role) เช่น *AggregatorBridge*, *TradingMatcher*, *SettlementService* ทั้งนี้ตัวตนถูกพิจารณาจากชั้น transport (L4) มีส่วนหัวของแอปพลิเคชัน (L7) ผู้เรียกที่ไม่ผ่านการตรวจสอบจะได้รับบทบาท *Unknown* อนึ่ง เส้นทางรับข้อมูลมาตรวัดที่ Aggregator Bridge ใช้กลไกพิสูจน์ตัวตนระดับคำขอด้วย API key (ตรวจสอบผ่าน IAM พร้อม static key สำรอง) ประกอบกับลายเซ็น Ed25519 รายค่าอ่านมิใช่ mTLS/SPIFFE ส่วนการยกเว้นตรวจสอบ (insecure mode) ที่ให้สิทธิ์เต็มไว้สำหรับโหมดพัฒนาเท่านั้น

ในขอบเขตบริการ-สู่-เซน Chain Bridge เป็นบริการเดียวที่ถือกุญแจและชำระธุรกรรม โดยกุญแจลงนามถูกจัดเก็บใน HashiCorp Vault Transit (Ed25519) และไม่เคยเข้าสู่หน่วยความจำของกระบวนการ การลงนามถูกจำกัดด้วยชื่อกุญแจที่ได้รับอนุญาตเท่านั้น สำหรับเส้นทางเขียนแบบอะซิงโครนัสผ่าน NATS JetStream [24] ผู้เผยแพร่ (publisher) ต้องลงนามของข้อความ

(envelope) ด้วยลายเซ็น ECDSA P-256 เหนือไบต์แบบ canonical ของของซึ่งตรวจสอบเทียบกับกุญแจสาธารณะในใบรับรอง client ของผู้เผยแพร่ และ Chain Bridge ตรวจสอบลำดับ ใบรับรอง → CA → SPIFFE SAN เทียบกับ service identity → ลายเซ็น ก่อนขั้นตอน RBAC โดยหัวข้อที่เคลื่อนย้ายมูลค่าเช่น การ mint โทเคนพลังงาน ถูกบังคับให้ต้องมีของที่ลงนามเสมอ

C. Adversary Model and Mitigations

แบบจำลองผู้โจมตีพิจารณาผู้โจมตีที่สามารถสร้าง ดักจับ หรือส่งซ้ำข้อความบนเครือข่าย และอาจพยายามปลอมตัวตนของอุปกรณ์หรือบริการ แต่ไม่สามารถเข้าถึงกุญแจส่วนตัวที่จัดเก็บใน Vault หรือกุญแจประจำอุปกรณ์ได้ ภัยคุกคามที่พิจารณาและกลไกรองรับสรุปไว้ในตารางที่ III

ตารางที่ III Threats considered and the mechanism that mitigates each.		
ภัยคุกคาม	คำอธิบาย	กลไกรองรับ
T1 ค่าอ่านปลอม/ส่งซ้ำ	ปลอมหรือเล่นซ้ำค่าอ่านมาตรวัด	ตรวจสอบลายเซ็น Ed25519 ต่อค่าอ่าน (fail-closed) + บัญชี <i>gen_mint</i> PDA ต่อคู่ (<i>meter_id, window</i>) ซึ่งดูกลืนค่าอ่านที่ถูกส่งซ้ำเข้าหน้าต่างเวลาเดิม
T2 ปลอมตัวตนบริการ	แอบอ้างเป็นบริการในเมซ	mTLS + SPIFFE identity → service role; ผู้ไม่ผ่านได้ <i>Unknown</i>
T3 ปลอม publisher / mint	ส่งคำสั่งเขียนเซนปลอมผ่าน NATS	ของ envelope ลงนามผูกกับใบรับรอง; หัวข้อ mint บังคับลงนาม
T4 ส่งซ้ำคำสั่งลงนามเซน	ชำระคู่คำสั่งเดิมซ้ำ	บัญชี order nullifier (PDA) — ชำระได้ครั้งเดียว
T5 mint ซ้ำ/เกินสิทธิ์	สร้างโทเคนเกินการผลิตจริง	คีย์ idempotency (<i>meter_id, window</i>) + PDA + การร่วมลงนาม REC

D. Residual Trust and Out-of-Scope

ระบบยังคงมีความเชื่อถือที่เหลืออยู่ซึ่งต้องระบุอย่างชัดเจน ประการแรก Aggregator Bridge และ governance/oracle authority เป็นผู้รับรองเงื่อนไขปริมาณพลังงานคงเหลือและ Grid stability ในชั้น off-chain โดยเงื่อนไขเหล่านี้ไม่ได้ถูกตรวจสอบข้ามเซน ดังนั้นผู้ใช้ต้องเชื่อถือความถูกต้องของชั้นตรวจสอบนอกเซนนี้ การประนีประนอม (compromise) หรือการสมรู้ร่วมคิด (collusion) ของ Aggregator Bridge หรือ oracle authority ยังไม่ถูกป้องกันด้วยกลไกเชิงรหัสวิทยาในต้นแบบปัจจุบัน แนวทางลด trust เพิ่มเติม เช่น multi-oracle attestation หรือ cryptographic proof เป็นงานในอนาคต (ดูหัวข้อ IX) ประการที่สอง ความปลอดภัยของชั้นฉันทามติ (PoH ร่วมกับ Tower BFT) ตั้งอยู่บนสมมติฐานเครือข่าย permissioned ที่ควบคุมการเข้าร่วม validator แบบ PoA โดย validator เป็นหน่วยงานได้รับอนุญาตและมีพฤติกรรมตามนโยบายซึ่งเป็นสมมติฐานเชิงสถาปัตยกรรมที่ยังไม่ได้ประเมินเชิงปริมาณ ประการที่สาม การยกเว้นตรวจสอบสำหรับโหมดพัฒนา (insecure mode และ plaintext fallback) ต้องถูกปิดในการใช้งานจริง มิฉะนั้นจะทำลายสมมติฐานความเชื่อถือข้างต้นทั้งหมด

IV. SETTLEMENT MODEL

A. Architecture Overview

ระบบแบ่งความรับผิดชอบออกเป็นสองโดเมนความเชื่อถือ (trust domain) ที่เชื่อมต่อกันผ่านจุดเดียว โดเมนนอกเซน (off-chain) ทำหน้าที่ตรวจสอบและจับคู่ ประกอบด้วยชั้นมาตรวัด (Smart Meter Simulator ตามมาตรฐาน AMI และ DLMS/COSEM) ที่ป้อนข้อมูลเข้าสู่ Aggregator Bridge เพื่อยืนยันลายเซ็น Ed25519 และคัดกรองข้อมูล ก่อนส่งต่อให้ Trading Service จับคู่คำสั่งด้วยอัลกอริทึม Continuous Double Auction (CDA) โดยมี IAM Service และ Notification Service สนับสนุนด้านการระบุตัวตนและการแจ้งเตือน ส่วนโดเมนบนเซน (on-chain) ทำหน้าที่ชำระธุรกรรมและบันทึกหลักฐาน ประกอบด้วยกลุ่มโปรแกรม Anchor (registry, trading, oracle, energy-token, governance และ treasury) บนเครือข่าย Solana-compatible แบบ permissioned (consortium) ที่บันทึกสถานะลงบัญชี PDA และ Event log

การข้ามจากโดเมนนอกเชนไปยังโดเมนบนเชนเกิดขึ้นผ่าน Chain Bridge เพียงทางเดียว ซึ่งเป็นบริการเดียวที่ติดต่อกับ Solana RPC โดยตรง โดยรับคำสั่งเขียนผ่าน NATS JetStream และให้บริการอ่านสถานะผ่าน ConnectRPC บนช่องทางที่ป้องกันด้วย mTLS โปรแกรมบนเชนเป็น Anchor program [25] ที่ทำงานบน Solana Virtual Machine (SVM) [26] และใช้ Program Derived Address (PDA) เพื่อสร้างบัญชีที่ตรวจสอบ ownership ได้แบบ deterministic การออกแบบที่แยกสองโดเมนเช่นนี้ทำให้การตรวจสอบเชิงวิศวกรรมไฟฟ้าและการควบคุมสิทธิ์เกิดขึ้นก่อนการบันทึกธุรกรรม ขณะที่บล็อกเชนทำหน้าที่เป็นชั้นชำระธุรกรรมและตรวจสอบย้อนหลังที่บาง (thin settlement and audit layer) รายละเอียดของบริการ การเชื่อมต่อ และแผนภาพสถาปัตยกรรมทั้งหมดอยู่ในหัวข้อ VI ที่เน้นบทความนี้เป็นการประเมินเชิงสถาปัตยกรรมบนระบบจำลอง ไม่ใช่ผลผลิตจากเครือข่าย Solana production

B. Settlement Model

แบบจำลองการชำระธุรกรรมแยกความรับผิดชอบออกเป็นสองชั้นที่มีขอบเขตความเชื่อต่างกันอย่างชัดเจน ชั้นนอกเชน (off-chain) ทำหน้าที่ตรวจสอบและจับคู่ ประกอบด้วย Aggregator Bridge ที่ยืนยันลายเซ็น Ed25519 ของคำอ่านมาตรวัด ตรวจสอบรูปแบบข้อมูลตามมาตรฐาน DLMS/COSEM และประเมินเงื่อนไขปริมาณพลังงานคงเหลือ (available surplus) ร่วมกับ Grid stability และ Trading Service ที่จับคู่คำสั่งซื้อขายด้วยอัลกอริทึม Continuous Double Auction (CDA) ผลลัพธ์ของชั้นนี้คือคู่คำสั่งที่ผ่านการตรวจสอบแล้วพร้อม order payload ที่ลงนามโดยทั้งสองฝ่าย ส่วนชั้นบนเชน (on-chain) คือ Anchor program [25] ที่ทำหน้าที่เป็นชั้นชำระธุรกรรมและบันทึกหลักฐานแบบบาง (thin settlement and audit layer) โดยรับเฉพาะคู่คำสั่งที่ผ่านการตรวจสอบแล้วและบันทึกผลที่ตรวจสอบย้อนหลังได้

ก่อนบันทึก settlement โปรแกรมบนเชนบังคับใช้เฉพาะเงื่อนไขที่ตรวจสอบได้จาก payload และสถานะบัญชี ได้แก่ การตรวจ account ownership ลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขายบน order payload การกันส่งข้ามบัญชี order nullifier และความถูกต้องของสถานะ escrow ส่วนเงื่อนไขที่ต้องอาศัยข้อมูลภายนอก เช่น ปริมาณพลังงานคงเหลือและ oracle attestation ถูกบังคับใช้ในชั้น off-chain ก่อนการ submit และไม่ถูกตรวจข้ามเชน การแบ่งหน้าที่เช่นนี้ทำให้บล็อกเชนไม่ต้องไว้วางใจข้อมูลภายนอกโดยตรง แต่ยืนยันได้ว่าทุกการชำระมาจากคู่คำสั่งที่ทั้งสองฝ่ายลงนามจริง โครงสร้างบัญชีทั้งหมดใช้ Program Derived Address (PDA) เพื่อแยก state ของแต่ละธุรกรรมและตรวจสอบ ownership ได้แบบ deterministic รายละเอียดของโปรแกรมและบัญชี PDA อธิบายในหัวข้อ VI.E [27]

C. Atomic Delivery-versus-Payment

คู่คำสั่งที่จับคู่แล้วถูกชำระผ่านคำสั่ง `settle_offchain_match` ของโปรแกรม trading ซึ่งทำงานแบบส่งมอบพร้อมชำระเงิน (Delivery-versus-Payment: DvP) ภายในธุรกรรมเดียว กล่าวคือการโอนเงินและการโอนพลังงานเกิดขึ้นพร้อมกันหรือไม่เกิดขึ้นเลย หากการโอนใดล้มเหลวธุรกรรมทั้งหมดจะถอยกลับ (revert) จึงไม่มีสถานะค้างที่ฝ่ายหนึ่งได้รับโดยอีกฝ่ายไม่ได้รับ ก่อนการโอน โปรแกรมตรวจลายเซ็น Ed25519 ของทั้งผู้ซื้อ (instruction sysvar ดัชนี 0) และผู้ขาย (ดัชนี 1) บนข้อความที่ผูกฟิลด์สำคัญของคำสั่ง ได้แก่ order_id, user, energy_amount, price_per_kwh, side, zone_id และ expires_at ทำให้ดัดแปลงปริมาณหรือราคาหลังการลงนามไม่ได้ จากนั้นตรวจเงื่อนไขการครอสราคา (price crossing) คือราคาจับคู่ต้องอยู่ในช่วง $p_s \leq p^* \leq p_b$ ตรวจ side ของทั้งสองฝั่ง และตรวจเวลาหมดอายุ

การชำระเป็นแบบ partial-fill โดยบัญชี order nullifier (seed [nullifier; user; order_id]) เก็บปริมาณที่ถูกเติมสะสม (filled_amount) แทนค่าบูตลิน คำสั่งจึงถูกเติมได้หลายครั้งจนเต็มปริมาณที่ลงนามไว้ แต่ผลรวมจะไม่เกินปริมาณนั้น ($match \leq energy_amount - filled$) ทั้งสองฝั่ง สำหรับธุรกรรมข้ามโซน (cross-zone) ที่ใช้สายส่งระหว่างโซน โปรแกรมยังบังคับเพดานการไหลสะสมบนเชน ($committed_flow + match \leq capacity$) ส่วนธุรกรรมภายในโซนเดียวกันได้รับการยกเว้นเพราะไม่ใช้สายส่งระหว่างโซน เมื่อผ่านเงื่อนไขทั้งหมด มูลค่ารวมและส่วนแบ่งคำนวณดังนี้ โดยเงิน (currency)

ไหลจาก buyer escrow ไปยังตัวเก็บค่าธรรมเนียม/wheeling/loss และ seller escrow ส่วนพลังงาน (energy token) ไหลจาก seller escrow ไปยัง buyer escrow การโอนออกจากบัญชี escrow ทั้งหมดลงนามโดย market_authority PDA ในฐานะเจ้าของบัญชี escrow (ลงนาม CPI การโอนโทเคน) ขณะที่การอนุมัติการชำระธุรกรรมมาจากลายเซ็น Ed25519 ของผู้ซื้อและผู้ขายตามที่อธิบายข้างต้น นอกจากคำสั่งจับคู่เดี่ยวแล้ว โปรแกรมยังมีคำสั่งชำระแบบกลุ่ม (`batch_settle_offchain_match`) ที่รวมได้สูงสุด 4 คู่ต่อหนึ่งธุรกรรมเพื่อลดต้นทุน โดยใช้เงื่อนไขตรวจสอบชุดเดียวกันทั้งหมด (ลายเซ็น Ed25519, order nullifier, การครอสราคา และเพดานการไหลข้ามโซน) และผูกบัญชีจาก payload ที่ลงนามด้วยการตรวจ Program Derived Address (PDA) ของทุกบัญชีที่ส่งเข้ามา

$$V = match \cdot p^* \quad (1.1)$$

$$fee = \lfloor V \cdot \varphi / 10000 \rfloor \quad (1.2)$$

$$net_s = V - fee - w - c_{loss} \quad (1.3)$$

สมการชุดนี้เป็นรูปแบบจำนวนเต็มแบบปิดลง (fixed-point) บนเชนของ Equation 4.1 ถึง Equation 4.3 ในหัวข้อ V.A โดย V คือมูลค่ารวมที่ออกจาก buyer escrow, φ คือค่าธรรมเนียมตลาด (market_fee_bps), w คือ wheeling charge, c_{loss} คือต้นทุน loss (ดู Equation 2) และ net_s คือยอดสุทธิที่ผู้ขายได้รับ ทั้งนี้การคำนวณทั้งหมดใช้ checked arithmetic แทนการ clamp ค่า กล่าวคือการคูณมูลค่าปฏิเสธกรณี overflow และการหักลบยอดสุทธิของผู้ขายจะ ปฏิเสธ ธุรกรรม (require! ด้วย error *ChargesExceedValue*) เมื่อผลรวมของค่าธรรมเนียม wheeling และ loss เกินมูลค่า V แทนการปิดยอดลงเป็นศูนย์ พร้อมเพดานค่าธรรมเนียมเครือข่ายรวมไม่เกิน 20% ของ V และเมื่อตลาดตั้งค่าให้ชำระด้วย THBC ระบบบังคับให้บันทึกมูลค่ารวม (gross) ผ่าน CPI `record_settlement` ไปยังโปรแกรม treasury เพื่อให้ตัวนับยอดชำระกระทบยอดกับกระแสเงินที่ออกจาก escrow จริง

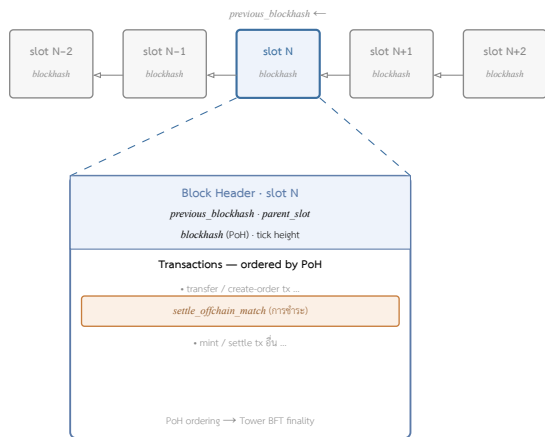
D. Consensus

สัญญาอัจฉริยะ (smart contract) ของระบบเป็นโปรแกรม Anchor ที่ทำงานบน Solana Virtual Machine (SVM) การจัดลำดับและการบรรลุฉันทามติของธุรกรรมจึงเป็นหน้าที่ของเครือข่าย Solana-compatible ที่นำโปรแกรมไป deploy มิได้ถูกนิยามหรือปรับแต่งในระดับโปรแกรม กลไกฉันทามติของแพลตฟอร์มแยกออกเป็นสองส่วนที่ทำงานร่วมกัน ได้แก่ Proof of History (PoH) ที่เป็นนาฬิกาเชิงรหัสวิทยา (verifiable clock) สำหรับเรียงลำดับเหตุการณ์ตามเวลาก่อนการลงมติ และ Tower BFT ซึ่งเป็นกลไกฉันทามติแบบ Byzantine Fault Tolerant ที่พัฒนาต่อจากแนวคิด Practical BFT (PBFT) [17] สำหรับยืนยันบล็อกและประกาศสิ้นสุดบล็อก (finality) [26]

ในการประเมินของงานนี้ โปรแกรม Anchor ถูกรันบนสภาพแวดล้อมทดสอบระดับ SVM (LiteSVM) และ validator แบบโหนดเดียว (localnet) เพื่อตรวจสอบความถูกต้องและวัดต้นทุน compute-unit ของเส้นทาง settlement (ดูหัวข้อ VIII.E) ดังนั้นคุณสมบัติด้านฉันทามติ เช่น การเลือก leader การลงมติของ validator และเวลา finality แบบหลายโหนด จึงเป็นคุณสมบัติของแพลตฟอร์มที่ไม่ได้ถูกวัดในงานนี้ ส่วนการควบคุมสิทธิ์การเข้าร่วมและการกำกับดูแลของ consortium เป็นชั้น governance ระดับโปรแกรมที่แยกต่างหากจากฉันทามติระดับเครือข่าย รายละเอียดอยู่ในหัวข้อ VI.D

E. โครงสร้างบล็อกและธุรกรรมการชำระ

เนื่องจากโปรแกรมทำงานบนเครือข่าย Solana-compatible โครงสร้างบล็อกและส่วนหัวบล็อก (block header) จึงเป็นไปตามนิยามของแพลตฟอร์ม มิได้ถูกปรับแต่งในระดับโปรแกรม แต่ละบล็อกผูกกับช่วงเวลา (slot) ที่มีเป้าหมายประมาณ 400 มิลลิวินาที (ดูหัวข้อ VI.D) ลำดับของธุรกรรมภายในถูกกำหนดด้วยลำดับ Proof of History (PoH) ก่อนการลงมติด้วย Tower BFT ส่วนแต่ละธุรกรรมอ้างอิง blockhash ล่าสุดเพื่อกำหนดอายุและกันการเล่นซ้ำ (replay) ในระดับธุรกรรม โครงสร้างเหล่านี้เป็นคุณสมบัติของแพลตฟอร์ม [26] งานนี้จึงเน้นโครงสร้างของธุรกรรมการชำระที่โปรแกรมออกแบบเอง มากกว่ารูปแบบบล็อกห่วงโซ่บล็อกที่เชื่อมต่อกันด้วย previous blockhash แสดงในรูปที่ 1



รูปที่ 1. โครงสร้างบล็อกบนเครือข่าย Solana-compatible (กำหนดโดยแพลตฟอร์ม ไม่ได้ปรับแต่งในระดับโปรแกรม): ท่วงโซ่บล็อกต่อเนื่อง (slot N-2 ถึง N+2) ที่แต่ละบล็อกอ้างอิงบล็อกก่อนหน้าผ่าน previous blockhash ด้านล่างขยายบล็อก slot N ส่วนหัวบล็อก (block header) บรรจุ blockhash ที่ได้จาก PoH และ parent_slot ขณะที่ธุรกรรมภายใน รวมถึงคำสั่งชำระ settle_offchain_match (ดูหัวข้อ IV.E) ถูกจัดลำดับด้วย Proof of History ก่อนการประกาศ finality ด้วย Tower BFT.

ธุรกรรมการชำระหนึ่งรายการประกอบด้วยคำสั่ง settle หนึ่งคำสั่ง ร่วมกับคำสั่งตรวจสอบลายเซ็น Ed25519 สองคำสั่งต่อหนึ่งคู่จับคู่ (ของผู้ซื้อและผู้ขาย) ที่โปรแกรมตรวจสอบผ่าน instruction sysvar โดย payload ของลายเซ็น (signature, public key และข้อความ canonical รวมรวม 189 ไบต์ต่อคำสั่ง) อยู่ใน ส่วนข้อมูลคำสั่ง (instruction data) มีในบัญชี (accounts) ทำให้ตาราง address lookup table (ALT) ซึ่งบีบอัดเฉพาะรายการบัญชี ไม่สามารถลดขนาดส่วนนี้ได้ ด้วยเหตุนี้โปรแกรมจะอนุญาตการชำระแบบกลุ่มสูงสุด 4 คู่ต่อธุรกรรม (batch settle_offchain_match) แต่เพดานเชิงปฏิบัติถูกจำกัดด้วยขนาดแพ็คเกจธุรกรรม 1,232 ไบต์ คือการรวมสองคู่ (คำสั่งตรวจสอบลายเซ็น 4 คำสั่งรวม 760 ไบต์ ร่วมกับ BatchMatchPair ที่ serialize แล้วรวม 370 ไบต์ และรายการดัชนีบัญชีกับ header) ก็เกินขนาดแพ็คเกจแล้ว การเพิ่มจำนวนคู่ต่อธุรกรรมจริงจึงต้องเปลี่ยนวิธีบรรจุลายเซ็น (เช่น บัญชีลายเซ็นที่ตรวจไว้ล่วงหน้า หรือ multisig แบบรวมนอกเซน) มีโซ่เพียงเพิ่มจำนวนคู่ ส่วนบัญชีที่ธุรกรรมแต่ละต้องประกอบด้วยบัญชี escrow ของผู้ซื้อและผู้ขาย บัญชี order nullifier ทั้งสองฝั่ง บัญชี zone market และบัญชีผู้เก็บค่าธรรมเนียม /wheeling/loss ในคลัง ข้อจำกัดด้านโครงสร้างนี้อธิบายว่าเหตุใดเพดานการรวมกลุ่มและปริมาณงานของการชำระจึงผูกกับรูปแบบธุรกรรม ซึ่งสอดคล้องกับผลการวัดในหัวข้อ VIII.F

เส้นทางการสร้างเหรียญจากพลังงานส่วนเกิน (surplus generation-mint) อยู่ที่ปลายตรงข้ามของสเปกตรัมนี้ เมื่อหน้าต่างการชำระปิดลงโดยมีการผลิตสุทธิเป็นบวก Aggregator Bridge จะสร้างเหรียญส่วนเกินผ่านคำสั่ง mint_generation ของโปรแกรม energy-token ซึ่งมีข้อมูลคำสั่งขนาดคงที่ 40 ไบต์ (discriminator 8 ไบต์, ตัวระบุมาตรฐาน 16 ไบต์, เวลาเริ่มหน้าต่างแบบมิลลิวินาที 8 ไบต์ และจำนวนเหรียญ 8 ไบต์) และไม่มี payload ลายเซ็นฝังในคำสั่งเลย เพราะที่มา (provenance) ถูกยืนยันด้วยผู้ร่วมลงนาม REC validator ที่ลงทะเบียนแล้วในระดับลายเซ็นธุรกรรม ต่างจากการชำระที่ต้องฝังข้อความ Ed25519 ไว้ในข้อมูลคำสั่ง ธุรกรรมที่ Chain Bridge ส่งประกอบด้วยสามคำสั่งระดับบน (compute-budget, การสร้างบัญชี associated token แบบ idempotent และ mint_generation) รวมขนาดรวม 700–750 ไบต์บนสาย (ลายเซ็น 64 ไบต์สองชุด, header กับ blockhash รวม 80 ไบต์, คีย์บัญชี 32 ไบต์รวมสามรายการ และข้อมูลคำสั่งรวม 50 ไบต์) ซึ่งอยู่ต่ำกว่าขนาดแพ็คเกจ 1,232 ไบต์มาก การสร้างเหรียญให้ผู้รับรายเดียวจึงมิได้ถูกจำกัดด้วยทั้งแพ็คเกจและ compute (หัวข้อ VIII.G) และมีพฤติกรรมการขยายขนาดตรงข้ามกับการชำระ กล่าวคือสามารถรวมผู้รับหลายรายไว้ในธุรกรรมเดียว (build_generation_mint_instructions) และเข้าใกล้เพดานแพ็คเกจเมื่อจำนวนผู้รับเพิ่มขึ้น มิใช่ถูกจำกัดที่หนึ่งหน่วยงานต่อธุรกรรม ทั้งนี้สิทธิอนุญาตของการสร้างเหรียญมิได้อยู่ในธุรกรรม แต่อยู่ในคำขอผ่าน NATS ที่นำหน้า โดย aggregator ลงนามบนการ serialize แบบ canonical ที่ติด domain tag และนำหน้าความยาวแต่ละฟิลด์ (correlation/idempotency key, กระเป๋าผู้รับ, ตัวระบุมาตรฐาน, หน้าต่าง และปริมาณพลัง

งาน) ด้วยคีย์ P-256 ที่ Chain Bridge ผูกใบรับรองเข้ากับอัตลักษณ์บริการ (SPIFFE) ของผู้ขอ จึงปฏิเสธอัตลักษณ์ปลอมก่อนสร้างธุรกรรมใด ๆ และการสร้างเหรียญแบบครั้งเดียวพอดี (exactly-once) ถูกบังคับบนเซนด้วยบัญชีบันทึกต่อคู่ (มาตรฐาน, หน้าต่าง) ซึ่งมีธง minted ที่ทำให้การส่งซ้ำกลายเป็น no-op ดังนั้นการชำระที่ลองใหม่หลังระบบล้มจึงไม่อาจสร้างเหรียญซ้ำได้

F. ข้อข้อความ mint ที่ลงนามและการส่งมอบแบบไม่สูญหาย

สิทธิอนุญาต (authorization) ของการสร้างเหรียญส่วนเกินไม่ได้ฝังอยู่ในธุรกรรมบนเซนดังเช่นการชำระ แต่ถูกยกออกไปไว้ในซองข้อความ (envelope) ที่ Aggregator Bridge ส่งให้ Chain Bridge ผ่าน NATS บนหัวข้อ chain.tx.mint แนวคิดคือแยกการพิสูจน์ที่มา (provenance) ออกจากการสร้างธุรกรรม กล่าวคือ Chain Bridge จะยอมประกอบและลงนามธุรกรรม mint_generation ด้วยคีย์ platform_admin ก็ต่อเมื่อซองข้อความผ่านการตรวจสอบอัตลักษณ์ระดับแอปพลิเคชันแล้วเท่านั้น การลงนามบนซองจึงเป็นชั้นควบคุมสิทธิ์ที่ทำงานก่อนบล็อกเซน มิใช่ส่วนหนึ่งของ payload ที่โปรแกรมบนเซนตรวจ

การลงนามกระทำการเข้ารหัสแบบ canonical ที่ออกแบบให้ไม่กำวมและกันการเล่นซ้ำ (replay) โครงร่างประกอบด้วยแท็กโดเมน (domain tag) gridtokenx-nats-envelope-v1 ที่ปิดท้ายด้วยไบนารี ตามด้วยแท็กชนิดข้อความ (mint) เพื่อแยกโดเมนของแต่ละชนิด (signed cancel เล่นซ้ำเป็น submit ไม่ได้) จากนั้นแต่ละฟิลด์ถูกเข้ารหัสเป็น ซีโอฟิลด์ + ไบนารี + ความยาวแบบ u64 little-endian + ค่าไบนารี การนำหน้าความยาวต่อฟิลด์ทำให้ไม่สามารถเลื่อนขอบเขตไบนารีฟิลด์เพื่อเล่นซ้ำลายเซ็นที่จับไว้ได้ ลำดับฟิลด์ถูกกำหนดตายตัว (มิใช่ลำดับตาม JSON) และฟิลด์ auth ที่บรรจุลายเซ็นถูกกันออกจากไบนารีที่ลงนามเสมอเพราะลายเซ็นครอบคลุมตัวเองไม่ได้ โครงสร้างฟิลด์ทั้งหมดของของ mint แสดงในตารางที่ IV โดยฟังก์ชันเข้ารหัสถอดโครงสร้าง (destructure) ทุกฟิลด์อย่างครบถ้วนโดยเจตนา หากมีการเพิ่มฟิลด์ใหม่ในสัปดาห์ใดไม่ระบุว่าให้ลงนามหรือยกเว้น โค้ดจะคอมไพล์ไม่ผ่าน ป้องกันฟิลด์ที่เดินทางบนสายแต่หลุดจากลายเซ็นโดยไม่ตั้งใจ

ตารางที่ IV

โครงสร้างไบนารีที่ลงนามแบบ canonical ของซองข้อความ mint บนหัวข้อ NATS chain.tx.mint (message kind mint) แต่ละแถวคือหนึ่งฟิลด์ที่เข้ารหัสเป็น ๖ + 0x00 + ความยาว u64-LE + ค่า เรียงตามลำดับที่ลงนามจริง ส่วนหน้าด้วย domain tag gridtokenx-nats-envelope-v1\0mint\0 และฟิลด์ auth ถูกกันออกจากไบนารีที่ลงนาม.

ฟิลด์	การเข้ารหัสค่า	ความหมาย
correlation_id	UTF-8	UUID ต่อครั้งที่ส่ง; ใช้กำหนดหัวข้อรับผล
idempotency_key	UTF-8	mint: {serial} : {window} คีย์กันชำระดับผล
reply_subject	UTF-8	หัวข้อ NATS ที่ผู้ส่งรอรับผล
recipient_wallet	UTF-8	กระเป๋าผู้รับ (base58)
service_identity	UTF-8	SPIFFE URI ของบริการผู้ขอ
created_at_ms	u64 LE (8B)	เวลาที่ส่ง (มิลลิวินาที)
energy_kwh	f64 LE (8B)	ปริมาณพลังงาน (kWh)
meter_id	16 ไบต์ดิบ	ตัวระบุมาตรวัด (UUID)
window_start_ms	i64 LE (8B)	เวลาเริ่มหน้าต่าง 15 นาที

ไบนารี canonical ข้างต้นเป็นเพียง อินพุตของลายเซ็น เท่านั้น สิ่งเดินทางบนสาย NATS จริงคือซอง JSON (MintEnergyMessage) ที่บรรจุฟิลด์เหล่านั้นพร้อมฟิลด์ auth แนบท้าย โดย auth (EnvelopeAuth) ประกอบด้วยแท็กสคีม ecdsa-p256-sha256-v1 ใบรับรอง mTLS ปลายทาง (leaf cert PEM) และลายเซ็น ECDSA P-256/SHA-256 ที่เข้ารหัส ASN.1-DER แล้ว base64 ผู้ลงนามคือ aggregator ที่ถือคีย์ไคลเอนต์ mTLS เดียวกันที่ใช้เป็นขอบเขตความเชื่อถือของ gRPC เมื่อไม่มีคีย์ใบรับรอง/คีย์ (โหมด dev แบบไม่ปลอดภัย) ของจะถูกส่งแบบไม่ลงนามและถูกยอมรับเฉพาะขณะที่ Chain Bridge เดินในโหมดบันทึกอย่างเดียว (log-only)

ฝั่ง Chain Bridge ตรวจสอบขั้นตามลำดับก่อนสร้างธุรกรรม คือ (1) ใบรับรองในซองต่อสายไปยัง CA ที่เชื่อถือ (2) ค่า SPIFFE SAN ในใบรับรองต้องเท่ากับฟิลด์ service_identity ที่ของประกาศ ผูกค่ากล่าวอ้างอัตลักษณ์ที่ประกาศเองเข้ากับใบรับรองที่ CA ออก และ (3) ตรวจสอบลายเซ็น P-256 เทียบกับไบนารี canonical ที่ คำนวณใหม่ จากฟิลด์ที่รับมา ดังนั้นการคัดแปลงฟิลด์ใด ๆ ระหว่างทาง (เช่น เปลี่ยนกระเป๋าผู้รับหรือปริมาณพลังงาน) จะทำให้ไบนารี

ที่คำนวณใหม่ต่างออกไปและลายเซ็นตรงไม่ผ่าน การบังคับตรวจสอบลายเซ็นเปิดใช้ด้วยธง `CHAIN_BRIDGE_REQUIRE_SIGNED_NATS` มิฉะนั้นทำงานแบบ log-only เพื่อการย้ายระบบแบบค่อยเป็นค่อยไป จุดสำคัญเชิงวิศวกรรมคือฝั่งผู้ลงนามและฝั่งผู้ตรวจใช้การเข้ารหัส canonical ชุดเดียวกันจากไครเอตที่แบ่งใช้ร่วม (*gridtokenx-blockchain-types*) มิใช่การคัดลอกโครงไบต์แยกกัน จึงมีการทดสอบ golden-vector ที่ประกอบไบต์ที่คาดหวังขึ้นใหม่โดยอิสระเพื่อจับความคลาดเคลื่อนของสคิมาข้ามไครเอต

การส่งมอบของออกแบบให้ไม่สูญหายแม้เมื่อบล็อกเชนหรือ validator ล่มชั่วคราว ของถูก publish เข้าสู่ JetStream และรอรับ PubAck (การเก็บลงสตรีมแบบทน) ก่อนรอผลลัพธ์ ปิดช่องที่ผู้บริโภควางหายไปชั่วคราว นอกจากนี้ส่วนเกินที่ชำระออกจากหน้าต่างแล้วแต่ยังลงเซนไม่ได้จะถูก เข้าคิว (enqueue) ลง durable outbox บน Redis (*gridtokenx:billing:mint_outbox* พิลด์ `{serial}:{window}`) และมี drain loop คอยลองซ้ำจนกว่าการสร้างเหรียญจะได้รับการยืนยันบนเซนจึงลบออก คิวนี้อยู่รอดการรีสตาร์ท (โหลดกลับจาก Redis) และกระเปาะผู้รับถูก resolve ใหม่ทุกครั้งที่ลอง ทำให้มาตรวัดที่ลงทะเบียนกระเปาะ ภายหลัง หน้าต่างของตนยังได้รับเหรียญในการลองครั้งถัดไป การลองซ้ำปลอดภัยเพราะ Chain Bridge กันซ้ำด้วย `mint:{serial}:{window_start_ms}` ประกอบกับบัญชี PDA (มาตราวัด, หน้าต่าง) บนเซนเป็นด่านสุดท้าย การลองซ้ำของเหรียญที่ลงเซนไปแล้วแต่ผลตอบกลับหาย จึงไม่สร้างเหรียญซ้ำ รายการที่ยังลงเซนไม่ได้เกินกำหนดเวลา (*MINT_OUTBOX_MAX_AGE_SECS* ค่าเริ่มต้น 7 วัน) จะถูก พัก (park) ออกจากเส้นทางลงซ้ำแทนการลบทิ้งเฉียบ ๆ เพื่อให้ผู้ดูแลนักกลับเข้าคิวได้ ส่วนคำขอแบบกลุ่ม (batch) บนหัวข้อ *chain.tx.mintbatch* ใช้โครงสร้างเดียวกันแต่ผนวกฟิลด์จำนวนรายการ (*item_count*) และห่อแต่ละรายการด้วยกรอบความยาวของตนเอง จึงผูกทั้ง จำนวน และ ลำดับ ผู้รับเข้ากับลายเซ็น การเพิ่ม ลด หรือ สลับลำดับผู้รับจะทำให้ลายเซ็นตรงไม่ผ่าน

มีฟิลด์หนึ่งของช่องที่ข้ามขอบเขตการแทนค่า (representation) ซึ่งควรระบุให้ชัด เพราะเป็นจุดที่แยกค่าอ่านที่สุจริตออกจากค่าที่ประสงค์ร้ายก่อนสร้างเหรียญ ปริมาณพลังงานเดินทางเป็นเลขทศนิยม *f64* หน่วย kWh แต่การสร้างเหรียญบนเซนทำงานด้วยจำนวนเต็มหน่วยฐาน (base unit) ที่ 9 ตำแหน่งทศนิยม (1 kWh = 10^9 หน่วยฐาน ตามความละเอียดของ energy token) การแปลงค่าใช้ฟังก์ชันร่วมเพียงตัวเดียวทั้งฝั่งผู้ผลิต (aggregator) และฝั่งผู้ตรวจ (Chain Bridge) ตามหลักแหล่งเดียว (single source) เช่นเดียวกับการเข้ารหัส canonical จึงไม่มีทางที่ทั้งสองฝั่งจะเห็นขอบเขตหรือการสเกลต่างกัน ฟังก์ชันนี้ปฏิเสธค่าที่ไม่ควรถึง *mint_to* ได้แก่ ค่าที่ไม่จำกัด (*NaN/±∞*) ค่าที่ไม่เป็นบวก ค่าที่เกินเพดาน *MAX_MINT_KWH* ที่ 10^6 kWh (1 GWh สูงกว่าหน้าต่าง 15 นาทีที่สุจริตมากแต่ยังต่ำกว่าจุดอื่น) และค่าบวกที่ตัดเศษแล้วเหลือศูนย์หน่วยฐาน เพดานนี้เป็นกลไกสำคัญมิใช่เพียงประดับ เพราะการแปลงเลขทศนิยมเป็นจำนวนเต็มในภาษา Rust จะอิ่มตัว (saturate) เป็น *u64::MAX* เมื่อคำนวณหาผลหารวนกลับ (wrap) หากไม่มีเพดานนี้ของเดียวอาจร้องขอการสร้างเหรียญมหาศาลได้ เพดานจึงบีบค่าที่สเกลแล้วให้อยู่ราว 10^{15} ซึ่งอยู่ในช่วง *u64* อย่างปลอดภัย จากนั้นโปรแกรมบนเซนตรวจซ้ำอีกชั้นเป็นการป้องกันเชิงลึก (defense in depth) โดยปฏิเสธจำนวนศูนย์ (*ZeroAmount* รหัสข้อผิดพลาด 6011) ก่อนการเขียนสถานะใด ๆ เพราะ *mint_to(, 0)* เป็น no-op เฉียบ ๆ ที่ยังคงติดธง *minted = true* ให้หน้าต่างนั้นและทำให้หน้าต่างเสียถาวร ร่วมกับการตรวจขอบหน้าต่าง 15 นาทีในหน่วยมิลลิวินาที (*MisalignedWindow* รหัส 6010) และเงื่อนไขที่ผู้ร่วมลงนาม REC validator ต้องต่างจากผู้มีอำนาจแพลตฟอร์ม (*RecValidatorIsAuthority* รหัส 6012) ทั้งนี้บัญชีบันทึกแบบครั้งเดียวพอที่ถูกตรวจ ก่อน เสมอ การส่งซ้ำจึงลัดเป็น no-op ก่อนที่ด้านเหล่านี้จะทำงาน

V. PRICING AND MARKET MECHANISM

A. Pricing and Settlement Model

แบบจำลองราคาของระบบแบ่งเป็นสามส่วน คือ การกำหนดราคาเคลียร์ในกลไก CDA การคำนวณค่าธรรมเนียมและยอดสุทธิในการ settlement และ

กลไกราคาของโทเคนในชั้น treasury สัญลักษณ์ที่ใช้ในสมการสรุปไว้ในตารางที่ V

ตารางที่ V Nomenclature for the pricing and settlement equations.	
สัญลักษณ์	ความหมาย
p_s	ราคาเสนอขายต่อหน่วย (sell ask)
p_b	ราคาเสนอซื้อต่อหน่วย (buy bid)
p^*	ราคาเคลียร์ / landed cost
λ	loss factor ($\lambda \geq 1$)
c_{loss}	ต้นทุนการสูญเสียต่อหน่วย
w	ค่าผ่านสาย (wheeling charge) ต่อหน่วย
m	ตัวคูณจูงใจ (incentive multiplier)
δ	ส่วนลดภายในโซน (intra-zone discount)
q	ปริมาณพลังงานที่จับคู่ (kWh)
V	มูลค่ารวมของธุรกรรม
f	ค่าธรรมเนียมตลาด
φ	อัตราค่าธรรมเนียมตลาด (bps)
W	ค่าผ่านสายรวม ($q \cdot w$)
L	ต้นทุนการสูญเสียรวม ($q \cdot c_{\text{loss}}$)
r	อัตราแลกเปลี่ยน GRX atom ต่อ THBC
ψ	ค่าธรรมเนียม swap (bps)
A	ตัวสะสมรางวัลต่อหน่วย stake
s, s_{total}	จำนวนที่ stake และจำนวน stake รวม
R	รางวัลที่เติมเข้าระบบ

การกำหนดราคาเคลียร์ (CDA clearing) ใช้ราคาฝั่งผู้ขาย (maker price) ปรับด้วยต้นทุนโครงข่าย โดยกำหนดต้นทุนการสูญเสีย (loss cost) ต่อหน่วยจากค่า loss factor $\lambda \geq 1$ ดังสมการ

$$c_{\text{loss}} = p_s(\lambda - 1) \quad (2)$$

ราคาเคลียร์หรือ landed cost คำนวณจากราคาเสนอขาย p_s บวกค่าผ่านสาย (wheeling charge) w และต้นทุนการสูญเสีย แล้วปรับด้วยตัวคูณจูงใจ (incentive multiplier) m และส่วนลดภายในโซน (intra-zone discount) δ

$$p^* = (p_s + w + c_{\text{loss}}) \cdot m \cdot \delta \quad (3)$$

โดย $\delta = 0.95$ เมื่อผู้ซื้อและผู้ขายอยู่โซนเดียวกัน และ $\delta = 1$ เมื่อข้ามโซน คำสั่งซื้อจะจับคู่ได้เมื่อ $p^* \leq p_b$ (เมื่อ p_b คือราคาเสนอซื้อ) และระบบจัดลำดับผู้ขายที่มี landed cost ต่ำสุดให้จับคู่ก่อนตามหลัก price-time priority ปริมาณที่จับคู่เท่ากับส่วนที่เหลือน้อยที่สุดของทั้งสองฝั่ง $q = \min(q_b, q_s)$

การคำนวณค่าธรรมเนียมและยอดสุทธิในการ settlement กำหนดมูลค่ารวม ค่าธรรมเนียมตลาด และยอดสุทธิที่ผู้ขายได้รับดังนี้

$$V = q \cdot p^* \quad (4.1)$$

$$f = \frac{V \cdot \varphi}{10000} \quad (4.2)$$

$$\text{net} = V - f - W - L \quad (4.3)$$

โดย φ คือค่าธรรมเนียมตลาดในหน่วย basis point (ค่าตั้งต้นบนเซน 25 bps หรือ 0.25%), $W = q \cdot w$ คือค่าผ่านสายรวม และ $L = q \cdot c_{\text{loss}}$ คือต้นทุนการสูญเสียรวม ยอด f, W, L และ net ถูกโอนแยกไปยังบัญชีผู้เก็บค่าธรรมเนียม ค่าผ่านสาย การสูญเสีย และผู้ขายตามลำดับ พารามิเตอร์ของโซนถูกตีความในหน่วย bps คือ $m = m_{\text{bps}}/10000$ และ $w = w_{\text{bps}}/10000$ โดยค่าตั้งต้นของ wheeling charge เท่ากับ 0 ภายในโซนและ 0.02 ข้ามโซน ส่วน loss factor เท่ากับ 1.01 ภายในโซนและ 1.03 ข้ามโซน

เพื่อให้เห็นภาพการใช้สมการข้างต้น พิจารณาตัวอย่างการจับคู่ภายในโซนเดียวกัน โดยกำหนดราคาเสนอขาย $p_s = 4.00$ บาทต่อ kWh ปริมาณ $q = 10$ kWh และใช้ค่าตั้งต้นภายในโซน ($\lambda = 1.01, w = 0, m = 1, \delta = 0.95, \varphi = 25$ bps) จาก Equation 2 ได้ $c_{\text{loss}} = 4.00(1.01 -$

1) = 0.04 และจาก Equation 3 ได้ราคาเคลียร์ $p^* = (4.00 + 0 + 0.04) \cdot 1 \cdot 0.95 = 3.838$ บาทต่อ kWh ซึ่งจับคู่ได้เมื่อราคาเสนอซื้อ $p_b \geq 3.838$ จากนั้นมูลค่ารวม $V = 10 \cdot 3.838 = 38.38$ บาท ค่าธรรมเนียม $f = 38.38 \cdot 25/10000 \approx 0.096$ บาท ค่าผ่านสายรวม $W = 0$ และต้นทุนการสูญเสียรวม $L = 10 \cdot 0.04 = 0.40$ บาท ทำให้ยอดสุทธิที่ผู้ขายได้รับเท่ากับ $\text{net} = 38.38 - 0.096 - 0 - 0.40 \approx 37.88$ บาท ทั้งนี้การคำนวณจริงในโค้ดใช้เลขทศนิยมตายตัวแบบปัดลง ค่าที่ได้จึงอาจต่างจากตัวอย่างนี้ในระดับเศษปัด

กลไกราคาของโทเคนในชั้น treasury ครอบคลุมการแลกเปลี่ยนระหว่าง GRX และ THBC stablecoin ที่ตรึงค่ากับเงินบาท โดยใช้อัตรา r (จำนวน GRX atom ต่อ THBC) และค่าธรรมเนียม swap ψ (bps)

$$\text{thbc} = \frac{g \cdot r}{10^9} \cdot (1 - \psi/10000) \quad (5.1)$$

$$g = \frac{\text{thbc} \cdot 10^9}{r} \quad (5.2)$$

โดยการ redeem ตาม Equation 5.2 ไม่มีค่าธรรมเนียม และการรักษาค่าตรึงใช้เงื่อนไข supply ต่อ reserve แบบ 1:1 คือ $\text{supply}_{\text{thbc}} \leq \text{reserve}_{\text{attested}}$ ส่วนรางวัลการ stake ใช้ตัวสะสม (accumulator) แบบ MasterChef โดยรางวัลค้างรับของผู้ stake คำนวณจากจำนวนที่ stake s

$$\text{reward} = s \cdot A/10^{12} - \text{debt} \quad (6)$$

เมื่อมีการเติมรางวัล R ตัวสะสม A จะถูกปรับเป็น $A \leftarrow A + (R \cdot 10^{12})/s_{\text{total}}$ แบบ pro-rata ตามสัดส่วนการ stake และการ slash จะหักจำนวนที่ร้องขอแต่ไม่เกินเงินต้นที่ stake ไว้ (capped ที่ principal) แล้วกระจายคืนสู่ผู้ stake ที่เหลือผ่านตัวสะสมเดียวกัน

ในเส้นทาง CDA ที่ใช้งานจริง ตัวคูณจุดทศนิยมตั้งเป็น $m = 1$ กล่าวคือ incentive multiplier มีผลเฉพาะเส้นทาง settlement แบบ feed-in หรือ grid-export มิใช่การจับคู่ CDA นอกจากนี้ค่าตั้งต้นของค่าธรรมเนียมตลาดบนเซน (25 bps) แตกต่างจากค่าตั้งต้นในไฟล์ตั้งค่านอกเซน (50 bps) และพารามิเตอร์ในโปรแกรม governance จัดเก็บแบบสเกล $\times 1000$ ขณะที่ผู้บริโภคค่าจริงในชั้น trading คือความแบบ bps ($/10000$) ซึ่งเป็นค่าที่ระบบใช้งานจริง ทั้งนี้การคำนวณในโค้ดใช้เลขทศนิยมตายตัว (fixed-point integer) แบบปัดลง (floor division) และยอดสุทธิของผู้ขายใช้ checked arithmetic ที่ปฏิเสธธุรกรรมเมื่อค่าธรรมเนียมรวมเกินมูลค่าจับคู่ (พร้อมเพดานค่าธรรมเนียมเครือข่าย 20%) แทนการปัดยอดลงเป็นศูนย์ ดังนั้นสมการข้างต้นจึงเป็นแบบจำลองเชิงค่าจริงที่อาจต่างจากผลคำนวณจริงในระดับเศษปัด

B. การวิเคราะห์ความไวของยอดสุทธิและส่วนเพิ่มจากการซื้อขายแบบ P2P

เพื่อแสดงนัยเชิงเศรษฐศาสตร์ของแบบจำลองราคาข้างต้น ส่วนนี้วิเคราะห์ความไว (sensitivity) ของยอดสุทธิที่ผู้ขายได้รับต่อพารามิเตอร์ของโซนและค่าธรรมเนียม โดยเป็นการคำนวณเชิงแบบจำลองล้วน (model-derived) จากสมการ settlement ตาม Equation 3 และ Equation 4.3 มิใช่การวัดรายได้จากการรันระบบจริง กำหนดราคาเสนอขายฐาน $p_s = 4.00$ บาทต่อ kWh ปริมาณจับคู่ $q = 10$ kWh และตัวคูณจุดทศนิยม $m = 1$ (ตามเส้นทาง CDA จริง) คงที่ แล้วแปรค่าสถานะภายในโซน/ข้ามโซน อัตราค่าธรรมเนียม φ และ loss factor λ ตามตารางที่ VI

ตารางที่ VI

Model-derived seller-net sensitivity computed from Equation 3 and Equation 4.3 at

$p_s = 4.00$ B/kWh, $q = 10$ kWh, $m = 1$. “net” is the seller’s settled amount; wheeling (w) and loss (λ) are pass-through to the buyer via the landed price p^* .

สถานการณ์	δ	w	λ	φ (bps)	p^*	net (฿)	net/kWh
S1 ภายในโซน	0.95	0	1.01	25	3.838	37.88	3.79
S2 ข้ามโซน	1	0.02	1.03	25	4.14	39.9	3.99
S3 ภายในโซน, ค่าธรรมเนียมสูง	0.95	0	1.01	100	3.838	37.6	3.76
S4 ข้ามโซน, loss สูง	1	0.02	1.05	25	4.22	39.89	3.99

จากตารางที่ VI เห็นรูปแบบสำคัญสองประการ ประการแรก ยอดสุทธิของผู้ขายแทบไม่เปลี่ยนเมื่อเพิ่มค่าผ่านสาย w หรือ loss factor λ (เทียบ S2 กับ S4 ที่ต่างกันในระดับเศษสตางค์) เพราะต้นทุนทั้งสองถูกบวกเข้าไปในราคา landed p^* ที่ผู้ซื้อจ่าย แล้วถูกแยกออกไปยังบัญชีผู้เก็บค่าผ่านสายและการสูญเสีย จึงเป็นต้นทุนแบบส่งผ่าน (pass-through) ที่ไม่ลดทอนยอดผู้ขายโดยตรง ผู้ขายจึงได้รับยอดใกล้เคียง $q \cdot p_s$ เสมอ ประการที่สอง ตัวแปรที่กระทบยอดผู้ขายอย่างมีนัยคือส่วนลดภายในโซน δ และอัตราค่าธรรมเนียม φ โดยการจับคู่ภายในโซน ($\delta = 0.95$) ลดยอดผู้ขายลงประมาณ 5% เพื่อจูงใจการบริโภคในพื้นที่ ขณะที่การขึ้นค่าธรรมเนียมจาก 25 เป็น 100 bps (S1 $\rightarrow$ S3) ลดยอดสุทธิเพียงเล็กน้อย

เมื่อเทียบกับการรับซื้อไฟส่วนเกินแบบอัตราคงที่ (flat feed-in tariff) ของโครงการโซลาร์ภาคประชาชนที่สมมติไว้ราว 2.20 บาทต่อ kWh ยอดสุทธิต่อหน่วยของผู้ขายในตลาด P2P (ประมาณ 3.76–3.99 บาทต่อ kWh) คิดเป็นส่วนเพิ่ม (uplift) ประมาณ 72–81% ขณะเดียวกันผู้ซื้อยังจ่ายราคา landed p^* (ประมาณ 3.84–4.22 บาทต่อ kWh) ต่ำกว่าค่าไฟขายปลีกตามปกติ จึงเกิดส่วนเกินจากการซื้อขาย (gains from trade) ทั้งสองฝั่ง ทั้งนี้อัตรารับซื้อ 2.20 บาทเป็นเพียงค่าอ้างอิงเชิงสมมติเพื่อการเปรียบเทียบ มิใช่ค่าที่วัดจากตลาดจริง

ข้อจำกัดของการวิเคราะห์นี้คือ ตัวเลขในตารางที่ VI เป็นผลเชิงแบบจำลองภายใต้พารามิเตอร์คงที่และปริมาณจับคู่เดียว (10 kWh) อีกทั้งการคำนวณจริงในโค้ดใช้เลขทศนิยมตายตัวแบบปัดลง ค่าที่ได้จึงอาจต่างในระดับเศษปัด และยังไม่ใช้การวัดการกระจายของรายได้ (revenue distribution) บนภาระงานจำลองเต็มรูปแบบ ซึ่งเปิดไว้เป็นงานในอนาคต (ดูหัวข้อ IX)

C. Continuous Double Auction Matching

กลไกตลาดใช้การจับคู่แบบ Continuous Double Auction (CDA) ที่จัดลำดับตามหลัก price-time priority และคำนึงถึงข้อจำกัดเชิงโทโพโลยีของโครงข่ายไฟฟ้า คำสั่งขายถูกแบ่งสมุดคำสั่ง (order book) ตามโซนแบบ zone-segmented โดยแต่ละโซนเก็บคำสั่งขายในโครงสร้างเรียงลำดับ (BTreeMap) ด้วยกุญแจ (*price, created_at, id*) ทำให้ลำดับของกุญแจให้ความสำคัญกับราคาต่ำสุดก่อน แล้วจึงเป็นเวลาสร้างคำสั่งที่เก่ากว่า และใช้ order id เป็นตัวตัดสินขั้นสุดท้าย จึงได้ price-time priority โดยปริยายโดยไม่ต้องเรียงลำดับซ้ำ คำสั่งที่เหลือปริมาณต่ำกว่าเกณฑ์ขั้นต่ำ (MIN_TRADE_AMOUNT) หรือหมดอายุแล้วจะไม่ถูกใส่ในสมุด

สำหรับคำสั่งซื้อแต่ละรายการ เครื่องจับคู่รวบรวมผู้ขายที่เป็นผู้สมัคร (candidate) จากเฉพาะโซนที่โครงข่ายสามารถส่งพลังงานถึงโซนของผู้ซื้อได้ผ่านการตรวจ topology pre-filtering สองชั้น คือชั้นแรกตรวจที่ปริมาณขั้นต่ำเพื่อตัดโซนที่ส่งถึงกันไม่ได้ออกทันที และชั้นที่สองตรวจซ้ำที่ปริมาณจับคู่จริง (*can_accommodate_flow(sell_zone, buy_zone, amount)*) เพื่อบังคับเขตแดนความจุสายส่ง ภายในแต่ละโซนที่เข้าถึงได้ ใช้การสืบค้นช่วง (range query) ดึงเฉพาะคำสั่งขายที่ราคาเสนอไม่เกินราคาเสนอซื้อ แล้วคำนวณ landed cost ตาม Equation 3 (รวม wheeling, ต้นทุน loss, ตัวคูณ และส่วนลดภายในโซน $\delta = 0.95$) คำสั่งขายที่มี landed cost ไม่เกินราคาเสนอซื้อ ($p^* \leq p_b$) เท่านั้นที่ผ่านเข้าเป็น candidate ระบบป้องกันการจับคู่กับตนเอง (self-trade) โดยข้ามคู่ที่ผู้ซื้อและผู้ขายเป็นผู้ขายเดียวกัน

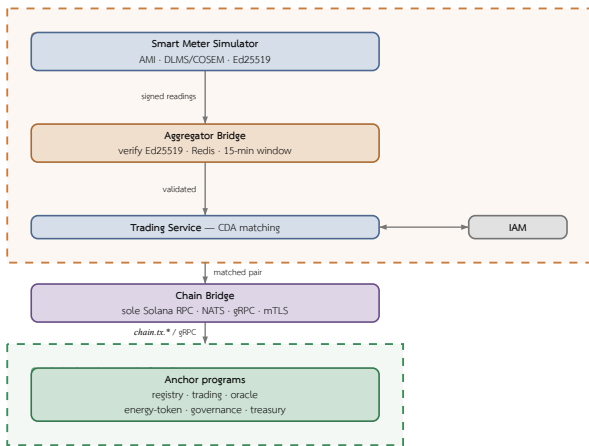
เมื่อได้ candidate จากทุกโซนที่เข้าถึงได้ ระบบรวบรวมรายการแล้วเรียงตาม landed cost จากต่ำไปสูง เพื่อให้ผู้ซื้อได้ราคารวมส่งถึง (landed) ที่ถูกที่สุดก่อน ไม่ว่าคำสั่งขายนั้นจะอยู่ในโซนใด จากนั้นทยอยจับคู่โดยปริมาณต่อคู่เท่ากับส่วนเหลือที่น้อยกว่าของสองฝั่ง ($q = \min(q_b, q_s)$) แบบ partial-fill และตัดคำสั่งขายที่ปริมาณเหลือต่ำกว่าเกณฑ์ออกจากสมุดทันที กรณีคำสั่งซื้อแบบ Fill-or-Kill (FOK) ระบบจะตรวจก่อนว่าปริมาณรวมของ candidate เพียงพอต่อทั้งคำสั่ง หากไม่พอจะไม่จับคู่เลย นอกจากนี้มีการรวมผลลัพธ์ (match

consolidation) เมื่อคู่ผู้ซื้อ-ผู้ขายและราคาเดียวกันต่อเนื่องกัน เพื่อลดจำนวนรายการ settlement ที่ต้องส่งขึ้นเซ่น ผลการจับคู่แต่ละรายการบันทึก match price (landed cost), wheeling charge, loss cost และโซนต้นทุน/ปลายทาง ก่อนส่งคู่ที่จับแล้วไปชำระแบบ atomic ตามหัวข้อ IV ต้นทุนการประมวลผลบนเซ่นของเส้นทาง settlement ที่เครื่องจับคู่ป้อนเข้ารายการในหัวข้อ VIII.E ส่วนอัตราการจับคู่ (matching throughput) ของเครื่องจับคู่ในชั้นหน่วยความจำรายการในหัวข้อ VIII.D

VI. SYSTEM DESIGN AND IMPLEMENTATION

A. Architecture Overview

สถาปัตยกรรมของระบบถูกออกแบบเพื่อรองรับการจำลองการซื้อขายพลังงานแบบ Peer-to-Peer ในบทความนี้นำเสนอแบบจำลองไมโครกริด โดยแบ่งเส้นทางข้อมูลออกเป็นชั้น Smart Meter, Aggregator Bridge, Trading Service, Anchor Smart Contract Program, Settlement Engine และ Frontend Dashboard โดยข้อมูลการผลิตและการใช้พลังงานถูกสร้างจาก Smart Meter Simulator แล้วส่งเข้าสู่ Aggregator Bridge เพื่อคัดกรองข้อมูลก่อนแปลงเป็นคำสั่งธุรกรรมสำหรับ Anchor program บนเครือข่าย Solana-compatible แบบ permissioned การแบ่งระบบออกเป็นโดเมนนอกเซ่นและบนเซ่น พร้อมเส้นทางข้อมูลหลักและจุดข้ามขอบเขตความเชื่อถือ สรุปไว้ในรูปที่ 2



รูปที่ 2. System architecture and the off-chain/on-chain trust boundary. The off-chain domain (orange, dashed) verifies Ed25519-signed meter telemetry, aggregates it, and matches orders via CDA; the on-chain domain (green, dashed) settles and audits the verified pairs. Chain Bridge is the sole crossing between the two — the only service touching Solana RPC, writing via NATS *chain.tx.** and reading via gRPC over mTLS.

การแยกองค์ประกอบในลักษณะนี้ทำให้การตรวจสอบเชิงวิศวกรรมไฟฟ้า และการควบคุมสิทธิ์ที่เกิดขึ้นก่อนบันทึกธุรกรรม ขณะที่ Anchor program ทำหน้าที่เป็นชั้นบังคับใช้กติกาที่ตรวจสอบได้และบันทึกสถานะเพื่อการตรวจสอบย้อนหลัง ส่วน Settlement Engine ทำหน้าที่จัดการธุรกรรมที่ลงนามแล้วติดตามผลจาก transaction signature และอ่าน Event log เพื่อนำสถานะกลับไปแสดงบน Frontend

B. Backend Services

ชั้น Backend ทำหน้าที่เป็นชั้นกลางระหว่าง Smart Meter Simulator (ดูหัวข้อ VI.C), Aggregator Bridge และ Smart Contract โดยรับผิดชอบการรวบรวมข้อมูล การตรวจสอบความถูกต้อง การจัดคิวธุรกรรม และการส่งคำสั่งไปยังบล็อกเชนหลังจากข้อมูลผ่านเงื่อนไขด้าน Grid stability แล้วเท่านั้น การออกแบบใช้แนวคิด Microservice เพื่อแยกภาระงานออกเป็นภาระงานย่อย ทำให้ขยายระบบเฉพาะส่วนที่มีปริมาณคำขอสูงและลดผลกระทบเมื่อบริการใดบริการหนึ่งเกิดความผิดพลาด รายละเอียดของช่องทางสื่อสารระหว่างบริการ (ConnectRPC บน mTLS และ NATS JetStream) และสมมติฐาน

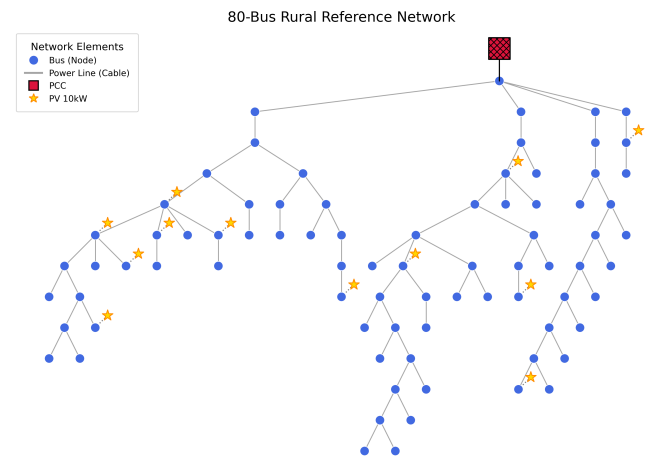
ความเชื่อถือที่เกี่ยวข้องอธิบายไว้ในหัวข้อ III องค์ประกอบหลักของชั้น Backend ประกอบด้วย

- IAM Service: จัดการตัวตน สิทธิ์การเข้าถึงข้อมูล On-chain และบทบาทของผู้ใช้งาน เช่น Prosumer, Consumer และ Operator
- Aggregator Bridge: รับข้อมูลการผลิตและการใช้ไฟฟ้าจาก Smart Meter ตรวจสอบรูปแบบข้อมูล เวลาอ้างอิง รหัสอุปกรณ์ และค่าพลังงานก่อนส่งต่อไปยังขั้นตอนวิเคราะห์
- Trading Service: ตรวจสอบและจับคู่คำสั่งซื้อขายพลังงานร่วมกับเงื่อนไขด้าน Grid stability เช่น ปริมาณพลังงานคงเหลือ ข้อจำกัดของโครงข่าย และสถานะการเชื่อมต่อของ Smart Meter
- Chain Bridge: ตัวกลางเดียวที่ส่งคำสั่งไปยัง Solana Program และติดตามผลลัพธ์จาก transaction signature หรือ Event log โดยบริการอื่นไม่เรียก Solana RPC โดยตรง
- Notification Service: ส่งการแจ้งเตือน เช่น Email และ Alert เมื่อคำสั่งซื้อขายหรือสถานะ settlement เปลี่ยนแปลง

การแยกบริการในลักษณะนี้ช่วยให้ระบบรองรับข้อมูล Realtime Smart Meter จำนวนมากได้ดีขึ้น เนื่องจากบริการรับข้อมูลขยายจำนวน instance ได้โดยไม่กระทบต่อการตรวจสอบธุรกรรมหรือบริการเชื่อมต่อบล็อกเชน นอกจากนี้ ชั้น Backend ยังเป็นจุดควบคุมความปลอดภัยก่อนบันทึกธุรกรรมลง Smart Contract ทำให้ระบบไม่พึ่งพาล็อกเชนเพียงอย่างเดียว แต่ผสานการตรวจสอบทางวิศวกรรมไฟฟ้าและการควบคุมสิทธิ์ของผู้ใช้งานเข้าด้วยกัน

C. Grid and Meter Simulation

Grid Modeling ออกแบบมาเพื่อจำลองระบบไฟฟ้าที่ซับซ้อน แหล่งผลิตพลังงานแบบกระจาย (Distributed Energy Resources: DERs) และการดำเนินการของ Virtual Power Plant (VPP) จึงช่วยให้การจำลอง Advanced Metering Infrastructure (AMI) และการจัดการโครงข่ายไฟฟ้ามีความแม่นยำสูง แกนหลักของระบบจำลองผสานการประเมินสถานะแบบเวลาจริงที่ผ่านการตรวจสอบทางฟิสิกส์ (Physics-validated State Estimation) และ Optimal Power Flow (OPF) โดยใช้ Pandapower ซึ่งทำให้ระบบสามารถสร้างข้อมูลมาตรวัดแบบ Deterministic สำหรับโครงข่ายไฟฟ้าขนาดใหญ่



รูปที่ 3. Grid bus network topology used for simulator.

Grid Simulator พัฒนาด้วย Python 3.11 [28] สำหรับจำลองพฤติกรรมของการใช้ไฟฟ้า Distribution Grid ใช้รูปแบบ Network Topology จากไฟล์ GridLAB-D GLM [29] แล้วแปลงเป็น grid modeling ซึ่งประกอบด้วย bus, line, load และ photovoltaic unit ดังแสดงในรูปที่ 3 เครื่องมือหลักที่ใช้ ได้แก่ FastAPI [30], NetworkX [31] สำหรับแทนโครงสร้าง topology, pvlib [32] สำหรับจำลอง photovoltaic generation และ pandapower [33] สำหรับคำนวณ power flow

กระบวนการจำลองทำงานแบบ discrete time-step โดยในแต่ละรอบประกอบด้วยสองขั้นตอนคือ การสร้างค่าอ่านที่ระดับมาตรวัด (ดูหัวข้อ VI.C.1) แล้วตามด้วยการแก้สถานะโครงข่ายที่ระดับ grid (ดูหัวข้อ VI.C.2) นอกจาก

ค่าอ่านสังเคราะห์ ระบบยังสามารถ replay ข้อมูล telemetry จริงจากไฟล์ CSV และเชื่อมโยงข้อมูลกับ bus ผ่าน meter registry ทำให้รองรับ hybrid simulation ระหว่าง measured data และ synthetic data ได้ ความถูกต้องของ simulator ตรวจสอบผ่านชุดทดสอบที่ครอบคลุม topology loading และ meter-to-bus mapping

1) Smart Meter Simulator

ในแต่ละ bus ของ topology ระบบสร้างประชากรมาตรวัด (meter population) ตามสัดส่วนชนิดของผู้ใช้ โดยมาตรวัดแต่ละตัว (SmartMeter) ประกอบขึ้นจากแบบจำลองอุปกรณ์ย่อยแบบแยกส่วน ได้แก่ โหลด (Load), แผงเซลล์แสงอาทิตย์ (Solar) เมื่อมีการติดตั้ง และระบบกักเก็บพลังงานแบตเตอรี่ (BESS) แบบเลือกได้ พฤติกรรมการใช้ไฟฟ้าจำลองด้วยแบบจำลองโหลดแบบ ZIP ที่ตอบสนองต่อแรงดัน (voltage-dependent ZIP load) ซึ่งกำลังไฟฟ้าจริงที่ดึงจากโครงข่ายขึ้นกับแรงดันต่อหน่วย (per-unit voltage) ตามสมการ

$$P(V) = P_{\text{base}}(Z \cdot V^2 + I \cdot V + P), \quad Z + I + P = 1 \quad (7)$$

โดยองค์ประกอบ Z (impedance) แปรผันตาม V^2 , องค์ประกอบ I (current) แปรผันตาม V และองค์ประกอบ P (power) คงที่ มาตรวัดในการประเมินนี้ตั้งสัดส่วน impedance ไว้ที่ 0.20 (ZIP_IMPEDANCE_FRACTION=0.20) และตั้งแรงดันให้อยู่ในช่วง $[0, 1.5]$ ต่อหน่วยก่อนคำนวณ ส่วนกำลังผลิตจากแผงเซลล์แสงอาทิตย์คำนวณด้วย pvlib [32] โดยใช้แบบจำลองท้องฟ้าใส (clear-sky) แบบ Ineichen ร่วมกับแบบจำลอง PVWatts สำหรับกำลังไฟฟ้ากระแสตรง แล้วลดทอนด้วยตัวประกอบสภาพอากาศ (weather derate factor) เช่น เมฆมาก (cloudy) คูณ 0.42 และมีเมฆบางส่วน (partly cloudy) คูณ 0.72 หากไม่มี pvlib ระบบจะถอยไปใช้โปรไฟล์การผลิตแบบฟังก์ชันเป็นค่าสำรอง

เพื่อให้การจำลองทำได้ (deterministic) มาตรวัดแต่ละตัวสุ่มสัญญาณรบกวนจากสตรีมเฉพาะของตน โดยเฉพาะค่าเริ่มต้น (seed) ของตัวสุ่มจากการ XOR ระหว่าง seed รวมของระบบกับไจเจสต์ SHA-256 ของรหัสมาตรวัด ทำให้การเพิ่มหรือลบมาตรวัดหนึ่งตัวไม่ทำให้ค่าอ่านของมาตรวัดตัวอื่นเคลื่อน ผลลัพธ์ต่อหนึ่งรอบคือเรคคอร์ดค่าอ่านพลังงาน (EnergyReading) ที่บรรจุพลังงานที่ผลิต พลังงานที่ใช้ พลังงานส่วนเกิน (surplus) และพลังงานส่วนขาด (deficit) เป็นหน่วย kWh ที่ไม่เป็นลบ พร้อมความยาวช่วงเวลา (interval) ของรอบนั้น ก่อนส่งออก มาตรวัดแต่ละตัวมีอัลกอริธึม Ed25519 เฉพาะตัว (per-meter key) ที่สร้างขึ้นแบบกำหนดได้จาก SHA-256 ของ *secret:meter_id* แล้วลงนามบนสายอักขระตามแบบแผน (canonical string) รูปแบบ *device_id:kwh:timestamp_ms* ส่งออกเป็นลายเซ็น base58 ทำให้ Aggregator Bridge ตรวจสอบที่มาของค่าอ่านได้รายจุดโดยไม่ต้องเก็บกุญแจลับของมาตรวัดไว้ที่ส่วนกลาง

2) Grid Simulation

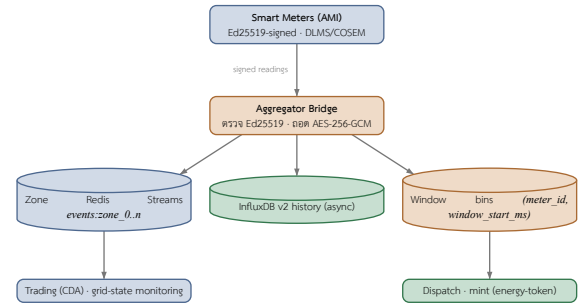
ในแต่ละรอบเวลา (tick) เมื่อรวบรวมค่าอ่านของมาตรวัดทุกตัวแล้ว ระบบจะคำนวณการอัดกำลังสุทธิ (net power injection) ของแต่ละ bus จากผลต่างระหว่างการผลิตและการใช้ไฟฟ้า แล้วแก้ปัญหา power flow เพื่ออัปเดตสถานะโครงข่าย ได้แก่ แรงดันต่อหน่วยของ bus, การไหลของกำลังในสาย (line flow), กำลังสูญเสีย (power loss) และระดับการใช้งานของสาย (line utilization) ตัวแก้หลักใช้ pandapower [33] แบบ backward/forward sweep (BFSW) ซึ่งเป็นการแก้ AC power flow แบบแม่นยำสำหรับโครงข่ายจำหน่ายแบบรัศมี (radial distribution) และเมื่อ pandapower ไม่พร้อมใช้งานหรือการคำนวณไม่ลู่เข้า (non-convergence) ระบบจะถอยไปใช้ตัวแก้แบบประมาณ DistFlow เป็นค่าสำรองเพื่อให้การจำลองดำเนินต่อไปได้

นอกจากการแก้ power flow พื้นฐาน ขึ้น grid simulation ยังจำลองกลไกควบคุมแรงดันและเหตุการณ์ผิดปกติของโครงข่ายจำหน่าย ได้แก่ การตอบสนองของอินเวอร์เตอร์แบบ volt-watt และ volt-VAR การปรับแทปหม้อแปลงแบบ on-load tap changer (OLTC) การฉีดความผิดปกติ (fault injection) ที่สาย bus หรือหม้อแปลง และการสับตัวเชื่อมต่อเชื่อมโยง (tie-switch) เพื่อถ่ายโอนโหลด ทำให้ชุดข้อมูลที่ส่งออกไปยัง Aggregator Bridge สะท้อนพฤติกรรมเชิงฟิสิกส์ของโครงข่ายภายใต้สภาวะการทำงานและสภาวะผิดปกติที่หลากหลาย มี

ใช้เพียงค่าพลังงานที่สุ่มขึ้นอย่างอิสระ ทั้งนี้ความสามารถด้านฟิสิกส์ของโครงข่ายข้างต้น (volt-VAR, OLTC, fault injection และ tie-switch) เป็นส่วนหนึ่งของชุดจำลอง แต่ยังไม่ได้ใช้เป็นตัวแปรในการประเมินที่รายงานในบทความนี้ ซึ่งวัตถุประสงค์การรับค่าอ่านที่ลงนามแล้ว ต้นทุนการชำระบนเซน และอัตราการจับคู่ (ดูหัวข้อ VIII) การประเมินผลกระทบของเงื่อนไข Grid stability เหล่านี้ต่อการจับคู่และการชำระจึงเป็นงานในอนาคต

เพื่อให้เป็นไปตามมาตรฐานอุตสาหกรรมและรับรองที่มาของข้อมูล

Aggregator Bridge รับข้อมูลมาตรวัดความถี่สูงแบบเวลาจริงแล้วกระจายผ่าน Redis Streams ที่แบ่งตามโซน (zone-partitioned) สำหรับการเฝ้าระวังสถานะโครงข่ายแบบพลวัต พร้อมรวมค่าอ่านเป็นหน้าต่างเวลา 15 นาทีเพื่อใช้ในการประเมินกำลังการผลิตและการสั่งการ (dispatch) ในชั้น Backend รูปแบบข้อมูลมาตรวัดเป็นไปตามมาตรฐาน DLMS/COSEM (IEC 62056) โดยถอดรหัสส่วน payload แบบ Binary ด้วย AES-256-GCM และเผยแพร่ต่อในรูปแบบ JSON นอกจากนี้ ระบบรับประกันความไม่ปฏิเสธเชิงรหัสวิทยา (Cryptographic Non-repudiation) ด้วยการตรวจสอบลายเซ็นเข้ารหัสแบบสมมาตร Ed25519 ทั้งแบบค่าอ่านรายจุดและแบบกลุ่ม (Batch) ก่อนรับข้อมูลเข้าสู่ระบบ ทั้งนี้ต้นแบบปัจจุบันยังไม่มีเส้นทางสร้าง Settlement Attestation บนเซนโดยตรงจากชั้นมาตรวัด ซึ่งเป็นแนวทางที่เปิดไว้สำหรับงานในอนาคต เส้นทางกระจายข้อมูลของ Aggregator Bridge สรุปไว้ใน รูปที่ 4



รูปที่ 4. Aggregator Bridge telemetry dissemination: verified readings fan out to zone-partitioned Redis Streams (realtime), an InfluxDB history sink (async, fire-and-forget), and 15-min aggregation windows keyed by (meter_id, window_start_ms). Each sink sits above the consumer it feeds; the InfluxDB history sink is terminal.

D. Consortium Network

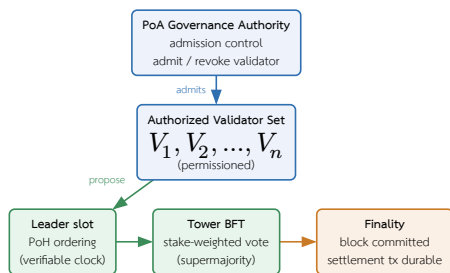
เครือข่ายบล็อกเชนในระบบนี้ออกแบบเป็น Consortium Network ภายใต้สมมติฐานการกำกับดูแลแบบ Proof of Authority (PoA) ตามที่อธิบายไว้ในหัวข้อ IV ส่วนนี้จึงเน้นรายละเอียดการออกแบบเครือข่ายและการกำกับดูแลเป็นหลัก การออกแบบทำให้ผู้ตรวจสอบบล็อกเป็นหน่วยงานที่มีหน้าที่ตรวจสอบหรือได้รับยินยอมจาก DSO เช่น ผู้ให้บริการรวบรวมโหลด (Load Aggregator) หน่วยงานกำกับดูแลหรือองค์กรที่ได้รับอนุญาต เหตุผลหลักในการเลือก PoA คือ Network Governance ที่ควบคุมได้ง่ายกว่าเครือข่ายสาธารณะ เช่น Solana Mainnet, Ethereum และความเหมาะสมต่อข้อกำหนดด้าน Regulatory compliance และ cost predictability สำหรับการทดลองหรือใช้งานในระบบพลังงานที่ต้องประมาณต้นทุนธุรกรรมได้ล่วงหน้า [15], [34] สิ่งสำคัญที่ต้องเน้นย้ำคือ PoA ในที่นี้เป็นชั้น governance และการควบคุมสิทธิ์การเข้าร่วม (admission control) ไม่ใช่การแทนที่กลไกฉันทามติระดับ Layer 1 ของเครือข่าย Solana-compatible ซึ่งยังคงอาศัย Proof of History (PoH) ร่วมกับ Tower BFT ในการเรียงลำดับและประกาศสิ้นสุดบล็อก (finality) ตามที่อธิบายไว้ในหัวข้อ IV

นโยบายด้านสิทธิ์และการกำกับดูแลถูกบังคับใช้ผ่านโปรแกรม governance บนเซน ซึ่งครอบคลุมสามกลไกหลัก กลไกแรกคือการกำกับหน่วยงานผู้มีสิทธิ์หลัก (PoA authority) ผ่านการโอนสิทธิ์แบบสองขั้นตอน (two-step handover) โดย propose_authority_change ให้ผู้มีสิทธิ์ปัจจุบันกำหนด pending authority พร้อมเวลาหมดอายุ และ approve_authority_change ต้องถูกเรียกโดย pending authority เองเพื่อรับโอน ทำให้ไม่สามารถยกสิทธิ์ให้บัญชีที่ไม่ยินยอมหรือผิดพลาดได้ และอนุญาตให้มีค่าของเปลี่ยนค่าได้เพียงรายการเดียวต่อครั้ง กลไกที่สองคือการควบคุม allow-list ของ Aggregator ที่ได้รับ

อนุญาตให้ลงนามค่าอ่านผ่าน admit_aggregator และ revoke_aggregator กลไกที่สามคือการลงคะแนนแบบ DAO ผ่าน create_proposal, cast_vote และ execute_proposal สำหรับปรับพารามิเตอร์เชิงเศรษฐศาสตร์ของแต่ละโซน (zone_config) ได้แก่ incentive multiplier, wheeling charge, loss factor และ maintenance mode

การลงคะแนนใช้น้ำหนักตามกำลังการผลิตสะสมของมาตรวัด (stake-by-generation) โดยน้ำหนักของผู้ลงคะแนนคำนวณเป็น $w = \max(100, \frac{\text{total_generation}}{1000})$ กล่าวคือทุก 1,000 kWh ของการผลิตสะสมเทียบเท่าหนึ่งหน่วยน้ำหนัก โดยมีค่าขั้นต่ำ 100 เพื่อให้ผู้เข้าร่วมรายเล็กยังมีเสียง การกันลงคะแนนซ้ำใช้บัญชี vote record แบบ PDA ต่อคู่ (proposal, voter) หนึ่งบัญชีต่อหนึ่งเสียง เมื่อสิ้นสุดช่วงเวลาลงคะแนน execute_proposal จะสรุปผลแบบอัตโนมัติ ข้อเสนอจะผ่าน (Passed) ก็ต่อเมื่อผลรวมคะแนนถึงเกณฑ์องค์ประชุมขั้นต่ำ (min_quorum_votes ที่กำหนดใน poa_config) และคะแนนเห็นด้วยมากกว่าคะแนนไม่เห็นด้วย มิฉะนั้นถือว่าตกไป (Rejected) จึงป้องกันการเปลี่ยนพารามิเตอร์ด้วยผู้ลงคะแนนจำนวนน้อยเกินไป

ความแตกต่างจาก Solana public mainnet คือเครือข่ายนี้ไม่เปิดให้ validator ภายนอกเข้าร่วมแบบ permissionless และสามารถกำหนดนโยบายด้านสิทธิ์ การอัปเดตโปรแกรม และการเก็บข้อมูลตามข้อกำหนดของโครงการได้ อย่างไรก็ตาม execution layer ยังคงอ้างอิงสถาปัตยกรรม Solana Virtual Machine และ Sealevel parallel runtime เพื่อให้ธุรกรรมที่ไม่ใช้ account เดียวกันสามารถประมวลผลแบบขนานได้ ค่า slot time ใกล้เคียง 400 ms และ compute budget ที่กล่าวถึงในบทความนี้เป็นเป้าหมายเชิงออกแบบที่อ้างอิงเอกสาร Solana ไม่ใช่ผลวัดของเครือข่าย permissioned การแบ่งบทบาทของชั้นกำกับดูแลแบบ PoA (admission control) ออกจากชั้นฉันทามติระดับ Layer 1 ที่ยังคงอาศัย PoH สำหรับการเรียงลำดับเวลาและ Tower BFT สำหรับการลงมติและประกาศ finality แสดงในรูปแบบที่ 5 [26]



รูปที่ 5. Separation of concerns in the consortium network (illustrative): the PoA layer governs admission control over an authorized validator set, while Layer 1 consensus is unchanged — PoH provides verifiable time ordering and Tower BFT provides stake-weighted voting and finality. Roles, not measured throughput.

ในระดับเครือข่ายดังกล่าวต้องกำหนดอย่างน้อยห้ารายการก่อนนำไปใช้งานจริง ได้แก่ จำนวน Validator และหน่วยงานเจ้าของโหนด วิธีผูก identity กับกุญแจ Validator นโยบายเพิ่มหรือลบ Validator กระบวนการอัปเดตโปรแกรมหรือ genesis/configuration และ fault model ที่ยอมรับได้ บทความนี้จึงถือว่า PoA Solana-compatible network เป็นสมมติฐานเชิงสถาปัตยกรรมของระบบจำลอง ไม่ใช่ข้อสรุปว่ามีการปรับแต่ง consensus ของ Solana public network แล้ว

E. Smart Contract Programs

Smart Contract แบ่งออกเป็นหลายโปรแกรมตามความรับผิดชอบ ได้แก่ registry, trading, oracle, energy-token, governance และ treasury รวมถึงโปรแกรมสำหรับการทดสอบประสิทธิภาพ (blockbench และ tpc-benchmark) ซึ่งพัฒนาด้วย Anchor Framework เพื่อกำหนด account validation, instruction handler และ Event log อย่างเป็นระบบ โปรแกรม registry มี register_user และ register_meter สำหรับลงทะเบียนผู้ใช้และ Smart Meter ส่วนโปรแกรม trading มี create_sell_order และ create_buy_order (รวมถึง submit_limit_order และ submit_market_order) สำหรับส่งคำสั่งซื้อขายและซื้อพลังงาน, match_orders และ clear_auction สำหรับจับคู่คำสั่งซื้อขายที่ Backend เสนอมา,

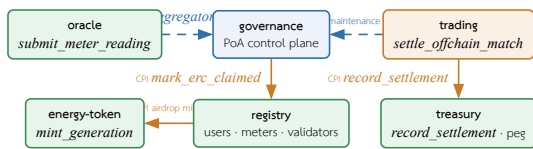
settle_offchain_match และ execute_atomic_settlement สำหรับชำระธุรกรรมแบบ atomic settlement โดย settle_offchain_match จะตรวจสอบลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขายและใช้ order nullifier เพื่อป้องกันการส่งซ้ำ และโปรแกรม energy-token มี mint_generation และ mint_to_wallet สำหรับสร้าง energy token ที่ต้องผ่านการร่วมลงนามของ REC certifying authority (Renewable Energy Certificate) โดย mint_generation ใช้คีย์ (meter_id, ช่วงเวลา settlement) เพื่อรับประกัน idempotency ต่อหนึ่งหน้าค่าเวลา และ burn_tokens สำหรับเผา energy token ผ่านคำสั่ง SPL นอกจากนี้โปรแกรม oracle มี submit_meter_reading สำหรับรับค่าอ่านจาก AMI, trigger_market_clearing สำหรับกำหนดขอบเขตหน้าต่างเวลา 15 นาที (900 วินาที) และ aggregate_readings สำหรับรวมตัวนับค่าอ่านที่ผ่านและไม่ผ่านการตรวจสอบ ลำดับเวลาของหน้าต่างรวมข้อมูลและการเคลียร์ตลาดแสดงในรูปแบบที่ 7 โปรแกรม governance ทำหน้าที่เป็นชั้นควบคุมแบบ PoA สำหรับการรับรองและเพิกถอน Renewable Energy Certificate (REC) การกำกับสิทธิ์ผู้มีอำนาจ การจัดการ allow-list ของ aggregator และการลงคะแนนแบบ DAO (โครงสร้างบัญชีและบทบาทชั้นควบคุมอธิบายในหัวข้อ VI.E.2 รายละเอียดการกำกับดูแลเครือข่ายในส่วน Consortium Network) โปรแกรม treasury จัดการการ stake โทเคน GRX การจ่ายรางวัล และการรักษาค่าตริงของ THBC stablecoin ผ่านคำสั่งเช่น stake_grx, unstake_grx, claim_rewards, swap_grx_for_thbc, redeem_thbc_for_grx และ record_settlement ที่เรียกผ่าน Cross-Program Invocation (CPI) จากโปรแกรม trading ส่วนโปรแกรม blockbench และ tpc-benchmark ใช้รัน workload มาตรฐาน ได้แก่ Blockbench micro-benchmark, YCSB, SmallBank และ TPC-C สำหรับการประเมินประสิทธิภาพบนเครือข่าย Solana-compatible [25], [35] รหัสโปรแกรมบนเซต (program ID) ของโปรแกรมหลักทั้งหมดซึ่งประกาศด้วย declare_id! สรุปไว้ในตารางที่ VII

ตารางที่ VII

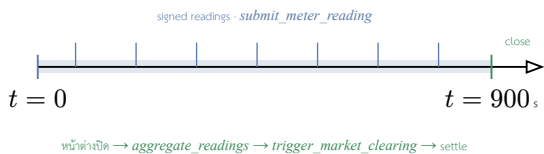
On-chain program identities (Anchor *declare_id!*) of the six core programs on the Solana-compatible consortium network. These deterministic addresses pin the deployed artifact for reproducibility.

โปรแกรม	Program ID (<i>declare_id!</i>)
registry	FcSd5x4X1nzJMKLZC4tMZXnQ1pLrGsEFeoH8N4mwJX7
trading	CnWDEUhtvSixeLSyViWgAnnu9YouBAYVGcrrFm1s9WcX
oracle	64Vgos61STZ8pW9NnHi2iGtXMTQr7NqBoMorK6Zg8RJu
energy-token	6FZKcVKCLFSLMxypFJGU4K14xUBnxNW9VAuKGhmjqGX
governance	FokVuBSXP11aeL7VZwD8n8aVAhWqVpyPZETToSxdvTS
treasury	FfxSQYKUmX9NgdCC9TDPmZSYjWYE1h4ruu3JatzHN5tn

โครงสร้างบัญชีใช้ Program Derived Address (PDA) เพื่อแยก state ของระบบออกเป็นบัญชีย่อยตาม seed เฉพาะ เช่น registry และ user account, meter account, market และ market_shard, order และ trade record, escrow, order nullifier สำหรับป้องกันการส่งซ้ำ, oracle_data และ mint authority (gen_mint, thbc_mint) การใช้ PDA ทำให้โปรแกรมตรวจสอบ ownership และ deterministic address ของบัญชีได้โดยไม่ต้องพึ่ง private key ของบัญชี state แต่ละรายการ งานที่ใช้ทรัพยากรสูง เช่น การคำนวณราคาแบบซับซ้อน การจับคู่ order book ขนาดใหญ่ หรือการวิเคราะห์ Grid stability จึงถูกย้ายไปทำใน Backend และ Aggregator Bridge ก่อนส่งผลลัพธ์ที่ยืนยันแล้วเข้าสู่ Smart Contract เพื่อให้อยู่ภายใต้ข้อจำกัดด้าน compute ของธุรกรรม [27], [36] โดยกลไกจับคู่คำสั่งใน Trading Service ใช้อัลกอริทึม Continuous Double Auction (CDA) แบบ price-time priority ที่แบ่ง order book ตามโซนและคำนึงถึงข้อจำกัดการไหลของพลังงาน (wheeling และ loss) ก่อนส่งคู่คำสั่งที่จับคู่แล้วไปชำระแบบ atomic บน Smart Contract รายละเอียดสูตรการกำหนดราคาและค่าธรรมเนียมอธิบายในหัวข้อ V.A ความสัมพันธ์ระหว่างโปรแกรมทั้งหมดเป็นสองชนิด คือการเรียกข้ามโปรแกรม (Cross-Program Invocation: CPI) ที่เขียนสถานะ และการอ่านสถานะแบบควบคุม (control-plane read) ที่โปรแกรมหนึ่งอ่านบัญชีของอีกโปรแกรมเพื่อกำหนดสิทธิ์การทำงานโดยไม่เรียก CPI ดังสรุปในรูปแบบที่ 6



รูปที่ 6. Inter-program relationships among the six Anchor programs. Solid arrows are Cross-Program Invocations that write state (governance→registry on REC issue, registry→energy-token on airdrop, trading→treasury on settlement). Dashed arrows are control-plane reads with no CPI: trading reads **governance** for the maintenance gate and REC validity, and oracle validates an admitted-aggregator entry before accepting a reading. Governance (blue) is the policy source; trading (orange) initiates on-chain settlement.



รูปที่ 7. Market-clearing timeline: signed meter readings ingest continuously over the 15-minute (900 s) aggregation window; at window close the oracle aggregates readings, triggers clearing, and the matched pair is settled on-chain.

1) Trading: วงจรคำสั่งซื้อขายและ escrow บนเชน

โปรแกรม trading จัดการวงจรชีวิตของคำสั่งซื้อขายและบัญชี escrow บนเชน เสริมกับเส้นทางการชำระในตัวข้อ IV ผู้ใช้สร้างคำสั่งผ่าน create_sell_order และ create_buy_order (รวมถึง submit_limit_order และ submit_market_order) ซึ่งสร้างบัญชี Order แบบ PDA รายการคำสั่งที่ผูกกับเจ้าของ พร้อมกำหนดอายุคำสั่ง (expires_at) ไว้ที่ 24 ชั่วโมง (86,400 วินาที) นับจากเวลาสร้าง ทั้งนี้คำสั่งซื้อขายต้องอ้างใบรับรอง REC ที่ยังไม่หมดอายุก่อนจึงจะสร้างได้ ซึ่งสอดคล้องกับการกำกับด้วย REC ในตัวข้อ VI.E.2 ก่อนเข้าสู่ตลาด ผู้ใช้ฝากโทเคนเข้าบัญชี escrow ผ่าน deposit_escrow และถอนคืนส่วนที่ไม่ถูกใช้ผ่าน withdraw_escrow โดยบัญชี escrow เป็น SPL token account ภายใต้อาณัติ PDA ที่ลงนามโดย market_authority

การจับคู่คำสั่งมีสองเส้นทางที่เสริมกัน เส้นทางแรกคือเครื่องจับคู่ในชั้น off-chain (trading-engine) ที่รัน CDA แบบ price-time priority เหนือสมุดคำสั่งทั้งหมดด้วยปริมาณงานสูง (ดูหัวข้อ V.C และอัตราที่วัดได้ในหัวข้อ VIII.D) เส้นทางที่สองคือคำสั่ง match_orders บนเชนที่ตรวจสอบและบันทึกการจับคู่รายคู่โดยแต่ละเฉพาะบัญชี Order สองรายการกับ zone_market แล้วเขียน trade record โดยไม่เคลื่อนย้ายโทเคน จึงมีต้นทุน compute ต่ำ (ดูรูปที่ 11) ส่วนคำสั่ง cancel_order ลบคำสั่งที่ยังค้างในสมุด

บัญชีระดับตลาดถูกแยกออกเป็นบัญชี zone_market รายโซนที่เก็บความลึกของสมุดคำสั่ง (order book depth) ความจุสายส่ง (capacity) และการไหลที่ผูกพันแล้ว (committed_flow) แยกออกจากบัญชี Market กลาง เพื่อให้การส่งคำสั่งและการบังคับเขตแดนการไหลข้ามโซนขนานกันได้ตามแบบ Sealevel (ดูหัวข้อ VI.E.7) เมื่อคำสั่งผ่านการจัดคู่แล้วจะถูกส่งไปชำระแบบ atomic ผ่าน settle_offchain_match ตามแบบจำลองการชำระในตัวข้อ IV

2) Governance Program: ชั้นควบคุม PoA บนเชน

โปรแกรม governance ทำหน้าที่เป็นชั้นควบคุม (control plane) แบบ Proof of Authority บนเชน ออกแบบเป็นโปรแกรมเดียวที่รวม 20 instruction แบ่งตามหน้าที่เป็นห้าระบบย่อย (กลุ่มคำสั่งหลัก 19 รายการ รวมกับคำสั่งสืบค้นสถิติ get_governance_stats อีกหนึ่งรายการ) จุดสำคัญเชิงสถาปัตยกรรมคือ governance ไม่ได้ทำงานโดดเดี่ยว แต่เป็นแหล่งความจริง (source of truth) ที่โปรแกรมอื่นบนเชนอ่านสถานะไปกำหนดพฤติกรรมของตน กล่าวคือ governance บันทึก “นโยบาย” ขณะที่ trading และ oracle เป็นผู้ “บังคับใช้” นโยบายนั้น ณ ขอบเขตของโปรแกรมตนเอง ระบบย่อยทั้งห้าประกอบด้วย

- ระบบสิทธิ์ผู้มีอำนาจ (Authority): initialize_governance สร้างบัญชี singleton เริ่มต้น และการโอนสิทธิ์ที่แบบสองขั้นตอน propose_authority_change → approve_authority_change โดยผู้รับโอนต้องลงนามเอง พร้อม cancel_authority_change และเวลาหมดอายุ 48 ชั่วโมง ไม่มีเส้นทางโอนสิทธิ์แบบขั้นตอนเดียว (อธิบายเพิ่มในหัวข้อ VI.D)

- ระบบควบคุมพารามิเตอร์ (Config gates): update_governance_config (เปิด/ปิดการตรวจ REC และการโอน), update_erc_limits, set_maintenance_mode (สวิตช์หยุดทั้งระบบ), update_authority_info และ set_oracle_authority ทุกคำสั่งต้องลงนามโดย authority
- ระบบใบรับรองพลังงานหมุนเวียน (REC certificates): issue_erc ตรวจสอบตามนโยบายใน GovernanceConfig แล้วเรียก Cross-Program Invocation (CPI) ไปยัง registry เพื่อทำเครื่องหมายว่าพลังงานถูกอ้างสิทธิ์แล้ว (mark_erc_claimed) ร่วมกับ validate_erc_for_trading, transfer_erc และ revoke_erc (ตัวระบุ erc หมายถึงการออกใบรับรองภายใต้อำนาจ ERC ส่วนตัวใบรับรองเองคือ REC)
- ระบบลงคะแนน DAO: create_proposal → cast_vote (ถ่วงน้ำหนักตามกำลังผลิต) → execute_proposal โดยจำกัดให้แค่เฉพาะพารามิเตอร์ของ ZoneConfig ที่กำหนดไว้เท่านั้น ไม่แต่ต้องอำนาจ PoA (รายละเอียดการถ่วงน้ำหนักและองค์ประกอบในหัวข้อ VI.D)
- ระบบ allow-list ของ Aggregator: admit_aggregator และ revoke_aggregator (เฉพาะ authority) ซึ่งแต่ละรายการคือบัญชี PDA เฉพาะ ทำหน้าที่รับเข้า (admission) โทเคน validator นอกเชนที่ละราย

บัญชีสถานะ (state account) ของโปรแกรมเป็นบัญชี PDA ที่กำหนด address ได้แบบ deterministic จาก seed เฉพาะ ดังสรุปในตารางที่ VIII

ตารางที่ VIII

บัญชีสถานะ (PDA) ของโปรแกรม governance: seed และข้อมูลที่เก็บ บัญชี GovernanceConfig เป็น singleton ครั้งขนาดคงที่ 405 ไบต์เพื่อรองรับการอัปเดต ขณะที่บัญชีที่เหลือสร้างหนึ่งรายการต่อหนึ่งเอนทิตี (per-cert / per-node / per-zone / per-proposal).

บัญชี (PDA)	Seed	เก็บข้อมูล
GovernanceConfig	[poa_config]	singleton: authority, pending_authority, นโยบาย REC, maintenance flag, min_quorum_votes และตัวนับ
ErcCertificate	[erc_certificate, cert_id]	REC: owner, พลังงาน (kWh), สถานะ, วันหมดอายุ
AggregatorEntry	[aggregator, pubkey]	โทเคน validator นอกเชนที่ถูก admit (allow-list)
ZoneConfig	[zone_config, zone_id]	พารามิเตอร์รายโซน: wheeling charge, incentive, loss factor
Proposal	[proposal, zone, id]	ข้อเสนอ DAO: พารามิเตอร์เป้าหมาย, คะแนนเห็นด้วย/ไม่เห็นด้วย, สถานะ, เวลาหมดอายุ
VoteRecord	[vote, proposal, voter]	การลงคะแนนหนึ่งเสียงต่อคู่ (proposal, voter)

จุดที่ทำให้ governance มีสถานะเป็น control plane อย่างแท้จริงคือวิธีที่โปรแกรมอื่นบริโภคสถานะของมัน โปรแกรม trading อ่านบัญชี GovernanceConfig ในทุกคำสั่งสร้างคำสั่งซื้อขาย แล้วเรียก is_operational() เพื่อปิดกั้นการทำงานเมื่อระบบอยู่ในโหมดบำรุงรักษา (maintenance) พร้อมตรวจสอบความสมบูรณ์ของ REC ก่อนรับคำสั่งซื้อขาย ขณะที่โปรแกรม oracle ตรวจสอบบัญชี AggregatorEntry เพื่ออนุญาตเฉพาะโทเคนที่ถูก admit แล้ว (หรือ chain bridge ที่กำหนดไว้) ให้ส่งค่าอ่านผ่าน submit_meter_reading ได้ทั้งสองกรณีเป็นการอ่านสถานะแบบ read-only ด้วยการ deserialize บัญชีเองและตรวจ owner กับ address ของ PDA โดยไม่เรียก CPI ซึ่งหลีกเลี่ยงต้นทุนของ CPI และทำให้การบังคับใช้นโยบาย (gating) เป็นเพียงการตรวจบัญชีที่ส่งเข้ามาในธุรกรรมนั้น ในแง่นี้บัญชี AggregatorEntry จึงเป็นบันทึกบนเชนของการรับเข้าโทเคน validator นอกเชน ที่ถูกบังคับใช้จริง ณ จุดที่ oracle รับค่าอ่านเข้าสู่ระบบ

โครงสร้างนี้สะท้อน PoA สองชั้นที่แยกจากกัน ได้แก่ ชั้นปฏิบัติการ (operational) ที่กำหนดว่าใครได้รับอนุญาตให้รัน validator บนเครือข่าย permissioned และชั้นแอปพลิเคชัน (application-layer) ที่ GovernanceConfig.authority ควบคุมสิทธิ์การบริหารโปรแกรม โดย DAO ถูกจำกัดอำนาจ (power-bounded) ให้ปรับได้เฉพาะพารามิเตอร์ของโซน ไม่สามารถยึดอำนาจ authority หรือแก้ allow-list ของ validator ได้ นอกจากนั้นบัญชี GovernanceConfig ยังถูกตรึงขนาดไว้คงที่ที่ 405 ไบต์พร้อมพื้นที่สำรอง (_reserved) สำหรับการอัปเดต ทำให้เพิ่มฟิลด์ใหม่ได้โดยไม่ต้องย้าย

(migrate) บัญชีเดิม ซึ่งเป็นวินัยด้านการออกแบบที่จำเป็นเมื่อโปรแกรมอื่นต่างพึ่งพา layout ของบัญชีนี้ในการ deserialize เอง

วงจรชีวิตของใบรับรอง REC ออกแบบรอบคุณสมบัติห้ามอ้างสิทธิ์ซ้ำ (anti-double-claim) ก่อนออกใบรับรอง คำสั่ง `issue_erc` คำนวณพลังงานที่ยังไม่ถูกอ้างสิทธิ์เป็น $\text{unclaimed} = \text{total_generation} - \text{claimed_erc_generation} - \text{settled_net_generation}$ แบบ saturating แล้วบังคับว่าปริมาณที่ขอออกต้องไม่เกินค่านี้ ($\text{energy_amount} \leq \text{unclaimed}$) มิฉะนั้นปฏิเสธ พร้อมตรวจนโยบายจาก GovernanceConfig ผ่าน `can_issue_erc()` (ระบบต้องอยู่ในสถานะทำงานและเปิดการตรวจ REC และปริมาณต้องอยู่ในช่วง `min_energy` ถึง `max_erc`) เมื่อผ่านเงื่อนไข โปรแกรมเรียก CPI ไปยัง registry เพื่อเพิ่มตัวนับพลังงานที่ถูกอ้างสิทธิ์ (`mark_erc_claimed`) ทำให้พลังงานหน่วยเดียวกันถูกรับรองซ้ำไม่ได้ ใบรับรองที่ออกใหม่มีสถานะ Valid แต่ตั้งค่า `validated_for_trading` เป็นเท็จจนกว่าจะผ่านคำสั่ง `validate_erc_for_trading` (ต้องอยู่ในสถานะทำงาน สถานะ Valid และยังไม่หมดอายุ) ซึ่งเป็นเงื่อนไขบังคับก่อนใบรับรองจะนำไปค้าคำสั่งซื้อขายได้ ส่วน `revoke_erc` และ `transfer_erc` ควบคุมการเพิกถอน (กั้นการเพิกถอนซ้ำ) และการโอนสิทธิ์ (อนุญาตเฉพาะเมื่อเปิดการโอนใบรับรองหรือเป็นการโอนจากผู้ออกเอง) โดยทุก handler ปรับปรุงตัวนับรวมใน GovernanceConfig (จำนวนใบรับรองที่ออก/ตรวจ/เพิกถอน และพลังงานสะสมที่รับรอง)

วงจรข้อเสนอ DAO มีการป้องกันหลายชั้นนอกเหนือจากการถ่วงน้ำหนักเสียงตามกำลังผลิต (อธิบายในหัวข้อ VI.D) คำสั่ง `create_proposal` ปฏิเสธช่วงเวลาลงคะแนนที่ไม่เป็นบวก และบังคับว่าผู้เสนอต้องเป็นเจ้าของมาตรวัดที่อ้างอิง โดยอ่านบัญชี MeterAccount แบบ zero-copy แล้วตรวจว่า `meter.owner` ตรงกับข้อเสนอ คำสั่ง `cast_vote` รับเฉพาะเมื่อข้อเสนออยู่ในสถานะ Active และเวลายังไม่เลย `expires_at` มิฉะนั้นปฏิเสธด้วย ProposalExpired และผู้ลงคะแนนต้องเป็นเจ้าของมาตรวัดเช่นกัน การลงคะแนนซ้ำถูกกั้นด้วยบัญชี VoteRecord PDA ต่อคู่ (proposal, voter) ซึ่งการสร้างบัญชีครั้งที่สองจะล้มเหลวเพราะบัญชีมีอยู่แล้ว คำสั่ง `execute_proposal` บังคับให้เวลาผ่าน `expires_at` ก่อน แล้วสรุปผลอัตโนมัติ คือถ้าผลรวมเสียงต่ำกว่าองค์ประชุม (`min_quorum_votes`) ถือว่าตกไป ถ้าถึงองค์ประชุมและเสียงเห็นด้วยมากกว่าไม่เห็นด้วยจึงผ่าน (กรณีเสียงเท่ากันถือว่าตก) และเมื่อผ่านจะปรับได้เฉพาะพารามิเตอร์ ZoneConfig สี่รายการเท่านั้น ได้แก่ `incentive_multiplier`, `wheeling_charge`, `loss_factor` (ต้องเป็นค่าบวก) และ `maintenance_mode` ต่อยาขอบเขตอำนาจที่จำกัดของ DAO

กลไก maintenance แสดงคุณสมบัติของสวิตช์หยุดทั้งระบบ (system-wide kill switch) ผ่านการเขียนบัญชีจุดเดียว เมื่อ `authority` เรียก `set_maintenance_mode(true)` ค่า boolean เพียงค่าเดียวบนบัญชี singleton GovernanceConfig จะพลิก ส่งผลให้ทุกคำสั่งสร้างคำสั่งซื้อขายในโปรแกรม trading ทุกโซนถูกปฏิเสธด้วย MaintenanceMode ทันทีโดยไม่ต้อง deploy โปรแกรม trading ใหม่ ทั้งนี้การอ่านฝั่ง trading ข้ามดิสคริมีเนเตอร์ (discriminator) 8 ไบต์แรกของบัญชีแล้วถอดรหัส Borsh ตาม layout ของ struct โดยตรง การ gate จึงหน่วงต่อการเปลี่ยนดิสคริมีเนเตอร์ของบัญชี governance ได้ตรงไปตรงมาที่ลำดับฟิลด์ไม่เปลี่ยน ซึ่งสอดคล้องกับการตรึงขนาดบัญชีและพื้นที่สำรองที่กล่าวไว้ข้างต้น

3) Registry: การลงทะเบียนและหลักประกันของ Validator

โปรแกรม registry เป็นทะเบียนกลางของผู้ใช้ มาตรวัด และผู้ตรวจสอบ (validator) คำสั่ง `register_user` และ `register_meter` สร้างบัญชี UserAccount และ MeterAccount แบบ zero-copy ที่ผูกกับเจ้าของผ่าน PDA นอกจากบทบาททะเบียนแล้ว โปรแกรมนี้ยังถือหลักประกัน (security bond) ของผู้ตรวจสอบ โดย `register_validator` กำหนดให้ต้องวางเงินค้ำ (stake) GRX ขั้นต่ำ 10,000 GRX (`MIN_VALIDATOR_STAKE`) ก่อนได้รับสถานะ validator ขณะที่ `stake_grx` โอน GRX เข้าคลังกลางและมีช่วงรอถอน (cooldown) 24 ชั่วโมง เมื่อผู้ตรวจสอบประพฤติผิด คำสั่ง `slash_validator` ที่ลงนามโดย authority แบบ PoA จะหักหลักประกันแบบปรับตามความรุนแรง คือ $\text{slash} = [\text{bond} \times \text{slash_bps} / 10000]$ โดยจ่ายชดเชยผู้เสียหายไม่เกินความเสียหายที่พิสูจน์ได้ ($\text{compensation} =$

$\text{min}(\text{slash}, \text{proven_loss})$) และส่วนที่เหลือเข้ากองทุน slash เพื่อตัดแรงจูงใจการกล่าวหาเพื่อหวังรางวัล การหักเต็มจำนวนเปลี่ยนสถานะเป็น Slashed (ถาวร ห้ามลงทะเบียนซ้ำ) ส่วนการหักบางส่วนจนต่ำกว่าขั้นต่ำเปลี่ยนเป็น Suspended (กู้คืนได้ด้วยการเติมเงินค้ำ) กลไก stake-and-slash นี้เป็นแรงจูงใจเชิงเศรษฐศาสตร์ที่เสริมการควบคุมการเข้าร่วมแบบ PoA นอกจากนี้การมอบโทเคนเริ่มต้น 10 GRX แก่ผู้ใช้ใหม่ถูกแยกออกจาก `register_user` ไปเป็นคำสั่ง `claim_airdrop` ต่างหาก (เรียก CPI ไปยัง `energy-token` เพื่อ mint โดยลงนามด้วย PDA ของ registry) เพื่อให้การลงทะเบียนไม่ล้มเหลวทั้งธุรกรรมหากการ mint มีปัญหา

4) Oracle: การรับค่าอ่านแบบขนานและการเคลียร์ตลาด

โปรแกรม oracle รับค่าอ่านมาตรวัดเข้าสู่เซ่นผ่าน `submit_meter_reading` โดยออกแบบให้เขียนลงบัญชี MeterState รายมาตรวัด (`seed [meter, meter_id]`) ขณะที่บัญชี OracleData ส่วนกลางถูกอ่านอย่างเดียว ค่าอ่านของมาตรวัดต่างตัวจึงมีชุดบัญชีที่เขียน (`write set`) ไม่ทับซ้อนกันและประมวลผลแบบขนานได้ตามแบบจำลอง Sealevel อย่างไรก็ตามโค้ดระบุข้อจำกัดตามจริงว่า การขนานเกิดขึ้นเมื่อผู้ลงนามจ่ายค่าธรรมเนียม (fee payer) ต่างกันเท่านั้น หากค่าอ่านหลายรายการใช้ผู้ลงนาม gateway เดียวกัน รายการเหล่านั้นยังถูกประมวลผลแบบลำดับเพราะบัญชีผู้จ่ายถูกล็อกเขียน ก่อนรับค่าอ่าน โปรแกรมตรวจความผิดปกติ (anomaly) ด้วยการตรวจช่วงค่าพลังงาน (`min/max`) และอัตราส่วนการผลิตต่อการใช้ ซึ่งคำนวณแบบจำนวนเต็มเพื่อเลี่ยงเลขทศนิยม ($\text{produced} \times 100 \leq \text{max_ratio} \times \text{consumed}$) ส่วนสิทธิ์การส่งค่าอ่านจำกัดเฉพาะ chain bridge ที่กำหนด หรือโหนดที่อยู่ใน allow-list ของ governance ผ่านการตรวจบัญชี AggregatorEntry ใน `authorize_node_caller` (ดูหัวข้อ VI.E.2) การเคลียร์ตลาดทำผ่าน `trigger_market_clearing` ที่บังคับให้ขอบหน้าต่าง epoch ตรงกับ 900 วินาที ($\text{epoch_timestamp} \% 900 == 0$) และบันทึก `last_cleared_epoch` เพื่อกันการเคลียร์ซ้ำหรือย้อนเวลา

5) Energy Token: การออกโทเคนแบบ Idempotent และการกำกับด้วย REC

โปรแกรม energy-token จัดการการสร้าง (mint) โทเคนพลังงานภายใต้การกำกับของคณะผู้รับรอง REC เส้นทาง การ mint ทั้งสาม (`mint_to_wallet`, `mint_generation`, `mint_tokens_direct`) บังคับให้ผู้ลงนามต้องอยู่ในชุดผู้รับรอง REC (สูงสุด 5 ราย) เมื่อมีการตั้งผู้รับรองไว้แล้ว (`rec_validators_count > 0`) จึงเป็นการร่วมลงนามของหน่วยรับรองพลังงานหมุนเวียนก่อนสร้างโทเคน คำสั่ง `mint_generation` ที่สร้างโทเคนจากการผลิตจริงรับประกันความไม่ซ้ำซ้อน (idempotency) ต่อหนึ่งหน้าต่างเวลา ด้วยบัญชี GenerationMintRecord รายคู่ (`meter_id`, `window_start_ms`) ที่สร้างแบบ `init_if_needed` และบังคับให้ขอบหน้าต่างตรงกับ 900,000 มิลลิวินาที โดยตั้งงม minted เป็นจริงหลังการ mint สำเร็จเท่านั้น การเรียกซ้ำที่หน้าต่างเดิมจึงคืนค่าเป็น no-op และหากการ mint ล้มเหลว ริงยังเป็นที่ทำให้ลองใหม่ได้ คำสั่ง `mint_tokens_direct` เป็นเป้าหมาย CPI ที่ registry เรียกเพื่อมอบโทเคนเริ่มต้น ขณะที่ `burn_tokens` เผาโทเคนผ่านคำสั่ง SPL Token-2022 ทั้งนี้บัญชี TokenInfo ที่เก็บชุดผู้รับรองและตัวนับเป็นแบบ zero-copy เพื่อให้เส้นทางความถี่สูงอ่านได้โดยไม่ล็อกเขียน

6) Treasury: การบันทึกการชำระและการตรึงค่า THBC

โปรแกรม treasury เป็นปลายทาง CPI ของการบันทึกการชำระจากโปรแกรม trading และเป็นแกนการเงินของระบบ การบันทึกการชำระแบบกลุ่ม (`record_settlement_batch`) เขียนบัญชี SettlementRecord รายคู่ (`zone_id`, `batch_id`) ที่เก็บรากเมอร์เคิล (`merkle_root` ขนาด 32 ไบต์) มูลค่ารวม และภาษีมูลค่าเพิ่ม (`vat_amount`, `vat_rate_bps`) เป็นข้อมูลพื้นฐานสำหรับตรวจสอบย้อนหลัง โดยรากเมอร์เคิลถูกชุดธุรกรรมในกลุ่มไวบนเซกซ์ของต้นไม้ (leaf) ของต้นไม้ถูกเก็บนอกเซกซ์ คำสั่งฐานนี้กระหนาบยอดรวมส่วนกลาง (`total_settled_thbc`) โดยตรง เพื่อรองรับการชำระพร้อมกันจำนวนมากโดยไม่ให้บัญชี treasury กลายเป็นคอขวด จึงมีคำสั่งแบบขนาน (`record_settlement_batch_sharded`) ที่บันทึก SettlementRecord เช่นเดิมแต่เพิ่มยอดลงบัญชีตัวสะสมรายชาร์ดแทนยอดรวมส่วนกลาง โดยแบ่งออกเป็น 16 ชาร์ด ($\text{settle_shard_for}(\text{key}) = \text{key}[0] \% 16$) แล้วกระหนาบยอดเข้าสู่ศูนย์กลางภายหลังด้วย `aggregate_settlement_shards` ด้านการตรึงค่า

stablecoin THBC คำสั่ง `swap_grx_for_thbc` บังคับว่าอุปทานใหม่ต้องไม่เกินทุนสำรองที่ได้รับการรับรอง ($new_supply \leq attested_reserve$) และการรับรองต้องยังไม่หมดอายุ ($now - attestation_ts \leq attestation_ttl$) ส่วน `redeem_thbc_for_grx` จำกัดการไถ่ถอนไม่ให้เกินหลักประกันในคลัง ($grx_out \leq swap_vault.amount$) นอกจากนี้การวางเงินค้ำ GRX เพื่อรับรางวัลใช้ตัวสะสมแบบ MasterChef ($acc_reward_per_share$ สเกลด้วย 10^{12}) ที่เพิ่มขึ้นตามรางวัลที่เติมเข้า ($\delta = amount \times ACC / total_staked$) และเมื่อมีการ slash หลักประกันจะถูกกระจายต่อให้ผู้วางเงินค้ำที่เหลือผ่านการเพิ่มตัวสะสมเดียวกัน

7) การประมวลผลแบบขนานด้วย Sealevel และการแบ่งชาร์ต

รูปแบบการออกแบบบัญชีที่ปรากฏขึ้นในหลายโปรแกรมคือการใช้บัญชี PDA รายเอนทิตี เพื่อให้ธุรกรรมที่ไม่เกี่ยวข้องกันมีชุดบัญชีที่เขียนไม่ทับซ้อนและประมวลผลแบบขนานได้ตามแบบจำลอง Sealevel ของ Solana ตัวอย่างเช่น บัญชี MeterState รายการวัดในโปรแกรม oracle, บัญชี Order และ order nullifier รายการคำสั่งในโปรแกรม trading และบัญชี escrow รายผู้ใช้ สำหรับตัวนับรวมที่หากเขียนลงบัญชีเดียวจะกลายเป็นคอขวด (เช่น จำนวนผู้ใช้สะสม หรือยอดชำระสะสม) ระบบใช้การแบ่งชาร์ตจำนวน 16 ชาร์ตแบบกำหนดได้จากไบนารีแรกของกุญแจ ($key.to_bytes() [0] \% 16$) ทั้งในโปรแกรม registry (ผู้ใช้และมาตรวัด) และ treasury (ยอดชำระ) แล้วจึงกระหนาบยอดเข้าตัวนับศูนย์กลางภายหลังด้วยคำสั่งระดับผู้ดูแล (`aggregate_shards`, `aggregate_settlement_shards` และ `aggregate_readings`) ที่ป้องกันการนับซ้ำด้วยบิตแมสก์ ผลคือบัญชีส่วนกลาง เช่น GovernanceConfig, OracleData และ Market ถูกออกแบบให้อ่านแบบล่าช้าได้ (stale) บนเส้นทางที่มีความถี่สูง โดยยอมรับว่าค่าธรรมเนียมหลังเล็กน้อยเพื่อแลกกับ throughput ที่ขนานได้ การออกแบบนี้สอดคล้องกับเป้าหมาย slot time ใกล้ 400 มิลลิวินาทีที่อ้างถึงในหัวข้อ VI.D ส่วนปริมาณงานธุรกรรมจริงบนเซิร์ฟเวอร์ validator เดียวและรายงานในหัวข้อ VIII.F โดยตัวเลขบนเครือข่าย permissioned หลายโหนดที่รวมต้นทุน consensus ยังเป็นงานในอนาคต (ดูหัวข้อ VIII.E)

F. Data Model and Trust Boundary

ข้อมูลหลักของคำสั่งซื้อขายประกอบด้วย `order_id`, `user_id` (บทบาท prosumer หรือ consumer), `meter_id`, `side` (offer หรือ bid), `energy_amount` (quantity_kwh), `price_per_kwh`, `status`, `expires_at`, `epoch_id` และ `zone_id` โดยปริมาณ `energy_amount` ต้องไม่เกิน `available_surplus_kwh` ที่ Aggregator Bridge รับรองสำหรับผู้ขายในช่วงเวลาเดียวกัน ซึ่งเป็นเงื่อนไขที่ตรวจสอบในชั้น off-chain ส่วนการป้องกันการส่งซ้ำใช้บัญชี nullifier บนเชน (order nullifier) แทนการเก็บ nonce ไว้ในตัวคำสั่ง ส่วน settlement record ประกอบด้วย `trade_id`, คู่ `buy_order_id/sell_order_id`, `energy_amount` ที่ clear, `price`, `fee_amount/net_amount`, `wheeling_charge`, `loss_factor`, `erc_certificate_id`, `blockchain_tx` และ settlement timestamp (`created_at/confirmed_at`) ทั้งนี้สถานะ escrow ถูกเก็บแยกเป็นบัญชี PDA ต่างหาก (SPL token account ภายใต้ seed `escrow`) ไม่ได้อยู่ในตัว settlement record

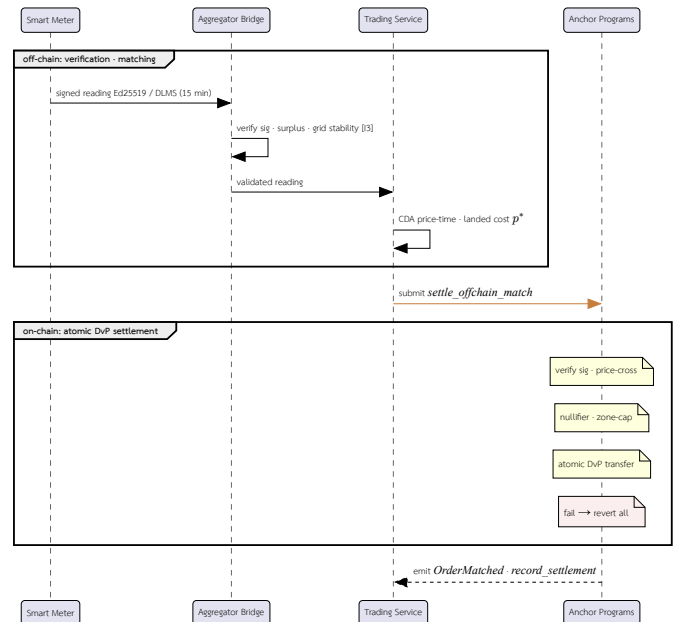
ความสามารถในการตรวจสอบย้อนหลัง (auditability) ที่ทำให้บล็อกเชนทำหน้าที่เป็นชั้น audit มาจาก Event log ที่โปรแกรมปล่อยในทุกขั้นตอนสำคัญของวงจรการซื้อขาย ได้แก่ การสร้างคำสั่ง (SellOrderCreated, BuyOrderCreated), การจับคู่และการชำระผ่านเหตุการณ์ OrderMatched ที่บันทึกคำสั่ง ผู้ซื้อ-ผู้ขาย ปริมาณ ราคา มูลค่ารวม และค่าธรรมเนียม โดยปล่อยจากทั้ง `settle_offchain_match` และ `batch_settle_offchain_match`, การยกเลิกคำสั่ง (OrderCancelled), การฝากหลักประกัน (EscrowDeposited) และการเคลียร์ตลาด (AuctionCleared) ส่วนการบันทึกการชำระแบบกลุ่มในชั้น treasury ปล่อยเหตุการณ์ SettlementBatchRecorded ที่ถูกรวมเมอร์เคิลของกลุ่มไว้บนเชน เหตุการณ์เหล่านี้เป็นบันทึกที่ไม่ถูกแก้ไขย้อนหลัง (append-only) ซึ่งผู้ตรวจสอบภายนอกใช้ประกอบประวัติธุรกรรมที่ตรวจทานได้โดยไม่ต้องเชื่อถือชั้น off-chain โดยตรง สอดคล้องกับบทบาท settlement and audit layer ที่กล่าวไว้ในหัวข้อ IV

ขอบเขตความรับผิดชอบถูกกำหนดให้ชัดเจนดังนี้ Backend และ Aggregator Bridge เป็นผู้ประเมินข้อมูลกำลังผลิตและการใช้ไฟฟ้า เงื่อนไข Grid stability, Islanding safety, ข้อจำกัดของโครงข่าย และเงื่อนไขปริมาณ kWh ที่ไม่เกินค่ารับรอง (available_surplus) จากนั้นจึงปล่อยให้คำสั่งที่ผ่านเงื่อนไขถูกส่งไปยังขั้นตอน settlement บนเชน ส่วน Anchor program ตรวจสอบเฉพาะ account ownership, signer, ลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขาย บน order payload, timestamp validity (expires_at), การกันส่งซ้ำผ่านบัญชี order nullifier, เงื่อนไข slippage/zone-capacity, สถานะ escrow และสถานะ order/trade ที่ยังไม่ถูกใช้ซ้ำ กล่าวคือเงื่อนไขด้านปริมาณพลังงานและ oracle attestation ถูกบังคับใช้ในชั้น off-chain ก่อนการ submit ไม่ได้ตรวจสอบข้ามเชน ดังนั้นบล็อกเชนในต้นแบบนี้ทำหน้าที่เป็น settlement and audit layer ไม่ใช่ตัวคำนวณ power-flow หรือ grid-stability engine แบบเต็มรูปแบบ

G. Trade Lifecycle Sequence

ลำดับการซื้อขายถูกแบ่งขอบเขตการทำงานระหว่าง off-chain และ on-chain อย่างชัดเจน ดังรูปที่ 8 ฝั่ง off-chain รับผิดชอบการรวบรวมข้อมูล Smart Meter การยืนยันตัวตน การตรวจสอบเวลาอ้างอิง การประเมินเงื่อนไข Grid stability และการคำนวณคำสั่งที่เสนอให้ clear ส่วนฝั่ง on-chain รับผิดชอบเฉพาะการยืนยัน account state, ลายเซ็น Ed25519 ของผู้ซื้อและผู้ขายบน order payload ที่ลงนามไว้, bounded matched pair, escrow และการบันทึก settlement event

กระบวนการนี้ช่วยให้ธุรกรรมพลังงานมีความโปร่งใสและตรวจสอบย้อนหลังได้ พร้อมลดความเสี่ยงจากการนำข้อมูล Smart Meter ที่ยังไม่ผ่านการยืนยันเข้าสู่บล็อกเชนโดยตรง การแยก boundary ระหว่าง off-chain verification และ on-chain settlement จึงเป็นสำคัญในการรักษาทั้งประสิทธิภาพของระบบและความปลอดภัยของไมโครกริด ข้อจำกัดของแนวทางนี้คือผู้ใช้ต้องเชื่อถือ Aggregator Bridge และ governance ของ oracle_authority หากต้องการลด trust เพิ่มเดิมควรเพิ่ม multi-oracle attestation หรือหลักฐาน cryptographic proof ในอนาคต



รูปที่ 8. Trade lifecycle as a sequence diagram: off-chain participants (Smart Meter → Aggregator Bridge → Trading Service) verify and match, then submit to the on-chain Anchor Programs (via the Chain Bridge) which settle and audit. Notes tag the on-chain checks enforced during settlement (อธิบายในหัวข้อ IV); any failed CPI reverts the whole atomic DvP settlement.

VII. EXPERIMENTAL SETUP

ส่วนนี้สรุปรายละเอียดการพัฒนา (implementation) เวอร์ชันของสิ่งประดิษฐ์ (artifact) ตัวแปรของภาระงาน (workload parameters) และนิยามของตัวชี้วัด เพื่อให้การประเมินในหัวข้อ VIII ทำซ้ำได้ ทั้งนี้ผู้วิจัยระบุข้อจำกัดด้านการทำซ้ำที่ยังเหลืออยู่ไว้อย่างตรงไปตรงมาในตอนท้ายของส่วนนี้

A. Implementation and Artifact

บริการฝั่ง Backend ทุกตัวพัฒนาด้วยภาษา Rust (Axum) บน Tokio async runtime และสื่อสารระหว่างกันผ่าน ConnectRPC บน mTLS ส่วนเส้นทางเขียนข้อมูลลงบล็อกเชนแยกออกผ่าน NATS JetStream [24] ส่วนชุดจำลองโครงข่ายและ Smart Meter พัฒนาด้วย Python 3.11 [28] (รายละเอียดเครื่องมือในหัวข้อ VI.C) ระบบทั้งหมดจัดเรียงเป็น superproject ที่รวมแต่ละบริการเป็น git submodule ทำให้ระบุเวอร์ชันของสิ่งประดิษฐ์ได้จาก commit ของ superproject ร่วมกับ pointer ของแต่ละ submodule บทความนี้ระบุเวอร์ชันของสิ่งประดิษฐ์และการทดลอง มิใช่เป็น pin เดียวสำหรับทั้งบทความ เพราะการวัดแต่ละชุดเกิดขึ้นคนละจุดบนสายการพัฒนาเดียวกัน การทดลองเส้นทาง telemetry ingest (หัวข้อ VIII.B และหัวข้อ VIII.C) อ้างอิงสถานะของ superproject ที่ commit `4b05661` ซึ่งตรง pointer ของบริการหลัก ได้แก่ Aggregator Bridge, Chain Bridge, Anchor programs, blockchain-core และ Smart Meter Simulator ไว้อย่างเฉพาะเจาะจง ส่วนการวัดต้นทุนและปริมาณงานบนเชน (หัวข้อ VIII.E, หัวข้อ VIII.F และหัวข้อ VIII.G) ดำเนินการบนชุดเบสซิมาร์กของ gridtokenx-anchor ที่ commit `58cfc79` ซึ่งเป็นบรรพบุรุษ (ancestor) ของ pointer ที่ superproject `4b05661` ตรงไว้ ขณะที่การทดลองระดับฟลิตทั้ง live mint และการเขียน telemetry บนเชน (หัวข้อ VIII.H) อยู่หลัง `4b05661` โดยจำเป็น เพราะวัดเส้นทางโค้ดที่เพิ่งเพิ่มหรือปรับจูนระหว่างการรันเหล่านั้นเอง ได้แก่ การเพิ่ม concurrency ของ mint consumer งบเวลารอตอบกลับของ aggregator และการแก้ label ของ request metrics ที่รายงานไว้ในหัวข้อดังกล่าว จึงอ้างอิงสถานะของ superproject ถึง commit `cdd9d00` การทำซ้ำตัวเลขได้ตัวเลขหนึ่งจึงต้องใช้ commit ที่ระบุไว้กับการทดลองนั้น มิใช่ pin เดียวของทั้งบทความ

B. Topology and Endpoints

ในเส้นทางที่ประเมิน Smart Meter Simulator ส่งค่าอ่านเข้าสู่ Aggregator Bridge ผ่าน IoT gateway (HTTP) และช่องทาง gRPC สำหรับ ingest จากนั้น Aggregator Bridge ตรวจลายเซ็นและกระจายค่าอ่านที่ผ่านการตรวจสอบเข้าสู่ Redis Streams ที่แบ่งตามโซน ส่วนเส้นทางเขียนเชนถูกส่งผ่าน Chain Bridge ด้วยหัวข้อ NATS ตระกูล `chain.tx.*` (เช่น `chain.tx.submit` และ `chain.tx.mint`) การแยกพอร์ตและช่องทางในลักษณะนี้ทำให้เส้นทางรับข้อมูล (ingest) ขยายขนาดได้อิสระจากเส้นทาง settlement

C. Deployed Components Outside the Measured Scope

เส้นทางที่ประเมินข้างต้นเป็นเพียงส่วนหนึ่งของระบบที่ติดตั้งใช้งานจริง เพื่อให้อ่านผลในหัวข้อ VIII ได้ตรงตามขอบเขต บทความนี้ระบุอย่างชัดเจนว่าองค์ประกอบใดที่รันอยู่แต่อยู่นอกขอบเขตการวัด การติดตั้งจริงยังประกอบด้วย API gateway (APISIX) เป็นทางเข้าฝั่งผู้ใช้ (เส้นทาง ingest ที่วัดข้ามส่วนนี้ไปและเรียก IoT gateway ของ Aggregator Bridge โดยตรง), แกนกลางเหตุการณ์ Kafka สามโบรกเกอร์ที่รับส่งเหตุการณ์ตัวต้นผู้ใช้ read model ของเจ้าของมาตรวัด และสายงานการแจ้งเตือน, บริการ meter-service ที่เป็นเจ้าของการลงทะเบียนและทะเบียนมาตรวัด, การแยกฐานข้อมูลรายบริการ (แต่ละบริการเป็นเจ้าของ schema ของตนเองผ่านตัวจัดเส้นทางเชื่อมต่อ) แทนฐานข้อมูลร่วมหนึ่งชุด, Chain Bridge สองเรพลิเคชันที่ใช้กลไกกันงานซ้ำร่วมกันเพื่อความ idempotent ของการส่งธุรกรรม, mint outbox แบบคงทนที่ส่งซ้ำจนกว่าเชนจะยืนยัน, ชุดสังเกตการณ์ระบบ (metrics/log/trace) และเว็บฟรอนต์เอนด์สามตัว (ส่วนติดต่อการซื้อขาย, block explorer และส่วนติดต่อของชุดจำลอง) องค์ประกอบเหล่านี้ไม่ถูกใช้เป็นตัวแปรในการทดลองใดของบทความนี้และไม่มีผลการรายงานผล อย่างไรก็ตามสององค์ประกอบมีผลต่อตัวเลขที่รายงานไว้และถูกอ้างอิงตรงจุดที่เกี่ยวข้อง กล่าวคือ mint outbox แบบคงทนคือกลไกที่พาส่วนทางที่ล่าช้าในตารางที่ XVI ให้สำเร็จครบ และชุดสังเกตการณ์ระบบคือจุดที่วินิจฉัย

เหตุ metrics cardinality ในหัวข้อ VIII.H การวัด gateway แกนกลางเหตุการณ์ และเส้นทางหลายฐานข้อมูล เป็นงานในอนาคต (หัวข้อ IX)

D. Workload Parameters

ภาระงานในการประเมินกำหนดด้วย Smart Meter จำนวน 80 เครื่อง ($NUM_METERS=80$) ที่ส่งค่าอ่านทุก 15 วินาทีของเวลาจำลอง ($SIMULATION_INTERVAL=15$) ตามค่าตั้งต้นของชุดจำลอง รอบการส่งทุก 15 วินาทีของเวลาจำลองสอดคล้องกับการบีบอัดเวลา (time compression) ประมาณ 60 เท่าเมื่อเทียบกับหน้าตาเวลา 15 นาที (900 วินาที) ของรอบการส่งข้อมูลของมาตรวัดจริง ในที่นี้นิยาม “ค่าอ่าน” (reading) ที่ใช้เป็นหน่วยวัดปริมาณงานว่าหมายถึงค่าอ่านมาตรวัดหนึ่งรายการที่ลงนามด้วย Ed25519 แล้วผ่านการตรวจสอบลายเซ็นและถูกกระจายเข้าสู่ Redis Stream ที่ Aggregator Bridge มิใช่ค่าส่งซื้อขายที่จับคู่แล้วหรือธุรกรรม settlement บนบล็อกเชน ตัวแปรหลักของภาระงานสรุปไว้ในตารางที่ IX

ตารางที่ IX
Workload parameters for the telemetry-ingest evaluation.

พารามิเตอร์	ค่า	ความหมาย
NUM_METERS	80	จำนวน Smart Meter ในการจำลอง
SIMULATION_INTERVAL	15 s	รอบการส่งค่าอ่านในเวลาจำลอง
Time compression	$\approx 60 \times$	เทียบกับหน้าตาจริง 900 s
Nominal ingest rate	5.33 readings/s	$80 \div 15$ ในเวลาจำลอง
Run duration	≈ 27 min	เวลาการรันจริงของการรันหนึ่งรอบ
Total readings	26,240	ตรวจลายเซ็นสำเร็จ ไม่มีการสูญหาย

E. Metrics

ตัวชี้วัดหลักของการประเมินเส้นทางรับข้อมูลคืออัตราการรับข้อมูล (ingest rate) ในสองนิยาม ได้แก่ อัตราเชิงออกแบบในเวลาจำลอง (nominal) เท่ากับ $80 \div 15 = 5.33$ รายการต่อวินาที และอัตราที่วัดจากเวลาานาฬิกาจริง (wall-clock) ซึ่งสูงกว่าเนื่องจากรอบการส่งของ simulator ถูกเร่งให้เร็วกว่าช่วง 15 วินาทีของเวลาจำลอง ร่วมกับจำนวนค่าอ่านสะสมที่ตรวจสอบสำเร็จและสัดส่วนการสูญหายของข้อมูล รายละเอียดผลการวัดอยู่ในหัวข้อ VIII.B

F. Reproducibility Limitations

การทดลองเส้นทางรับข้อมูลแบบรันยาวในหัวข้อ VIII.B เป็นการรันจริงเพียงรอบเดียว ส่วนการทดลองเพิ่มภาระงานแบบขั้นบันไดในหัวข้อ VIII.C ทำซ้ำ 5 รอบต่อขนาดฟลิตและรายงานค่าเฉลี่ยพร้อมส่วนเบี่ยงเบนมาตรฐานบนเครื่อง Apple M2 (8 cores, หน่วยความจำ 16 GiB) ที่บันทึกไว้ ข้อจำกัดด้านการทำซ้ำที่ยังเหลืออยู่คือ ภาระงานถูกสร้างจากโคลเอนด์ผู้ส่งเพียงตัวเดียว (ยังไม่ได้ทดสอบผู้ส่งแบบขนาน) ตัวเลขอัตราการรับข้อมูลที่รายงานจึงเป็น lower bound ของเส้นทางรับข้อมูลมากกว่าขอบเขตความสามารถสูงสุด และการวัดเส้นทาง settlement บนเชนยังจำกัดอยู่ที่ต้นทุน compute-unit ต่อคำสั่ง โดยยังไม่ครอบคลุม latency และ throughput แบบ end-to-end การทดลองถัดไปจึงควรเพิ่มผู้ส่งแบบขนานเพื่อหาเพดานที่แท้จริง บันทึกการตั้งค่า validator และขยายการวัดให้ครอบคลุมเส้นทาง settlement บนเชนแบบ end-to-end (ดูหัวข้อ IX)

VIII. EVALUATION

ส่วนนี้นำเสนอการประเมินเชิงสถาปัตยกรรมของระบบจำลอง ครอบคลุมอัตราการรับข้อมูล (telemetry ingest), อัตราการจับคู่คำสั่งในเชน off-chain และต้นทุนกับปริมาณงานของการชำระธุรกรรมบนเชน เพื่อสะท้อนความเหมาะสมของสถาปัตยกรรมสำหรับงานซื้อขายพลังงานแบบ Peer-to-Peer ในระดับไมโครกริด ในมิติด้าน Performance ส่วนนี้รายงานผลการวัดอัตราการรับข้อมูลของเส้นทาง telemetry ingest ทั้งจากการรันจริงหนึ่งรอบ (ดูหัวข้อ VIII.B) และจากการทดลองเพิ่มภาระงานแบบขั้นบันไดเพื่อหาขอบเขตการขยายตัวและสัดส่วนการสูญหาย (ดูหัวข้อ VIII.C) นอกจากนี้ยังรายงานต้นทุนการประมวลผลบนเชนของเส้นทาง settlement โดยตรงในรูปของ compute-unit ต่อการชำระหนึ่งคู่คำสั่ง (ดูหัวข้อ VIII.E) อีกทั้งยังรายงานปริมาณงานธุรกรรมบนเชนด้วยชุด workload มาตรฐานฐานข้อมูล (Blockbench, TPC-C และ

SmallBank ที่ฟอร์ตมาบนเครือข่าย Solana-compatible) ร่วมกับการวัดเส้นทางการชำระจริงในหัวข้อ VIII.F ทั้งนี้การประเมินเป็นการบ่งชี้คุณลักษณะ (characterization) ของระบบเดียว มิใช่การเปรียบเทียบเชิงปริมาณแบบ head-to-head กับแพลตฟอร์มอื่น ส่วนการวัด latency และ throughput แบบ end-to-end ของเส้นทาง settlement รวมถึงผลทดสอบ grid-stability model ในระดับภาคสนาม ยังเป็นงานในอนาคต

A. Performance

ผลการออกแบบชี้ให้เห็นว่าสถาปัตยกรรมแบบ Microservice สามารถลดการผูกติดกันของภาระงานระหว่างการรับข้อมูล Smart Meter การตรวจสอบคำสั่งซื้อขาย และการส่งธุรกรรมไปยังบล็อกเชนในเชิงโครงสร้างได้ การแยก aggregator-bridge, trading-service, chain-bridge และ settlement-engine ออกจากกันทำให้แต่ละบริการมีจุดปรับขนาดที่ชัดเจนตามประเภทภาระงาน โดยเฉพาะช่วงที่มีข้อมูลจาก Smart Meter จำนวนมากหรือมีคำสั่งซื้อขายเกิดขึ้นพร้อมกัน

ในเชิงการทำงานแบบเวลาจริง ระบบใช้ ConnectRPC และ NATS JetStream [24] เพื่อแยกงานที่ต้องตอบสนองเร็วออกจากงานที่ต้องรอการยืนยันธุรกรรมบนบล็อกเชน ดังนั้น Backend Service จึงถูกออกแบบให้ตอบสนองต่อคำขอของผู้ใช้และ Smart Meter ได้โดยไม่ต้องรอให้ธุรกรรมบน Solana-compatible network เสร็จสมบูรณ์ทันที แนวทางนี้คาดว่าจะลดผลกระทบจากความหน่วงของเครือข่ายบล็อกเชน แต่ยังคงทดสอบด้วย workload ที่กำหนดจำนวน Meter, sampling interval, queue depth และ transaction mix อย่างชัดเจนในงานอนาคต

B. Telemetry Ingest Throughput

เพื่อประเมินความสามารถของเส้นทางรับข้อมูลภายใต้ภาระงานจำนวนมาก ผู้วิจัยทำการจำลองด้วยภาระงานตามตารางที่ IX (Smart Meter 80 เครื่อง ส่งค่าอ่านทุก 15 วินาทีของเวลาจำลอง) โดยใช้นิยาม “ค่าอ่าน” ที่เป็นหน่วยวัดปริมาณงานตามหัวข้อ VII (ค่าอ่านที่ลงนาม Ed25519 ผ่านการตรวจสอบลายเซ็นและถูกกระจายเข้าสู่ Redis Stream แล้ว มีใช้คำสั่งจับคู่หรือธุรกรรม settlement บนเชน)

อัตราการรับข้อมูลเชิงออกแบบ (nominal ingest rate) ในเวลาจำลองเท่ากับ $80 \div 15 = 5.33$ รายการต่อวินาที ซึ่งสอดคล้องกับการบีบอัดเวลา (time compression) ประมาณ 60 เท่าเมื่อเทียบกับรอบการส่งข้อมูลของมาตรวัดจริงที่ใช้หน้าตาเวลา 15 นาที (900 วินาที) จากการรันจริงหนึ่งรอบเป็นเวลาประมาณ 27 นาที Aggregator Bridge รับและตรวจสอบลายเซ็นค่าอ่านได้ทั้งสิ้น 26,240 รายการจากมาตรวัด 80 เครื่องอย่างต่อเนื่องโดยไม่มีการสูญหายของข้อมูล โดยอัตราที่วัดได้จากเวลาณฬิการจริง (wall-clock) อยู่ที่ประมาณ 16 รายการต่อวินาที ซึ่งสูงกว่าอัตราเชิงออกแบบเนื่องจากรอบการส่งข้อมูลของ simulator ถูกเร่งให้เร็วกว่าช่วงเวลา 15 นาทีของเวลาจำลอง

ผลนี้แสดงให้เห็นว่าเส้นทางรับข้อมูล (ingest path) สามารถรองรับภาระการส่งข้อมูลของมาตรวัด 80 เครื่องได้อย่างเสถียร อย่างไรก็ตาม การวัดนี้ครอบคลุมเฉพาะเส้นทางรับและตรวจสอบข้อมูลมาตรวัดเท่านั้น ยังไม่รวมการวัดปริมาณงานและความหน่วงของเส้นทาง settlement บนเชน เช่น throughput และ latency ของการจับคู่คำสั่งและการบันทึกธุรกรรมบน Smart Contract นอกจากนี้การทดลองนี้ยังไม่ได้บันทึกข้อมูลฮาร์ดแวร์และโหมดของ validator ดังนั้นการทดลองถัดไปจึงควรกำหนด workload, hardware profile และ validator configuration ที่ทำซ้ำได้ พร้อมวัดเส้นทาง settlement แบบ end-to-end

การหาขอบเขตการขยายตัวและสัดส่วนการสูญหายภายใต้ภาระงานแบบขึ้นบันไดรายงานต่อในหัวข้อ VIII.C

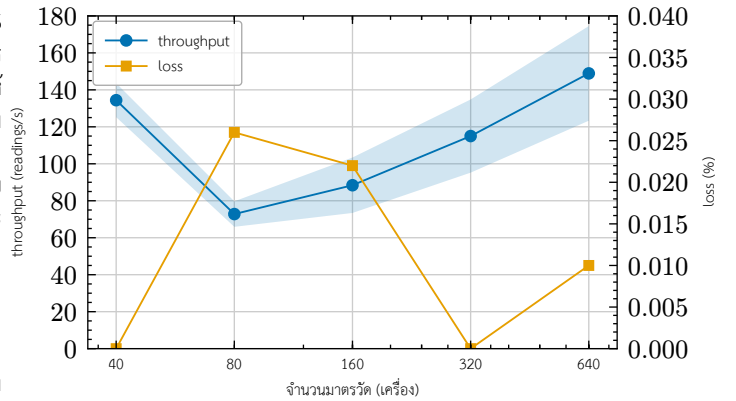
C. Ingest Scaling and Loss Under Load

เพื่อประเมินขอบเขตความสามารถของเส้นทางรับข้อมูลให้กว้างกว่าอัตราเชิงออกแบบ 5.33 รายการต่อวินาทีตามหัวข้อ VIII.B ผู้วิจัยเพิ่มภาระงานแบบขึ้นบันได (load ramp) ด้วยจำนวนมาตรวัด 40, 80, 160, 320 และ 640 เครื่อง โดยแต่ละขั้นรันเป็นเวลา 60 วินาที และทำซ้ำ 5 รอบเพื่อรายงานค่า

เฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน ในแต่ละรอบ simulator ส่งค่าอ่านที่ลงนาม Ed25519 แบบต่อเนื่องสูงสุด (ไม่มีการหน่วงระหว่างรอบส่ง) ตัวชี้วัดหลักวัดที่ขอบเขต HTTP คือสัดส่วนคำขอที่ Aggregator Bridge รับ ตรวจสอบลายเซ็น และกระจายเข้าสู่ Redis Stream สำเร็จ (HTTP 200) โดยนิยาม throughput = จำนวนรายการที่สำเร็จ ÷ เวลาที่ใช้ และ loss = จำนวนคำขอที่ล้มเหลว ÷ คำขอทั้งหมด ผลสรุปไว้ในตารางที่ X และแสดงแนวโน้มเป็นกราฟในรูปที่ 9 การทดลองรันบนเครื่อง Apple M2 (8 cores, หน่วยความจำ 16 GiB)

ตารางที่ X
Ingest throughput and loss vs. meter-fleet size (mean $\pm$ sd over 5 runs, 60 s each).

มาตรวัด (เครื่อง)	throughput (readings/s)	loss (สูงสุด)
40	134.4 $\pm$ 9.2	0
80	72.8 $\pm$ 6.9	2.6×10^{-4}
160	88.4 $\pm$ 15.0	2.2×10^{-4}
320	115.0 $\pm$ 19.8	0
640	148.9 $\pm$ 25.6	1.0×10^{-4}



รูปที่ 9. Ingest throughput (left axis, mean $\pm$ sd) and request loss (right axis) vs. meter-fleet size; loss stays $\leq 0.03\%$ across the tested range. Throughput is non-monotonic and single-client sender-bound — a lower bound, not a service-saturation curve.

ผลการวัดมีสองข้อสังเกตหลัก ข้อแรกคือสัดส่วนการสูญหายของข้อมูลอยู่ในระดับใกล้เคียงศูนย์ตลอดทุกขนาดฟลีต โดยค่าสูงสุดที่วัดได้คือประมาณ 2.6×10^{-4} (คิดเป็นประมาณ 0.03%) ที่ขนาด 80 เครื่อง และหลายรอบไม่มีการสูญหายเลย ผลนี้บ่งชี้ว่าเส้นทางรับข้อมูลรักษาสัดส่วนการสูญหายให้ใกล้ศูนย์ ($\leq 0.03\%$) ได้จนถึงภาระงาน 640 เครื่อง ข้อที่สองคืออัตราการรับข้อมูลที่วัดได้ไม่ได้อิงขึ้นแบบเอกฐานตามจำนวนมาตรวัด แต่แกว่งอยู่ในช่วงประมาณ 73–149 รายการต่อวินาที (ค่าเฉลี่ย) โดยมีส่วนเบี่ยงเบนมาตรฐานค่อนข้างสูง (สูงสุดประมาณ 26 รายการต่อวินาทีที่ 640 เครื่อง) เนื่องจากการทดลองนี้สร้างภาระงานจากไคลเอนต์ผู้ส่งเพียงตัวเดียวที่ทำงานแบบ asynchronous และส่งค่าอ่านรายการมาตรวัดต่อรอบตามลำดับ ความแกว่งและความไม่เป็นเอกฐานนี้จึงน่าจะสะท้อนข้อจำกัดฝั่งผู้ส่ง (เช่น ต้นทุนการสร้างและลงนามค่าอ่าน และการประมวลผลแบบ single-client) ร่วมกับจังหวะการกระจายแบบ asynchronous เข้าสู่ Redis มากกว่าการอิมิตัวของบริการรับข้อมูล ทั้งนี้ผู้วิจัยไม่สรุปสาเหตุที่แน่ชัดของรูปแบบนี้จากข้อมูลที่มี

จากผลที่ได้ อัตราเฉลี่ยสูงสุดที่วัดได้ (ค่าเฉลี่ย 5 รอบที่ฟลีต 640 เครื่อง) อยู่ที่ประมาณ 149 รายการต่อวินาที ซึ่งสูงกว่าอัตราเชิงออกแบบ 5.33 รายการต่อวินาทีหลายเท่าตัว และเกิดขึ้นที่ขนาดฟลีตใหญ่ที่สุดที่ทดสอบ (640 เครื่อง) อย่างไรก็ตาม อัตราที่วัดได้ไม่เพิ่มขึ้นแบบเอกฐานตามจำนวนมาตรวัด (ค่าเฉลี่ยที่ 80 เครื่องต่ำกว่าที่ 40 เครื่อง) ผู้วิจัยจึงไม่สรุปว่าเส้นทางรับข้อมูลถึงหรือไม่ถึงจุดอิ่มตัวจากข้อมูลชุดนี้ อย่างไรก็ตาม เนื่องจากภาระงานถูกสร้างจากไคลเอนต์ผู้ส่งเพียงตัวเดียวแบบส่งต่อเนื่องรายการมาตรวัดต่อรอบ (sequential per tick) ตัวเลขนี้จึงเป็น lower bound ที่สะท้อนว่า Aggregator Bridge รองรับไคลเอนต์ที่ส่งเต็มอัตราหนึ่งตัวได้โดยไม่มีการสูญหายใกล้ศูนย์ มากกว่าจะเป็นเพดานความสามารถที่แท้จริงของเส้นทางรับข้อมูล การวัดเพดานที่แท้จริงต้องใช้ผู้ส่งแบบขนานหลายตัวและแยกต้นทุนฝั่งผู้ส่ง (เช่น การ onboard และการลงนาม) ออกจากต้นทุนของบริการรับข้อมูล ซึ่งยังเป็นงานในอนาคต อนึ่ง อัตราในการทดลอง ramp นี้ (73–149 รายการต่อวินาที) สูงกว่าอัตราเวลาณฬิการจริงประมาณ 16 รายการต่อวินาทีของการรันต่อเนื่องหนึ่งรอบในหัวข้อ VIII.B เนื่อง

จากการทดลอง ramp ส่งแบบไม่หน่วงระหว่างรอบ (max-rate) เพื่อหาขอบเขตการขยายตัว ขณะที่การรันต่อเนื่องผูกจังหวะการส่งกับช่วงเวลา 15 วินาทีของเวลาจำลอง

D. Off-chain Matching Throughput

นอกเหนือจากเส้นทางรับข้อมูล ผู้วิจัยวัดต้นทุนการจับคู่คำสั่งของเครื่องจับคู่ Continuous Double Auction (CDA) ในชั้น off-chain ด้วย micro-benchmark (Criterion) ที่เรียกฟังก์ชันจับคู่หนึ่งรอบ (match cycle) เหนือสมุดคำสั่งซื้อ 1,000 รายการและคำสั่งขาย 1,000 รายการ (รวม 2,000 คำสั่ง) โดยกำหนดให้ราคาคำสั่งซื้อสูงกว่าคำสั่งขายทุกคู่ จึงจับคู่ได้เพิ่มเป็นคู่คำสั่งสูงสุดประมาณ 1,000 คู่ต่อรอบ การทดลองรันบนเครื่องเดียวกัน (Apple M2, 8 cores, หน่วยความจำ 16 GiB) เก็บ 100 ตัวอย่างหลังช่วง warm-up ผลสรุปในตารางที่ XI

ตารางที่ XI
In-memory CDA matching throughput (one match cycle over 1,000 bids × 1,000 asks, 100 samples).

ตัวชี้วัด	ค่า
เวลาต่อรอบจับคู่ 1,000 × 1,000 (median)	32.56 ms
ช่วงความเชื่อมั่น 95%	32.28–32.75 ms
อัตราประมวลผลคำสั่ง (≈)	6.1×10^4 orders/s
อัตราจับคู่คำสั่ง (≈)	3.1×10^4 pairs/s

เครื่องจับคู่ประมวลผลคำสั่งทั้งสองฝั่งหนึ่งรอบเต็มในเวลามัธยฐาน (median) 32.56 มิลลิวินาที (ช่วงความเชื่อมั่น 95% ประมาณ 32.28–32.75 มิลลิวินาที) คิดเป็นอัตราการประมวลผลประมาณ 6.1×10^4 คำสั่งต่อวินาที หรือประมาณ 3.1×10^4 คู่คำสั่งจับคู่ต่อวินาทีบนเซิร์ฟเวอร์เดียว ค่านี้วัดเฉพาะการจับคู่ในหน่วยความจำ (in-memory matching) ไม่รวมต้นทุนการชำระธุรกรรมบนเชน การคงสภาพข้อมูล (persistence) หรือการสื่อสารผ่านเครือข่าย จึงเป็นขอบเขตบน (upper bound) ของอัตราการจับคู่ที่แยกออกจากต้นทุน settlement อย่างชัดเจน ผลนี้บ่งชี้ว่าในสถาปัตยกรรมแบบแยกชั้น การจับคู่ในชั้น off-chain มีใช้คอขวดของระบบเมื่อเทียบกับต้นทุนการชำระธุรกรรมบนเชนในหัวข้อ VIII.E ทั้งนี้การจับคู่แบบ single-cycle 1,000 × 1,000 เป็นภาระงานสังเคราะห์เพื่อวัดต้นทุนต่อรอบ มิใช่อัตราคงตัวภายใต้กระแสคำสั่งจริงที่มีการกระจายราคาและโซนหลากหลาย ซึ่งเป็นงานในลำดับถัดไป

E. On-chain Settlement Cost

นอกเหนือจากเส้นทางรับข้อมูล ผู้วิจัยวัดต้นทุนการประมวลผลบนเชนของการชำระธุรกรรมโดยตรง โดยรายงานหน่วยประมวลผล (compute units, CU) ที่คำสั่งชำระแบบจับคู่บนเชน (off-chain match) ใช้จริง อ่านจากค่า *computeUnitsConsumed* ของธุรกรรมที่ยืนยันแล้วในการทดสอบ escrow settlement บน solana-test-validator 3.1.10 (Agave client) ค่านี้เป็นตัวชี้วัดต้นทุนบนเชนที่กำหนดได้แน่นอน (deterministic) และไม่ขึ้นกับฮาร์ดแวร์ของ validator ต่างจากความหน่วงบนเครือข่ายทดสอบเฉพาะที่ ซึ่งไม่สื่อถึงเครือข่ายเป้าหมายเชิงออกแบบ ผลสรุปในตารางที่ XII

ตารางที่ XII
Measured compute-unit cost of the settlement instruction (1 matched order pair).

คำสั่ง (instruction)	compute units
settlement ต่อ 1 คู่คำสั่ง	96,707

ค่า 96,707 CU ต่อการชำระหนึ่งคู่คำสั่ง¹ เทียบกับงบประมาณ compute ตั้งต้นต่อคำสั่งที่ 200,000 CU และเพดานสูงสุดต่อธุรกรรมที่ 1,400,000 CU บ่งชี้ว่าการชำระแต่ละครั้งใช้ทรัพยากรประมาณ 48% ของงบตั้งต้น อนึ่ง แม้เพดาน compute สูงสุดต่อธุรกรรมจะรองรับได้ในเชิงทฤษฎีราว 14 คู่ต่อธุรกรรม แต่ข้อจำกัดที่บังคับจริง (binding constraint) มิใช่ compute แต่เป็นขนาดแพ็กเก็ตธุรกรรม 1,232 ไบต์ ซึ่งเป็นจำนวนคู่ต่อธุรกรรมให้ต่ำกว่าเพดาน 4 คู่ที่โปรแกรม *batch_settle_offchain_match* อนุญาต (ดูหัวข้อ IV.E) ดังนั้นการลดต้นทุนต่อคู่ด้วย batch settlement จึงต้องเปลี่ยนวิธีบรรจุก่อนเข้า มิใช่

อาศัยงบ compute ที่เหลือเพียงอย่างเดียว ทั้งนี้ระดับต้นทุน 48% ต่อคำสั่งยังคงสอดคล้องกับการออกแบบให้บล็อกเชนทำหน้าที่เป็นชั้น settlement ที่ใช้ compute ต่ำตามหัวข้อ VI

F. ปริมาณงานธุรกรรมบนเชน (On-chain Transaction Throughput)

นอกเหนือจากต้นทุน compute ต่อคำสั่งในหัวข้อ VIII.E ผู้วิจัยวัดปริมาณงาน (throughput) ของการประมวลผลธุรกรรมบนเชนด้วยชุดภาระงาน OLTP มาตรฐานที่พอร์ตมาสู่ account model ของ Solana ได้แก่ BlockBench (ไม่โครเบนช์มาร์กแบบ SIGMOD 2017 ร่วมกับ YCSB), SmallBank และ TPC-C เสริมด้วยการวัดเส้นทางการชำระจริง (settle_offchain_match) ทั้งหมดรันบน solana-test-validator เดียว (single-node; เครือข่าย permissioned ที่กำกับการเข้าร่วมแบบ PoA) บนเครื่อง Apple M2 (8 cores) ที่ commit *58cfc79* ของ gridtokenx-anchor โดยความหน่วงเป็นเวลานาฬิกาจริงแบบ client→confirmed และหน่วย compute อ่านจาก *computeUnitsConsumed* ของธุรกรรมที่ยืนยันแล้ว ผลภาระงาน OLTP แบบลำดับสรุปในตารางที่ XIII และผลการกวาดค่าระดับการทำงานพร้อมกัน (concurrency sweep) ของ TPC-C สรุปในตารางที่ XIV

ตารางที่ XIII

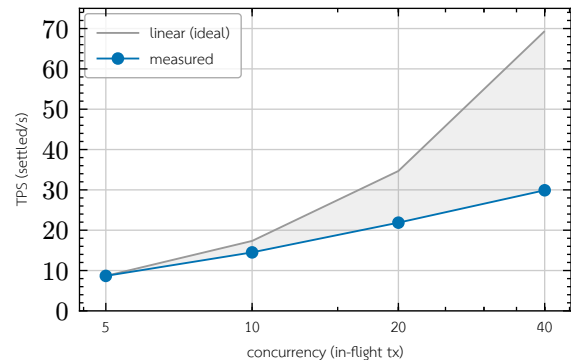
Standard OLTP workloads ported to the account model (sequential, n = 150). TPS here is latency-bound by the single-client submit loop, not a throughput ceiling; *ycsb_read* is an RPC account fetch with no consensus round-trip.

ภาระงาน (workload · op)	mean ms	TPS	CU/tx
BlockBench · do_nothing	719.95	1.39	648
BlockBench · cpu_heavy_sort	590.83	1.69	9,645
BlockBench · ycsb_read (RPC)	4.29	233.18	—
SmallBank · SendPayment	604.63	1.65	5,963
SmallBank · Amalgamate	631.62	1.58	5,936

ตารางที่ XIV

TPC-C concurrency sweep (TX = 500, 50% NewOrder / 50% Payment), 100% success at every level. Throughput from concurrent in-flight submission; CU/tx is flat across load.

concurrency	TPS	mean ms	p95 ms	CU/tx (mean)
5	8.67	575.00	726.10	21,263
10	14.50	679.34	879.95	20,633
20	21.87	856.74	1,656.15	21,269
40	29.90	1,057.79	1,707.00	21,508



รูปที่ 10. TPC-C throughput vs. concurrency on the single-node validator. Measured TPS scales sublinearly — 8x concurrency yields only 3.45x throughput — diverging from the linear ideal reference, with the saturation knee between concurrency 10 and 20.

ตัวเลขที่สำคัญที่สุดต่อระบบนี้คือปริมาณงานของเส้นทางการชำระจริง มิใช่ของ proxy ทั่วไป เมื่อวัดเส้นทาง *batch_settle_offchain_match* แบบ open-loop submission sweep (สร้างธุรกรรมล่วงหน้าแล้วยิงพร้อมกันตามระดับ concurrency และยิงชำระธุรกรรมที่ตกลงกันยืนยัน) ได้อัตราเพียง 0.5 TPS และคงที่ทุกระดับ concurrency (ตัวเลขส่วนนี้มาจากการรันบนทิกเฝ้ายรอบเดียว ซึ่งต่างจากชุด TPC-C/OLTP ที่ commit ผลดิบเป็น artifact ไร้จึงควรตีความเป็นค่าบ่งชี้เชิงออกแบบที่รบกวนวัดซ้ำ) (conc 5 → 0.51, conc 10 → 0.58 TPS; goodput 100%, ไม่มี revert บนเชน, CU ราว 86–89k) ทั้งยังไม่

¹ ต้นทุน compute ขึ้นกับรูปแบบการโอนเข้าเงินของธุรกรรม ค่าที่รายงานนี้เป็นค่าจากชุดทดสอบ escrow settlement หนึ่งรูปแบบ ส่วนเส้นทางที่มิใช่แลกเปลี่ยนแบบ classic-SPL สู่ Token-2022 วัดได้ราว 103k CU และการชำระแบบกลุ่มที่ใช้ Token-2022 ทั้งสองชุดวัดได้ราว 80–92k CU ตัวเลข 96,707 จึงเป็นค่าเฉพาะของรูปแบบที่วัด มิใช่ค่าคงที่สากล

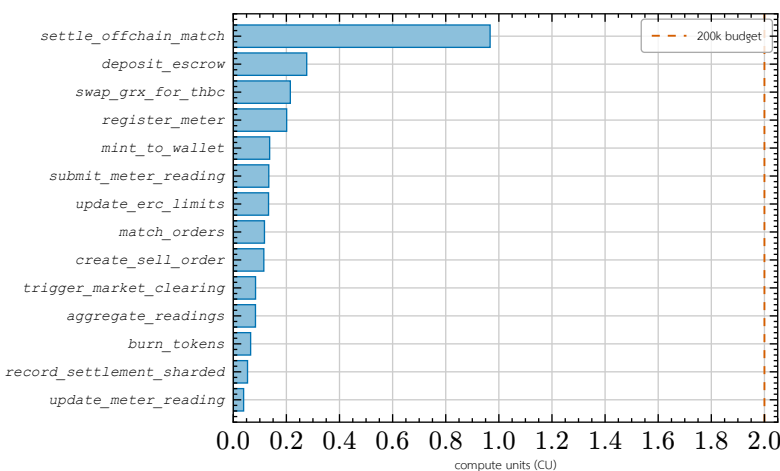
ขยายตามจำนวนธุรกรรมที่ส่งพร้อมกัน แม้กระจายการชำระข้ามทั้ง 16 ชาร์ตก็ได้ตัวเลขใกล้เคียง (0.57–0.59 TPS) เพราะคอขวดมิใช่ชาร์ต แต่เป็นชุดบัญชีส่วนกลางที่ทุกการชำระต้องเขียนเสมอ ได้แก่ บัญชีสะสมยอด *total_settled_thbc* และบัญชีผู้เก็บค่าธรรมเนียม/ค่าผ่านสาย/การสูญเสีย การชำระจึงถูกผูกกับการเขียนส่วนกลาง (global-write-bound) โดยการออกแบบ การแบ่งชาร์ตขนานได้เฉพาะการส่งคำสั่ง (order submission) ที่ใช้ PDA รายเอนทิตี มิใช่การกระทบยอดของการชำระ

ทั้งใน TPC-C sweep หน่วย compute ต่อธุรกรรมคงที่ราว 21,000 CU ทุกระดับ concurrency ซึ่งว่าคอขวดของปริมาณงานคือเวลาในการผลิตบล็อก (block time ราว 400–600 มิลลิวินาที) ร่วมกับการ serialize การเขียนบัญชีส่วนกลาง มิใช่การประมวลผลบนเซิร์ฟเวอร์หน่วย compute ต่อธุรกรรม (หัวข้อ VIII.E) จึงเป็นตัวชี้วัดต้นทุนที่ไม่ขึ้นกับภาระงานและเครื่อง ขณะที่ TPS เป็นตัวเลขที่ขึ้นกับ validator เดียวที่ใช้ทดสอบ การวาดค่ายังแสดงการขยายแบบ sublinear (concurrency 5 → 40 เพิ่มขึ้น 8 เท่า ให้ TPS เพิ่มขึ้นเพียง 3.45 เท่า) และจุดอิ่มตัว (saturation knee) ระหว่าง concurrency 10 ถึง 20 ที่ส่วนเบี่ยงเบนของความหน่วงและ p95 เพิ่มขึ้นชัดเจน ดังแสดงเทียบกับเส้นอ้างอิงเชิงเส้นในรูปที่ 10

เมื่อเทียบกับหน้าต่างการเคลียร์ตลาด 900 วินาที แม้อัตราการชำระจริง 0.5 TPS ยังรองรับได้ราว 450 การชำระต่อหน้าต่าง ซึ่งเกินความต้องการของภาระงาน 80 มาตราวัดในการประเมินนี้มาก ส่วนเพดาน proxy ที่ 29.9 TPS เทียบเท่า 2.69×10^4 ธุรกรรมต่อหน้าต่าง อย่างไรก็ตามผลทั้งหมดมาจาก validator เดียว จึงควรตีความภายใต้ข้อจำกัดนี้ ประการแรก การวัดบน validator เดียวไม่สะท้อนต้นทุน consensus (PoH ร่วมกับ Tower BFT) ของเครือข่าย permissioned หลายโหนด ซึ่งเป็นที่มาของข้อสังเกตว่า block time เป็นคอขวด ประการที่สอง BlockBench, SmallBank และ TPC-C เป็น proxy ทั่วไป มิใช่ภาระงานพลังงาน อัตรา TPS ของเส้นทางชำระจริงจึงต่ำกว่าตัวเลข proxy อย่างมีนัย ประการที่สาม เส้นทางแบบลำดับในตารางที่ XIII ที่ 1.4–1.7 TPS เป็น latency-bound มิใช่เพดาน throughput และประการที่สี่ ตัวเลข TPS ของการวาด TPC-C เป็นค่าจุดเดียวต่อระดับ concurrency โดยรายงานช่วงความเชื่อมั่นเฉพาะของความหน่วง (latency CI95) เท่านั้น ยังไม่ได้ทำซ้ำหลายรอบเพื่อรายงานช่วงความเชื่อมั่นของ TPS การวัดแบบ open-loop หลายโหนดพร้อมทำซ้ำเพื่อหา peak TPS ที่ระดับ SLA เป็นงานในลำดับถัดไป (ดูหัวข้อ IX)

G. Per-instruction Compute-unit Profile

นอกจากต้นทุนของเส้นทางชำระหลักในหัวข้อ VIII.E ผู้วิจัยวัดต้นทุน compute ต่อ instruction ของโปรแกรมบนเซิร์ฟเวอร์ทุกตัวในเส้นทางการทำงานจริง เพื่อยืนยันด้วยการวัดว่าภาระ compute บนเซิร์ฟเวอร์ถูกจำกัดให้บางตามการออกแบบ ค่าทั้งหมดอ่านจาก *computeUnitsConsumed* ของธุรกรรม โดยวัดในกระบวนการ (in-process) ด้วย litesvm เหนือไบนารีที่ build แบบ default-feature ที่ commit 58cfc79 ของ gridtokenx-anchor ยกเว้นคำสั่งชำระ *settle_offchain_match* ที่วัดบน validator จริง (หัวข้อ VIII.E) เนื่องจาก compute unit เป็นค่าที่กำหนดได้แน่นอน (deterministic) จึงเทียบกันได้ระหว่างสองวิธี ผลสรุปในรูปที่ 11



รูปที่ 11. Measured per-instruction compute-unit cost across the on-chain programs (litesvm *computeUnitsConsumed*, default-feature build; *settle_offchain_match* measured on a live validator per หัวข้อ VIII.E), sorted by cost. Only the signature-verifying settlement path approaches half the 200,000 CU per-instruction budget (dashed); every other instruction stays well below — the measured signature of a thin settlement layer.

จากรูปที่ 11 ทุก instruction ยกเว้นเส้นทางชำระที่ตรวจสอบลายเซ็น (*settle_offchain_match*) ใช้ compute ไม่เกินประมาณ 28k CU หรือราว 14% ของงบตั้งต้น 200k CU โดยเส้นทางที่มีความถี่สูงอยู่ในระดับต่ำสุด ได้แก่ *update_meter_reading* (3.9k) และ *record_settlement_sharded* (5.4k) ซึ่งเป็นไปตามการออกแบบให้บัญชี PDA รายเอนทิตีบนเส้นทางอื่นไม่ล็อกเขียนบัญชีส่วนกลาง ผลนี้ยืนยันด้วยการวัดจริง (มิใช่การคาดการณ์) ว่างานประมวลผลหนัก เช่น การตรวจสอบรูปแบบข้อมูล การประเมินเงื่อนไข Grid stability และการจัดคิวคำสั่งซื้อขาย ถูกย้ายไปทำในชั้น Backend ก่อน ขณะที่ Smart Contract บังคับใช้เฉพาะเงื่อนไขขั้นต่ำที่ตรวจสอบได้บนเซิร์ฟเวอร์ (account ownership, ลายเซ็น Ed25519, order nullifier และสถานะ escrow) จึงทำหน้าที่เป็นชั้น settlement และ audit ที่ใช้ compute ต่ำตามที่ต้องการ

เส้นทางสร้างเหรียญจากพลังงานส่วนเกิน ซึ่งทดสอบแบบครบวงจรบน validator จริง (มิใช่ใน litesvm) ยืนยันภาพของ compute ที่บางเช่นเดียวกันจากปลายอีกด้านของ pipeline ธุรกรรม *mint_generation* สำหรับผู้รับรายเดียว (หัวข้อ IV.E) ที่วัดบนเซิร์ฟเวอร์ราว 4,455 CU สำหรับการสร้างบัญชี associated token แบบ idempotent และราว 1,237 CU สำหรับการเรียกข้ามโปรแกรม (CPI) *MintTo* ของ Token-2022 ทำให้ทั้งธุรกรรมสร้างเหรียญอยู่ต่ำกว่า 200,000 CU ต่อคำสั่งถึงหนึ่งลำดับขนาดต่ำกว่าเส้นทางชำระที่ตรวจสอบลายเซ็นมาก เช่นเดียวกับการชำระ การสร้างเหรียญหนึ่งครั้งมิได้ถูกจำกัดด้วยทั้ง compute และแพ็คเกจ ต้นทุนของการขยายขนาดมาจากการรวมผู้รับหลายรายเท่านั้น ค่าธรรมเนียมธุรกรรม 10,000 lamports สอดคล้องกับลายเซ็นที่จำเป็นสองชุด คือ platform authority (ซึ่งเป็นผู้จ่ายค่าธรรมเนียมด้วย) และผู้ร่วมลงนาม REC validator ที่บังคับ อันเป็นการควบคุมแบบสองฝ่ายเดียวกับที่กำกับการสร้างเหรียญจากการผลิต

ในด้าน I/O และ storage สถาปัตยกรรมส่งข้อมูลมาตรฐานวัดความถี่ผ่าน Aggregator Bridge เพื่อคัดกรองก่อน แล้วบันทึกเฉพาะ Event log ของธุรกรรมที่จำเป็นต่อการตรวจสอบย้อนหลังลงบนเซิร์ฟเวอร์ (ดูรูปที่ 4) การวัดปริมาณ record, retention policy และ storage cost เชิงปริมาณยังเป็นงานในอนาคต

H. Fleet-Scale Solver Throughput and Live On-chain Mint Validation

นอกเหนือจากการวัดเส้นทางรับข้อมูล (หัวข้อ VIII.B, หัวข้อ VIII.C) และต้นทุน/ปริมาณงานบนเซิร์ฟเวอร์ (หัวข้อ VIII.E, หัวข้อ VIII.F) ผู้วิจัยดำเนินการทดลองเพิ่มเติมสองรายการเพื่อตรวจสอบขอบเขตความสามารถของชุดจำลองในมิติที่แตกต่างกัน ได้แก่ (ก) ต้นทุนการคำนวณของ solver และการสร้างค่าอ่านเมื่อขนาดพล็อตเพิ่มขึ้นมาก โดยไม่ผ่านเครือข่าย และ (ข) การยืนยันว่าปริมาณไฟฟ้าส่วนเกิน (surplus) ที่คำนวณได้จากการจำลองนั้นไปถึงการ mint บนเซิร์ฟเวอร์ได้ครบทุกขั้นตอน ทั้งสองการทดลองนี้เป็นการรันสำรวจ (exploratory) เพียงรอบเดียว มิใช่การทำซ้ำหลายรอบเพื่อรายงานค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานเช่นเดียวกับหัวข้อ VIII.C จึงควรตีความผลเป็นค่าบ่งชี้เชิงออกแบบที่รอการวัดซ้ำ มิใช่ตัวเลขอ้างอิงขั้นสุดท้าย

1) Solver Throughput at Fleet Scale

การทดลองแรกขับเคลื่อน *SimulationEngine.tick()* โดยตรง (ข้ามเส้นทางเครือข่ายทั้งหมด รวมถึง Aggregator Bridge และบล็อกเชน) เพื่อแยกวัดเฉพาะต้นทุนของการสร้างค่าอ่านอุปกรณ์ (device-model generation) และการแก้สมการ power-flow บนโครงข่ายอ้างอิง 80 บัส ที่ขนาดพล็อต 10,000, 50,000 และ 100,000 มาตราวัด โดยกำหนดสัดส่วนมาตราวัดที่มีเมง PV แบบสุ่มต่อมาตราวัดที่ 10% (มิใช่การป้อนหนึ่งมาตราวัดต่อบัสตามที่ตั้งต้นเดิม) ผ่านพารามิเตอร์ *SOLAR_PROSUMER_RATIO* ผลสรุปไว้ในตารางที่ XV

ตารางที่ XV

Per-tick solver wall-clock cost vs. fleet size (single exploratory run, 4 ticks/case, no network egress).

$\times 10^5$

จำนวนมาตรวัด	สัดส่วน PV	median tick	p95 tick	max tick
10,000	9.90%	16.5 s	17.7 s	17.7 s
50,000	9.81%	97.6 s	107.8 s	107.8 s
100,000	9.97%	230.4 s	397.5 s	397.5 s

สัดส่วนมาตรวัดที่มี PV อยู่สูงค่าเป้าหมาย 10% เมื่อขนาดพลิตเพิ่มขึ้น สอดคล้องกับทฤษฎีจำนวนมาก (law of large numbers) ของการสุ่มเลือกประเภทมาตรวัดต่อราย และยืนยันว่าการแก้ไขให้พารามิเตอร์ *SOLAR_PROSUMER_RATIO/GRID_CONSUMER_RATIO/HYBRID_PROSUMER_RATIO* มีผลจริงต่อพลิตที่สร้างจากโครงข่าย *glm* (ก่อนการแก้ไข สัดส่วนถูกกำหนดตายตัวที่ 70/20/10 โดยไม่อ่านค่าพารามิเตอร์เหล่านี้เลย) เวลาต่อรอบ (tick) ขยายตัวแบบเหนือเชิงเส้นเล็กน้อยเมื่อเทียบกับขนาดพลิต (100,000 มาตรวัดใช้เวลาประมาณ 14 เท่าของ 10,000 มาตรวัดสำหรับพลิตที่ใหญ่กว่า 10 เท่า) ซึ่งบ่งชี้ว่าคอขวดอยู่ที่การสร้างค่าอ่านต่อมาตรวัด (ทำงานแบบ CPU-bound บนเทรดเดียวผ่าน *asyncio.to_thread*) มิใช่การแก้สมการ power-flow ที่มีขนาดโครงข่ายคงที่ไม่ขึ้นกับจำนวนมาตรวัด

2) Live On-chain Mint Proof

การทดลองที่ส่องตรวจสอบเส้นทางตรงข้ามกับการทดลองแรก กล่าวคือใช้พลิตขนาดเล็ก (100 มาตรวัด) แต่เปิดเส้นทางเครือข่ายเต็มรูปแบบ ตั้งแต่การลงทะเบียนเจ้าของมาตรวัดผ่าน IAM Service (register → verify → login พร้อมการลงทะเบียน PDA บนเชน) การลงทะเบียนกุญแจ Ed25519 ของแต่ละมาตรวัดในทะเบียนอุปกรณ์ของ Aggregator Bridge การส่งค่าอ่านที่ลงนามแล้วในทุกกรอบผ่าน TLS แบบยืนยันตัวตนฝั่งเซิร์ฟเวอร์ พร้อม payload DLMS/COSEM ที่เข้ารหัสด้วย AES-256-GCM² ไปจนถึงธุรกรรม mint บนเชนจริงที่ Chain Bridge ลงนามและส่งเมื่อหน้าต่าง settlement ปิดลง ทั้งหมดรันต่อ solana-test-validator เดียวเครื่องเดียวกันที่ใช้ในหัวข้อ VIII.F

ผลการรัน 100 มาตรวัด 5 รอบ (tick) ยืนยันว่าทุกขั้นตอนของเส้นทางทำงานสอดคล้องกันแบบ end-to-end: การลงทะเบียนเจ้าของมาตรวัดทั้งหมดใช้เวลาประมาณ 11 วินาที ค่าอ่านที่ส่งทุกรายการผ่านการตรวจลายเซ็นและถูกกระจายเข้าสู่ Redis Stream ตามโซน และที่สำคัญที่สุดคือเกิดธุรกรรม mint บนเชนจริง 140 รายการครอบคลุม 20 มาตรวัดที่ไม่ซ้ำกัน แต่ละรายการมีลายเซ็นธุรกรรม Solana และหมายเลข slot ที่ตรวจสอบได้จริง (ตัวอย่างเช่น mint ปริมาณ 0.08568 kWh ด้วยลายเซ็น *3NbmxvqEKRLfM2yEYJLNy7axVcibwBXTzVb* ที่ slot 1424) ผลนี้ปิดช่องว่างที่การทดลองแรกกังวลไม่ครอบคลุม กล่าวคือ การทดลองแรกขับเคลื่อน *tick()* โดยตรงโดยไม่เรียก *start()* จึงไม่เคยติดต่อกับ Aggregator Bridge หรือเชนเลย ขณะที่การทดลองนี้พิสูจน์ว่าตัวเลขไฟฟ้าส่วนเกินสุทธิที่ชุดจำลองคำนวณนั้นสอดคล้องกับสิ่งที่ถูก mint จริงบนเชน

ทั้งนี้การทดลองนี้จึงจำกัดขนาดไว้เล็ก เนื่องจากการลงทะเบียนผ่าน IAM ใช้การติดต่อ HTTP หนึ่งรอบต่อเจ้าของมาตรวัดหนึ่งราย และการ mint แต่ละครั้งเป็นธุรกรรม Solana ที่ลงนามจริงต่อหน้าต่าง settlement หนึ่งหน้าต่าง การรันพลิตขนาด 10,000–100,000 มาตรวัดแบบเปิดเส้นทางเครือข่ายเต็มรูปแบบเช่นนี้จึงไม่อยู่ในขอบเขตของงานนี้ และควรเป็นงานในอนาคตที่ขยายทั้งกำลังการลงทะเบียนของ IAM และความสามารถของ validator ทดสอบให้รองรับ (ดูหัวข้อ IX)

3) ปริมาณงาน Mint จริงบนเชนที่ระดับพลิตขนาดใหญ่ (Fleet-Scale Live Mint Throughput)

การทดลองที่สามปิดช่องว่างส่วนหนึ่งของการทดลองก่อนหน้านี้ทั้งไว้เป็นงานอนาคต กล่าวคือขับเคลื่อนเส้นทางเครือข่ายเต็มรูปแบบ (การลงทะเบียนผ่าน IAM การส่งค่าอ่านแบบเข้ารหัสที่ลงนามแล้ว การชำระ และการ mint ผ่าน Chain Bridge) ที่ขนาดพลิตระดับพัน มิใช่ 100 มาตรวัด โดยใช้ชุดเครื่องมือทดสอบเฉพาะทางกับ solana-test-validator เดียวเครื่องเดียวกัน ค่าอ่านไฟฟ้าส่วนเกินสุทธิหนึ่งรายการต่อมาตรวัดถูกปรับย้อนเวลา (backdate) ให้ตกอยู่ในหน้าต่าง settlement 15 นาทีที่ปิดไปแล้ว เพื่อให้รอบกวาดล้าง (sweep) ประมวลผลได้ทันทีโดยไม่ต้องรอเวลาจริงผ่านไป ผลสรุปไว้ในตารางที่ XVI เช่น

เดียวกับสองการทดลองข้างต้น ผลเหล่านี้เป็นการรันสำรวจเพียงรอบเดียว มิใช่การทำซ้ำหลายรอบ

ตารางที่ XVI
ปริมาณงาน mint บนเชนจริงที่ระดับพลิตขนาดใหญ่ (การรันสำรวจเพียงรอบเดียว).

พลิต	Minted	Overall TPS	Steady TPS (10 s median)	Peak TPS (10 s)
1,000	1,000 / 1,000	29.3	—	—
10,000	10,000 / 10,000 ³	18.4	37.7	123.2
25,000 (tuned)	25,000 / 25,000	23.2	14.6	193.4

ที่ค่าตั้งต้นก่อนปรับแต่ง (baseline) การส่งพลิต 25,000 มาตรวัดในครั้งเดียวทำให้คิวของ mint consumer ที่ Chain Bridge รับภาระเกิน กล่าวคือ envelope เข้าคิวจนเกินเพดานความเก่า (staleness cap) 55 วินาทีของ aggregator เร็วกว่าที่ consumer ซึ่งจำกัด concurrency ไว้คงที่ที่ 8 การยืนยันพร้อมกัน (in-flight confirmation) จะระบายคิวได้ทัน envelope ที่ถูกปฏิเสธจึงถูกส่งซ้ำ (republish) ทำให้คิวล้นขึ้นอีก เกิดเป็นวงจรบ่อนกลีบเชิงบวก (positive-feedback congestion collapse) ที่ goodput ลดลงจาก 16.7 เหลือ 3.7 mint ต่อวินาที ก่อนหยุดการรันด้วยมือ (mint สำเร็จ 2,559 จาก 25,000 รายการ) การปรับสามจุด ได้แก่ เพิ่ม concurrency ของ mint consumer (8 → 64) ข้ามขั้นตอนจำลองก่อนส่งจริง (pre-flight simulation) ที่ซ้ำซ้อน และขยายงบเวลารอการตอบกลับของ aggregator (30 วินาที → 120 วินาที) ขจัดปัญหาการยุบตัวนี้ได้ทั้งหมด แกวที่ปรับแต่งแล้วในตารางข้างต้นจึงรันสำเร็จครบโดยไม่มีการปฏิเสธจากความเก่า (stale-reject) และไม่มีการขายจำนวนซ้ำจากการส่งซ้ำ (republish amplification) เลย เทียบกับค่าตั้งต้นที่มีการปฏิเสธจากความเก่าถึง 65,117 ครั้ง และ outbox ขนาด 22,528 รายการถูกส่งซ้ำทุกรอบ

การรันขนาด 10,000 มาตรวัดยังเผยอีกเชิงปฏิบัติการจริงที่ควรรายงานแยกต่างหาก กล่าวคือผู้ใช้ส่วนหนึ่งที่ลงทะเบียนแล้วสูญเสียการเชื่อมโยงกระเป๋าเงิน (wallet-link) เมื่อ IAM Service เองถูกระบบปฏิบัติการฆ่าเนื่องจากหน่วยความจำเกิน (OOM) ระหว่างช่วงลงทะเบียนที่ concurrency สูงที่สุด ต้นเหตุคือมิตเดิลแวร์วัดผล (metrics) ของ IAM ติดป้ายตัวนับ Prometheus ด้วยเส้นทางค่าขอตามตัวอักษรจริง มีใช้ตาม route template ทำให้ทุกการเรียก *PATCH/wallets/{id}* — ซึ่งเรียกหนึ่งครั้งต่อผู้ใช้หนึ่งรายเพื่อกำหนดกระเป๋าเงินหลัก — เพิ่มชุดป้าย (label series) ถาวรใหม่ทุกครั้ง หน่วยความจำที่ค้างอยู่จึงเพิ่มจนเต็มจนการลงทะเบียนหยุดนิ่ง มิใช่แค่ *3NbnK6Y1Z64BR4KK296M7Hj7k* จำกัดหน่วยความจำของคอนเทนเนอร์ ไฟฟ้าส่วนเกินมิได้สูญหายถาวรแต่อย่างใด มาตรวัดที่ได้รับผลกระทบถูกทำเครื่องหมาย “ไม่มีกระเป๋าเงินลงทะเบียน รอ retry” และ durable mint outbox ได้ mint ให้ใหม่เข้าสู่หน้าต่าง settlement เดิมเมื่อการเชื่อมโยงกระเป๋าเงินได้รับการซ่อมแซม (คือส่วนทาง 79 รายการในตารางที่ XVI) แต่เหตุการณ์นี้เป็นตัวอย่างที่เป็นรูปธรรมว่าเหตุใดตัวชี้วัดต่อค่าขอจึงต้องติดป้ายตาม route template มิใช่เส้นทางตามตัวอักษรจริง ในภาระงานใดก็ตามที่มี path parameter รายเอนเทิตี

4) ปริมาณงานการเขียนค่าอ่านบนเชนตามจำนวนมาตรวัด (On-chain Telemetry Write Scaling)

การทดลองที่สี่แยกวัดเฉพาะขั้นบนเชนของเส้นทางค่าอ่าน กล่าวคือคำสั่ง *submit_meter_reading* ของโปรแกรม oracle เพียงลำพัง โดยข้ามขั้น IAM และ Aggregator Bridge ทั้งหมด เพื่อตอบคำถามที่การทดลอง mint ข้างต้นยังไม่ตอบ ว่าความสามารถการเขียนสถานะบนเชนเสื่อมถอยหรือไม่เมื่อจำนวนบัญชีมาตรวัด (per-meter PDA) เติบโตถึงระดับแสน ชุดเครื่องมือทดสอบสร้างธุรกรรมที่ลงนามแล้วฝั่งไคลเอนต์ต่อ blockhash ที่แคชไว้ ส่งแบบ *skipPreflight* ผ่านวงเปิดที่จำกัดธุรกรรมค่าส่งไว้ 3,000 รายการ และยืนยันผลด้วยการสอบถามสถานะแบบกลุ่ม (*getSignatureStatuses* ครั้งละ 256 ลายเซ็น ทุก 1.5 วินาที ความหน่วงที่รายงานจึงถูกควอนไทซ์ที่ ±1.5 วินาที) ที่ขนาดพลิต 10,000, 50,000, 100,000, 150,000 และ 200,000 มาตรวัด ขนาดละ 2 รอบ (epoch) รอบแรกเป็นการสร้างบัญชี PDA รายมาตรวัด (init ต้นทุน compute สูงกว่า) รอบที่สองเป็นการเขียนทับสถานะเดิม (steady) ทุกขนาดรันต่อเนื่องบน solana-test-validator เดียวตัวเดียวกันโดยไม่ล้าง ledger ระหว่าง

² การตรวจใบรับรองฝั่งไคลเอนต์ของ IoT gateway มีอยู่ในโค้ดแล้วแต่ถูกปิดไว้ในการติดตั้งนี้ เส้นทาง ingest จึงเป็น TLS แบบยืนยันฝั่งเซิร์ฟเวอร์ มิใช่ mutual TLS โดยการพิสูจน์ตัวตนอุปกรณ์ที่ใช้จริงคือ API key ร่วมกับลายเซ็น Ed25519 ต่อค่าอ่าน ตามหัวข้อ III

³ 9,921 รายการ mint สำเร็จภายในหน้าต่างเฝ้าสังเกตโดยตรง ส่วนอีก 79 รายการ mint สำเร็จภายหลังผ่านกลไก retry ของ durable mint outbox — ดูเหตุการณ์ wallet-link ด้านล่าง

ขนาด กรณี 200,000 มาตราวัดจึงเริ่มทำงานบนชุดบัญชีที่มีอยู่แล้วกว่า 3.1 แสน PDA และเมื่อสิ้นสุดการทดลอง validator ถือบัญชี PDA มาตราวัดสะสมจากการทดลองนี้ 510,000 บัญชี รวมธุรกรรมทั้งการทดลอง 1,020,000 รายการ ผลสรุปไว้ในตารางที่ XVII เช่นเดียวกับการทดลองอื่นในหมวดนี้ ผลเป็นการรันสำรวจขนาดละรอบเดียว

ตารางที่ XVII

ปริมาณงานการเขียนค่าอ่านบนเซนตามจำนวนมาตรวัด (การรันสำรวจเพียงรอบเดียวต่อขนาด; ความหน่วงควอนไทล์ ± 1.5 s จากรอบสอบถามสถานะ).

มาตรวัด	สำเร็จทั้งหมด	TPS (init)	TPS (steady)	p50 ms	p95 ms	CU steady (med)
10,000	19,908 / 20,000	439.2	429.1	1,546	2,354	15,025
50,000	99,524 / 100,000	319.8	243.1	1,868	3,064	13,525
100,000	199,048 / 200,000	409.6	455.5	1,553	2,346	15,060
150,000	298,572 / 300,000	449.4	426.2	1,586	2,402	13,560
200,000	398,096 / 400,000	438.5	443.2	1,582	2,391	13,560

ข้อค้นพบหลักคือปริมาณงานคงที่ตามขนาด กล่าวคือ TPS ช่วง steady อยู่ในย่าน 426–455 ตั้งแต่ 10,000 ถึง 200,000 มาตรวัด (ค่า 243 ที่ขนาด 50,000 เป็นการชะลอชั่วคราวของ validator ที่ไม่ปรากฏขึ้นในขนาดที่ใหญ่กว่า) ความหน่วง p50 ราว 1.5–1.9 วินาที และต้นทุน compute ต่อธุรกรรมคงที่ (steady median 13,525–15,060 CU ต่ำกว่าบน 200,000 CU ต่อคำสั่งมาก) โดยไม่ขึ้นกับจำนวนบัญชีสะสม ผลยืนยันเชิงประจักษ์ว่าการออกแบบสถานะแบบ PDA รายการมาตรวัด (บัญชี *MeterState* ต่อมาตรวัด โดยเส้นทางร้อนไม่เขียนบัญชีส่วนกลาง *OracleData* เลย) ทำให้การเขียนค่าอ่านขนานกันได้ง่ายได้ Sealevel และจุด serialize ที่เหลืออยู่คือกระเป๋าสตางค์ gateway ตัวเดียวที่เป็นผู้จ่ายค่าธรรมเนียม (fee-payer ถูก write-lock ทุกธุรกรรม) มีขนาดของชุดบัญชี ธุรกรรมที่ไม่สำเร็จทั้งหมด (0.46–0.48% ของแต่ละขนาด) เป็นการปฏิเสธเชิงตรรกะแบบกำหนดได้ (deterministic) จากตัวตรวจความผิดปกติของ oracle เอง กล่าวคือค่าอ่านสังเคราะห์ส่วนน้อยละเมิดเพดานอัตราส่วนผลิตต่อบริโภคซึ่งตรวจด้วยการคูณไขว้เชิงจำนวนเต็ม (*AnomalousReading*) โดยไม่มีการปฏิเสธจากการส่ง คิวกัน หรือ blockhash หมดยุคแม้แต่รายการเดียวในทุกขนาด อนึ่ง ตัวนับรวมส่วนกลางบน *OracleData* คงค่าศูนย์ตลอดการทดลองตามการออกแบบ (เส้นทางร้อนเป็นแบบอ่านอย่างเดียวต่อบัญชีนี้ การกระหายยอดรวมเป็นหน้าที่ของคำสั่งเชิงบริหาร *aggregate readings* ที่เรียกเป็นคาบ) ข้อจำกัดสำคัญคือผลทั้งหมดมาจาก validator เดียวและผู้ส่งกระเป๋าสตางค์ เพดานที่วัดได้จึงเป็นสมบัติของการตั้งค่านี การกระจายภาระข้ามหลาย gateway fee-payer และการวัดบนเครือข่ายหลายโหนดยังเป็นงานอนาคต (ดูหัวข้อ IX)

5) การจำลองวงจรชีวิตโทเคนครบหนึ่งเดือนภายใต้สามแบบจำลองราคา (Month-long Community Lifecycle under Three Price Models)

การทดลองที่ห้าประกอบทุกชั้นของระบบเข้าด้วยกันเป็นวงจรครบรอบเดือน ปฏิทิน กล่าวคือรับค่าอ่านทั้งเดือนของชุมชนจำลองเข้าสู่ oracle บนเซน เปิดตลาดซื้อขายรายวัน แล้วขับวงจรชีวิตโทเคนเต็มเส้นทาง (mint $\rightarrow$ escrow deposit $\rightarrow$ settlement ที่ตรวจลายเซ็น Ed25519 $\rightarrow$ burn $\rightarrow$ การรับรอง REC ปลายเดือน) โดยรันซ้ำทั้งหมดภายใต้แบบจำลองราคา (หัวข้อ V.A) สามแบบ ได้แก่ การประมูลราคาเดียว (uniform clearing) การจับคู่แบบราคาต่อคู่ (CDA, discriminatory pricing) และการรับประกันซื้อคืนของการไฟฟ้าที่อัตราคงที่ (utility buyback) 2.20 บาทต่อ kWh ชุดข้อมูลคือชุมชน 80 มาตรวัด (prosumer 12 ราย ผู้บริโภค 68 ราย) ระยะ 30 วัน ช่วงบันทึก 15 นาที รวมค่าอ่าน 230,400 รายการ พลังงานรวมทั้งเดือน ผลิต 7,030 kWh บริโภค 139,421 kWh และส่วนเกินสุทธิ 1,734 kWh สร้างจากตัวจำลองมาตรวัดด้วยเมล็ดสุ่ม (seed) คงที่เพื่อให้ทุกแบบจำลองราคาเห็นข้อมูลป้อนเข้าชุดเดียวกันทุกประการ แต่ละแบบจำลองราคาต้องรันบน ledger ที่สร้างใหม่ทั้งชุด (validator ใหม่ deploy โปรแกรมใหม่ และ initialize สถานะใหม่ต่อรอบ) เนื่องจาก oracle บังคับให้ timestamp ต่อมาตรวัดเพิ่มขึ้นอย่างเข้มงวด และ registry บังคับขอบเขตการอนุรักษ์ปริมาณ mint การ replay ชุดข้อมูลเดิมซ้ำบน ledger เดิมจึงเป็นไปได้โดยการออกแบบ ค่าอ่านทั้งหมดใช้ timestamp ย้อนหลังตามชุดข้อมูลจริง (oracle ปฏิเสธเฉพาะ timestamp อนาคต) ทั้งเดือนจึงถูกบีบเวลา replay ต่อเนื่องได้โดยไม่ต้องรอเวลาจริง

ขาที่เข้าร่วมกันของทั้งสามการรันให้ผลแทบเหมือนกันทุกตัวเลข กล่าวคือการรับค่าอ่านเข้าสู่เซนสำเร็จครบ 230,400 รายการทั้งสามรอบโดยไม่มีการปฏิเสธหรือส่งซ้ำเลย ใช้เวลา 433.7–433.8 วินาที (ต่างกันไม่เกิน 0.1 วินาทีระหว่างสามการรันอิสระ — ตัวบ่งชี้การทำซ้ำได้ของชาร์ปข้อมูลที่ดีกว่าการรันสำรวจรอบเดียวในหมวดก่อนหน้านี้) คิดเป็น 531 ค่าอ่านต่อวินาที (53.1 ธุรกรรมต่อวินาทีที่ 10 ค่าอ่านต่อธุรกรรม ต้นทุน compute มีฐาน 15,017 CU ต่อค่าอ่าน ความหน่วงยืนยัน p50 ประมาณ 1.38 วินาที) ส่วนตลาดรายวัน 30 รอบ (ผู้ขาย 12 รายภายใต้เพดานขาย 10 kWh ต่อวัน ผู้ซื้อ 68 ราย) ส่งคำสั่งสำเร็จ 2,397 จาก 2,398 รายการ (หนึ่งรายการหมดอายุ) ที่ 17.94 ธุรกรรมต่อวินาที ผลคำนวณวงจรชีวิตโทเคนสรุปไว้ในตารางที่ XVIII

ตารางที่ XVIII

วงจรชีวิตโทเคนครบหนึ่งเดือน (80 มาตรวัด, 30 วัน, ค่าอ่าน 230,400 รายการต่อแบบจำลองราคา; การรันต่อเนื่องบน ledger ใหม่).

แบบจำลองราคา	Settled (kWh)	Burned (kWh)	มูลค่าชำระ (THB)	เวลารวมทั้งเดือน	อัตราเป็นเวลา
Uniform clearing	1,734.35	1,734.35	5,888	837.7 s	3,094x
CDA	1,734.35	1,734.35	5,418	837.7 s	3,094x
Utility buyback	— (retire ๗๓)	1,734.35	3,685	657.6 s	3,942x

ข้อค้นพบหลักมีสามประการ ประการแรก สมบัติการอนุรักษ์พลังงานคงอยู่ครบตลอดทั้งเดือนบนเซนจริง กล่าวคือปริมาณที่ mint เท่ากับที่ settle และเท่ากับที่ burn ที่ 1,734.35 kWh พอดีในทั้งสองแบบจำลองแบบ P2P (uniform และ CDA ซึ่งผ่านขา settlement ที่ตรวจลายเซ็น Ed25519 ครบ 356 รายการต่อรอบ ต้นทุน compute มีฐาน 99,045 CU ต่อรายการ) ส่วนแบบ buyback ผู้ขายเผาโทเคนคืนโดยตรงโดยไม่ใช้ขา P2P ตามนิยามของกลไก และปริมาณ burn ยังคงครบ 1,734.35 kWh เท่าเดิม การรับรอง REC ปลายเดือนเป็นศูนย์ในทุกแบบ เนื่องจากเพดานขาย 10 kWh ต่อวันแทบไม่ผูกมัดกับชุดข้อมูลนี้ (ปริมาณที่ถูกเพดานกันไว้รวมทั้งเดือนเพียง 0.004 kWh) ประการที่สอง ลำดับรายได้ของ prosumer สอดคล้องกับการวิเคราะห์เชิงกลไกในหัวข้อ V.B กล่าวคือมูลค่าการชำระรวม uniform (5,888 บาท) สูงกว่า CDA (5,418 บาท) และทั้งสองสูงกว่า buyback (3,685 บาท) อย่างมีนัย ยืนยันเชิงประจักษ์บนเซนจริงว่ากลไกค้นหาราคาแบบ P2P ให้ผลตอบแทนแก่ผู้ผลิตสูงกว่าการขายคืนการไฟฟ้าที่อัตราคงที่ ประการที่สาม เดือนปฏิทินเต็มถูกดูดซับในเวลา 837.7 วินาที (แบบ P2P) และ 657.6 วินาที (buyback ซึ่งขา settlement สั้นกว่า) คิดเป็นอัตราเป็นเวลา 3,094 เท่าและ 3,942 เท่าของเวลาจริงตามลำดับ แสดงว่าชุดเครื่องมือนี้ใช้ประเมินนโยบายราคาระดับเดือนได้ในเวลาระดับนาที ทั้งนี้ผลยังคงอยู่ภายใต้ข้อจำกัดเดียวกับการทดลองอื่นในหมวดนี้ คือ validator เดียวและหนึ่งการรันต่อแบบจำลองราคา (ชาร์ปข้อมูลเป็นข้อยกเว้นที่ได้ทำซ้ำสามรอบโดยบังเอิญเชิงโครงสร้าง) การวัดซ้ำเชิงสถิติและการรันบนเครือข่ายหลายโหนดยังเป็นงานอนาคต (ดูหัวข้อ IX)

IX. DISCUSSION AND LIMITATIONS

A. Discussion

ผลการออกแบบระบบจำลองชี้ให้เห็นว่าแนวทางการแยกชั้นการทำงานระหว่าง Smart Meter Simulator, Backend Service และ Solana Smart Contract ช่วยให้ระบบซื้อขายพลังงานแบบ Peer-to-Peer มีโครงสร้างที่ตรวจสอบได้และมีจุดขายระบบที่ชัดเจนขึ้น Backend Service ทำหน้าที่เป็นชั้นควบคุมหลักสำหรับตรวจสอบข้อมูล การจัดการสิทธิ์ และการประเมินเงื่อนไขด้าน Grid stability ก่อนส่งธุรกรรมไปยังบล็อกเชน ทำให้ Smart Contract ไม่ต้องรับภาระการประมวลผลทั้งหมดของระบบไม่ใคร่กริด

การใช้ Solana Smart Contract ในฐานะ Settlement และ Audit layer ช่วยเพิ่มความโปร่งใสของธุรกรรมพลังงาน เนื่องจากคำสั่งซื้อขายและสถานะการชำระพลังงานสามารถตรวจสอบย้อนหลังได้ อย่างไรก็ตาม ระบบยังต้องให้ความสำคัญกับความปลอดภัยของโครงข่ายไฟฟ้าเป็นลำดับแรก การอนุมัติธุรกรรมจึงไม่ควรพิจารณาเฉพาะราคาและปริมาณพลังงานเท่านั้น แต่ต้องพิจารณาเงื่อนไขด้านเสถียรภาพของระบบและสถานะการเชื่อมต่อของอุปกรณ์ร่วมด้วย

ผลการประเมินสามส่วนสนับสนุนสมมติฐานการออกแบบที่ให้บล็อกเชนเป็นชั้น settlement แบบบาง (thin settlement layer) ประการแรกเส้นทางรับข้อมูลรักษาสัดส่วนการสูญหายให้ใกล้ศูนย์ ($\leq 0.03\%$) จนถึง

ภาระงาน 640 เครื่อง (ดูหัวข้อ VIII.C) โดยอัตราการรับข้อมูลที่ได้เป็น lower bound ของไคลเอนต์ผู้ส่งและยังเพิ่มขึ้นแบบเอกฐานตามจำนวนมาตรวัด จึงยังไม่สรุปเรื่องจุดอิ่มตัวของบริการ บ่งชี้ว่าการย้ายงานตรวจสอบและรวบรวมข้อมูลที่มีความถี่สูงไว้ในชั้น off-chain ไม่ได้กลายเป็นคอขวดของระบบ ภายใต้ภาระระดับที่ทดสอบ ประการที่สอง ต้นทุนการชำระธุรกรรมบนเชนที่วัดได้ 96,707 CU ต่อคำสั่ง คิดเป็นประมาณ 48% ของงบ compute ตั้งต้นต่อคำสั่ง (ดูหัวข้อ VIII.E) แสดงว่าเมื่อพิจารณาเชิงธุรกิจส่วนใหญ่ถูกบังคับใช้ในเชนแล้ว ภาระที่เหลือบนเชนอยู่ในระดับที่เปิดช่องเชิง compute ให้รวมหลายคำสั่งในธุรกรรมเดียว (batch settlement) ได้ แม้การรวมจริงจะถูกจำกัดด้วยขนาดแพ็คเกจธุรกรรมก่อนเพดาน compute (ดูหัวข้อ IV.E) จึงต้องปรับวิธีบรรจุคำสั่งเพื่อให้ลดต้นทุนต่อคำสั่งได้จริงในงานถัดไป ประการที่สาม การที่ต้นทุนบนเชนเป็นค่าที่กำหนดได้แน่นอน (deterministic CU) มากกว่าจะผันแปรตามค่าธรรมเนียมตลาดแบบเครือข่ายสาธารณะ สอดคล้องกับเหตุผลเชิงการกำกับดูแลในการเลือกเครือข่าย consortium แบบ PoA (ดูหัวข้อ II) ที่ต้นทุนธุรกรรมคาดการณ์ได้และไม่ขึ้นกับความผันผวนของค่า gas

ในเชิงกลไกเศรษฐศาสตร์ การแยกบทบาทของโทเคนสามประเภท คือ โทเคนพลังงานที่รับรองด้วย REC สำหรับการส่งมอบ เหยี่ยว THBC ที่ตรึงค่าเงินบาทสำหรับการตั้งราคาและชำระเงิน และโทเคน GRX สำหรับการ stake และการกำกับดูแล ช่วยให้การชำระธุรกรรมเป็นแบบ DvP ที่ผูกการส่งมอบพลังงานเข้ากับการชำระเงินในธุรกรรมเดียว และให้สิทธิการกำกับดูแลเครือข่ายอยู่ภายใต้กรอบ consortium ที่ควบคุมการรับรอง REC และ allow-list ของ aggregator ได้อย่างชัดเจน ทั้งนี้ได้วิเคราะห์ภัยเชิงเศรษฐศาสตร์เบื้องต้นแบบจำลอง (model-derived) ของยอดสุทธิที่ผู้ขายได้รับและส่วนเพิ่มเทียบกับ feed-in tariff ไว้ในหัวข้อ V.B อย่างไรก็ตาม การวัดคุณสมบัติเชิงเศรษฐศาสตร์เต็มรูปแบบ เช่น ประสิทธิภาพการจัดสรรเชิงสวัสดิการ (allocative efficiency / welfare) และพฤติกรรมเชิงกลยุทธ์ของผู้เข้าร่วม ยังอยู่นอกขอบเขตของการประเมินเชิงสถาปัตยกรรมในงานนี้

เมื่อเทียบกับแพลตฟอร์ม permissioned ที่ใช้แพร่หลายในวาระกรรมการซื้อขายพลังงาน P2P อย่าง Hyperledger Fabric [15] (ดูตารางที่ II) การออกแบบในงานนี้ต่างกันเชิงสถาปัตยกรรมในสามจุด ประการแรก Fabric ใช้แบบจำลองการประมวลผลแบบ execute-order-validate ที่ chaincode รันบน endorsing peers ก่อนเรียงลำดับผ่าน orderer (Raft/BFT) ขณะที่งานนี้ใช้แบบจำลอง account ของ Solana ที่ประมวลผลธุรกรรมแบบขนานตาม Sealevel เมื่อชุดบัญชีที่เขียนไม่ทับซ้อนกัน (ดูหัวข้อ VI.E.7) ซึ่งเอื้อต่อการขยายการส่งคำสั่งรายเอนทิตี แต่ทำให้การชำระที่ระบทยอดส่วนกลางถูกผูกกับการเขียนส่วนกลาง (global-write-bound) ดังที่วัดได้ในหัวข้อ VIII.F ประการที่สอง ต้นทุนการประมวลผลในงานนี้แสดงเป็นหน่วย compute (CU) ที่กำหนดได้แน่นอนและมีเพดานชัดเจนต่อธุรกรรม ต่างจาก Fabric ที่ไม่มีแนวคิดงบประมาณ compute ต่อธุรกรรมแบบเดียวกัน ทำให้ต้นทุนบนเชนของงานนี้คาดการณ์และตรวจจบการถดถอย (regression) ได้ตรงไปตรงมา ประการที่สาม งานนี้บังคับการตรวจที่มาของค่าอ่านมาตรวัดด้วยลายเซ็น Ed25519 และการกันส่งข้ามผ่าน order nullifier ภายในตรรกะของโปรแกรมโดยตรง แทนการพึ่งนโยบาย endorsement ภายนอก ทั้งนี้การเปรียบเทียบนี้เป็นเชิงคุณภาพ (architectural) เนื่องจากยังไม่มีกรวัดแบบ head-to-head บนภาระงานพลังงานเดียวกัน ซึ่งเป็นงานเปรียบเทียบเชิงปริมาณที่เปิดไว้ในอนาคต

B. Limitations

ข้อจำกัดสำคัญของงานนี้คือระบบยังอยู่ในระดับการจำลอง โดยข้อมูลพลังงานมาจาก Smart Meter Simulator และยังไม่ได้ทดสอบร่วมกับอุปกรณ์ Smart Meter จริงหรือระบบไฟฟ้าภาคสนาม ดังนั้นผลลัพธ์จึงสะท้อนความเหมาะสมของสถาปัตยกรรมและกระบวนการทำงานเบื้องต้น มากกว่าประสิทธิภาพเชิงปริมาณในสภาพแวดล้อมจริง

นอกจากนี้การประเมินยังครอบคลุมเฉพาะบางส่วนของเส้นทางการทำงานทั้งหมด กล่าวคือวัดอัตราการรับข้อมูลของเส้นทาง telemetry ingest และต้นทุน compute ของคำสั่งชำระธุรกรรมบนเชนเป็นรายส่วน แต่ยังไม่ได้วัดความหน่วงแบบ end-to-end ตั้งแต่ค่าอ่านมาตรวัดจนถึงการยืนยันบนเชน และอัตราการจับคู่คำสั่งภายใต้ปริมาณคำสั่งหลายระดับ ส่วนปริมาณงานธุรกรรม (throughput) ของเส้นทางชำระวัดบน validator เดียวแล้ว (ดูหัวข้อ

VIII.F) แต่ยังไม่ครอบคลุมเครือข่าย permissioned หลายโหนดที่รวมต้นทุน consensus และยังไม่ได้ประเมิน batch settlement ที่กล่าวถึงในเชิงออกแบบอย่างเป็นทางการทดลองจริง ในด้านความเชื่อถือ เงื่อนไขปริมาณพลังงานและ Grid stability ถูกบังคับใช้ในชั้น off-chain โดย Aggregator Bridge และ oracle authority ซึ่งการสมรู้ร่วมคิดหรือการถูกประณามของส่วนนี้ยังไม่ถูกป้องกันด้วยกลไกเชิงรหัสวิทยา (ดูหัวข้อ III) ทั้งสมมติฐานความปลอดภัยของฉันทามติ (PoH ร่วมกับ Tower BFT) ภายใต้เครือข่าย permissioned ที่กำกับการเข้าร่วมแบบ PoA ก็เป็นสมมติฐานเชิงสถาปัตยกรรมที่ยังไม่ได้ประเมินเชิงปริมาณภายใต้ validator ที่มีพฤติกรรมเบี่ยงเบน

C. Threats to Validity

ผลการวัดในงานนี้มีข้อควรระวังในการตีความสามประการ ประการแรก (internal validity) ภาระงานในการทดลอง ingest ถูกสร้างจากไคลเอนต์ผู้ส่งเพียงตัวเดียวที่ทำงานแบบ asynchronous อัตราการรับข้อมูลที่ได้จึงเป็น lower bound ที่สะท้อนข้อจำกัดฝั่งผู้ส่งร่วมด้วย มิใช่เพดานความสามารถที่แท้จริงของบริการรับข้อมูล (ดูหัวข้อ VIII.C) ประการที่สอง (construct validity) อัตราเชิงออกแบบ 5.33 รายการต่อวินาทีคำนวณจากจำนวนมาตรวัดและรอบการส่ง มิใช่อัตราสูงสุดที่วัดได้ การเปรียบเทียบตัวเลขทั้งสองจึงต้องระบุบริบทของแต่ละค่าให้ชัด ประการที่สาม (external validity) ต้นทุน compute ของคำสั่งชำระธุรกรรมวัดบน solana-test-validator รุ่นเดียว และค่าดังกล่าวเป็นต้นทุนเชิงตรรกะของโปรแกรม (deterministic CU) ที่ไม่ขึ้นกับฮาร์ดแวร์ แต่ความหน่วงและปริมาณงานบนเครือข่าย production จริงอาจต่างออกไป ผลทั้งหมดจึงควรตีความในฐานะการประเมินเชิงสถาปัตยกรรมบนระบบจำลอง

D. Future Work

งานในอนาคตควรพัฒนาการเชื่อมต่อกับ Smart Meter จริงผ่านมาตรฐาน DLMS/COSEM และประเมินเส้นทางการทำงานทั้งระบบให้ครบถ้วนขึ้น ได้แก่ การวัดความหน่วงแบบ end-to-end และการขยายผลการวัด throughput ของ settlement ที่รายงานบน validator เดียวในหัวข้อ VIII.F ไปสู่เครือข่าย permissioned หลายโหนดที่รวมต้นทุน consensus พร้อมการวัดแบบ open-loop เพื่อหาเพดาน TPS ที่ระดับ SLA และการทดลอง batch settlement เพื่อยืนยันการลดต้นทุน compute ต่อคำสั่ง รวมถึงการวัดเพดานของเส้นทางการรับข้อมูลด้วยผู้ส่งแบบขนานหลายตัวที่แยกต้นทุนฝั่งผู้ส่งออกจากบริการรับข้อมูล ในด้านความเชื่อถือ ควรลด residual trust ของชั้น off-chain ด้วย multi-oracle attestation หรือ cryptographic proof และประเมินฉันทามติของเครือข่าย permissioned (ที่กำกับการเข้าร่วมแบบ PoA) เชิงปริมาณภายใต้โหนดที่มีพฤติกรรมเบี่ยงเบน นอกจากนี้ควรเพิ่มแบบจำลองด้าน Grid stability, Islanding detection และ Demand response เพื่อให้การประเมินความปลอดภัยของการซื้อขายพลังงานใกล้เคียงการใช้งานจริงมากขึ้น โดยการทดลองถัดไปควรกำหนด workload ที่ทำซ้ำได้ เช่น จำนวน Meter, sampling interval, transaction mix, validator count, queue depth, failure/retry policy และตัวชี้วัด market-clearing output อย่างชัดเจน

นอกเหนือจากข้างต้น ทิศทางการประเมินเชิงปริมาณที่เปิดไว้อย่างชัดเจนมีสามประการ ได้แก่

- **ประสิทธิภาพการจัดสรรเชิงสวัสดิการ (allocative efficiency / welfare):** วัดส่วนเกินจากการซื้อขาย (gains from trade) ของกลไก CDA เทียบกับฐานราคาเดียว (uniform-price clearing) และ feed-in tariff บนข้อมูลการจับคู่จริงจากภาระงานจำลองเต็มรูปแบบ เพื่อยืนยันส่วนเพิ่มเชิงเศรษฐศาสตร์ที่วิเคราะห์เชิงแบบจำลองไว้ในหัวข้อ V.B ด้วยการวัดจริงแทนการคำนวณจากพารามิเตอร์คงที่
- **ต้นทุนต่อการซื้อขายในหน่วยเงิน (cost per trade):** แปลงต้นทุน compute ต่อธุรกรรมเป็นค่าธรรมเนียมในหน่วย lamport และเงินบาทที่อัตราที่กำหนด แล้วรายงานสัดส่วนต้นทุนต่อมูลค่าธุรกรรม (fee-to-trade-value ratio) ซึ่งเป็นเกณฑ์ตัดสินสำคัญต่อการนำไปใช้จริง
- **ความพร้อมใช้งานและการเปรียบเทียบฐาน (liveness & baseline):** ประเมินความพร้อมใช้งาน (availability) ของเครือข่าย permissioned เมื่อมีโหนด validator ล้มเหลวบางส่วน (1-of-N down) และวัดเปรียบเทียบแบบ head-to-head เชิงปริมาณกับแพลตฟอร์ม

permitted อื่น โดยเฉพาะ Hyperledger Fabric บนภาระงานพลังงานเดียวกัน เพื่อต่อยอดการเทียบเชิงคุณภาพในตารางที่ II ให้เป็นการวัดรวมที่เทียบกันได้โดยตรง

X. CONCLUSION

บทความนี้นำเสนอการพัฒนากระบวนการซื้อขายพลังงานแสงอาทิตย์แบบ Peer-to-Peer สำหรับไมโครกริด โดยผสาน Smart Meter Simulator, Backend/Aggregator Bridge และ Solana-compatible Smart Contract เข้าด้วยกัน ระบบถูกออกแบบให้รองรับข้อมูลพลังงานแบบเวลาจริง ตรวจสอบคำสั่งซื้อขายผ่านบริการ Backend และบันทึกผลธุรกรรมที่ผ่านเงื่อนไขลงบนบล็อกเชนเพื่อเพิ่มความโปร่งใสและความสามารถในการตรวจสอบย้อนหลัง

ผลการประเมินเชิงสถาปัตยกรรมชี้ให้เห็นว่าแนวทาง Microservice ร่วมกับการจัดคิวธุรกรรมผ่าน NATS JetStream มีศักยภาพในการลดการผูกติดกันของภาระงานระหว่างการรับข้อมูล การตรวจสอบธุรกรรม และการบันทึกผลบนบล็อกเชนได้ นอกจากนี้ การจำกัดบทบาทของ Smart Contract ให้เป็นชั้น Settlement and Audit layer ช่วยกำหนดขอบเขต compute บนบล็อกเชนให้ชัดเจนขึ้น การประเมินครอบคลุมสี่ด้าน ได้แก่ เส้นทางรับข้อมูล (telemetry ingest) ที่รองรับอัตราเชิงออกแบบ 5.33 รายการต่อวินาทีได้อย่างต่อเนื่องโดยไม่มีการสูญเสีย และเมื่อเพิ่มภาระแบบขั้นบันไดถึง 640 เครื่อง (เฉลี่ยจาก 5 รอบ) ยังคงสัดส่วนการสูญเสียไว้ไม่เกิน 0.03% โดยไม่พบจุดอ้อมตัวภายในขีดของคลเอนด์ผู้ส่งเดียวที่ใช้ทดสอบ ด้านการจับคู่คำสั่งในชั้น off-chain เครื่องจับคู่ CDA ประมวลผลได้ประมาณ 3.1×10^4 คู่คำสั่งต่อวินาทีในหน่วยความจำ ด้านต้นทุนการชำระธุรกรรมบนเซนวัดได้ 96,707 compute units ต่อคู่คำสั่ง (ประมาณ 48% ของงบ compute ตั้งต้น) ซึ่งเปิดช่องให้รวมคู่คำสั่งแบบ batch ได้ และด้านปริมาณงานการชำระจริงวัดได้ราว 0.5 ธุรกรรมต่อวินาทีบน validator เดียว ซึ่งถูกผูกกับการเขียนบัญชีส่วนกลางโดยการออกแบบแต่ยังรองรับได้ราว 450 การชำระต่อหน้าตาแต่ละยี่ห้อ 900 วินาที เกินความต้องการของภาระงานที่ทดสอบมาก ผลลัพธ์ด้านสนับสนุนการออกแบบที่ให้บล็อกเชนเป็นชั้น settlement แบบบาง ส่วนการวัดความหน่วงแบบ end-to-end การทดลอง batch settlement และการประเมินบนอุปกรณ์และเครือข่ายจริงเป็นงานในลำดับถัดไป ซึ่งเป็นรากฐานสำคัญสำหรับการต่อยอดสู่การใช้งานในระดับอุตสาหกรรมพลังงานในอนาคต

REFERENCES

- [1] W. Tushar, T. K. Saha, C. Yuen, D. Smith, และ H. V. Poor, "Peer-to-Peer Trading in Electricity Networks: An Overview", *IEEE Transactions on Smart Grid*, ปี 11, ฉบับที่ 4, น. 3185–3200, 2020.
- [2] T. Morstyn, A. Teytelboym, และ M. D. McCulloch, "Bilateral Contract Networks for Peer-to-Peer Energy Trading", *IEEE Transactions on Smart Grid*, ปี 10, ฉบับที่ 2, น. 2026–2035, 2019.
- [3] A. Paudel, K. Chaudhari, C. Long, และ H. B. Gooi, "Peer-to-Peer Energy Trading in a Prosumer-Based Community Microgrid: A Game-Theoretic Model", *IEEE Transactions on Industrial Electronics*, ปี 66, ฉบับที่ 8, น. 6087–6097, 2019.
- [4] IEEE Standards Association, "IEEE Std 1547-2018: IEEE Standard for Interconnection and Interoperability of Distributed Energy Resources with Associated Electric Power Systems Interfaces". 2018.
- [5] IEEE Standards Association, "IEEE Std 2030.7-2017: IEEE Standard for the Specification of Microgrid Controllers". 2018.
- [6] J. Guerrero, A. C. Chapman, และ G. Verbic, "Decentralized P2P Energy Trading Under Network Constraints in a Low-Voltage Network", *IEEE Transactions on Smart Grid*, ปี 10, ฉบับที่ 5, น. 5163–5173, 2019.

- [7] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System". 2008.
- [8] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform". 2014.
- [9] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger". 2014.
- [10] D. Yaga, P. Mell, N. Roby, และ K. Scarfone, "Blockchain Technology Overview", technical report NIST IR 8202, 2018. doi: 10.6028/NIST.IR.8202.
- [11] E. Mengelkamp, P. Staudt, J. Gartner, และ C. Weinhardt, "Trading on Local Energy Markets: A Comparison of Market Designs and Bidding Strategies", *Applied Energy*, ปี 210, น. 870–880, 2018.
- [12] E. Munsing, J. Mather, และ S. Moura, "Blockchains for Decentralized Optimization of Energy Resources in Microgrid Networks", ใน *2017 IEEE Conference on Control Technology and Applications*, 2017, น. 2164–2171.
- [13] A. Esmat, M. de Vos, Y. Ghiassi-Farrokhi, P. Palensky, และ D. Epema, "A Novel Decentralized Platform for Peer-to-Peer Energy Trading Market with Blockchain Technology", *Applied Energy*, ปี 282, น. 116123, 2021, doi: 10.1016/j.apenergy.2020.116123.
- [14] J. Kang, R. Yu, X. Huang, S. Maharjan, Y. Zhang, และ E. Hossain, "Enabling Localized Peer-to-Peer Electricity Trading Among Plug-in Hybrid Electric Vehicles Using Consortium Blockchains", *IEEE Transactions on Industrial Informatics*, ปี 13, ฉบับที่ 6, น. 3154–3164, 2017.
- [15] E. Androulaki และคณะ, "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains", ใน *Proceedings of the Thirteenth EuroSys Conference*, 2018, น. 1–15. doi: 10.1145/3190508.3190538.
- [16] L. Lamport, R. Shostak, และ M. Pease, "The Byzantine Generals Problem", *ACM Transactions on Programming Languages and Systems*, ปี 4, ฉบับที่ 3, น. 382–401, 1982, doi: 10.1145/357172.357176.
- [17] M. Castro และ B. Liskov, "Practical Byzantine Fault Tolerance", ใน *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, 1999, น. 173–186.
- [18] M. Andoni และคณะ, "Blockchain Technology in the Energy Sector: A Systematic Review of Challenges and Opportunities", *Renewable and Sustainable Energy Reviews*, ปี 100, น. 143–174, 2019, doi: 10.1016/j.rser.2018.10.014.
- [19] Z. Tanis, A. Durusu, และ N. Altintas, "A Comprehensive Review on Peer-to-Peer Energy Trading: Market Structure, Operational Layers, Energy Cooperatives and Multi-energy Systems", *IET Renewable Power Generation*, ปี 19, ฉบับที่ 1, 2025, doi: 10.1049/rpg2.70075.
- [20] G. B. Bhavana, R. Anand, J. Ramprabhakar, V. P. Meena, V. K. Jadoun, และ F. Benedetto, "Applications of blockchain technology in peer-to-peer energy markets and green hydrogen supply chains: a topical review", *Scientific Reports*, ปี 14, ฉบับที่ 1, 2024, doi: 10.1038/s41598-024-72642-2.
- [21] G. B. Bhavana, R. Anand, J. Ramprabhakar, J. M. Guerrero, N. Thakkar, และ A. Ambikapathy, "Comparative evaluation and simulation of blockchain consensus mechanisms for secure and

scalable peer to peer energy trading in microgrids”, *Scientific Reports*, ปี 15, ฉบับที่ 1, 2025, doi: 10.1038/s41598-025-27431-w.

- [22] IEEE Standards Association, “IEEE Std 2030.8-2018: IEEE Standard for the Testing of Microgrid Controllers”. 2018.
- [23] SPIFFE Project, “X.509-SVID Specification”. 2018.
- [24] NATS.io, “JetStream”. 2024.
- [25] Coral, “Anchor Documentation”. 2026.
- [26] A. Yakovenko, “Solana: A New Architecture for a High Performance Blockchain”. 2018.
- [27] Solana Foundation, “Program Derived Addresses”. 2026.
- [28] Python Software Foundation, “Python 3.11 Documentation”. 2026.
- [29] D. P. Chassin, K. Schneider, และ C. Gerkensmeyer, “GridLAB-D: An Open-Source Power Systems Modeling and Simulation Environment”, ใน *2008 IEEE/PES Transmission and Distribution Conference and Exposition*, 2008, น. 1–5. doi: 10.1109/TDC.2008.4517260.
- [30] S. Ramirez, “FastAPI Documentation”. 2026.
- [31] A. A. Hagberg, D. A. Schult, และ P. J. Swart, “Exploring Network Structure, Dynamics, and Function Using NetworkX”, ใน *Proc. 7th Python in Science Conference*, 2008, น. 11–15.
- [32] K. S. Anderson, C. W. Hansen, W. F. Holmgren, A. R. Jensen, M. A. Mikofski, และ A. Driesse, “pvlb python: 2023 Project Update”, *Journal of Open Source Software*, ปี 8, ฉบับที่ 92, น. 5994, 2023, doi: 10.21105/joss.05994.
- [33] L. Thurner และคณะ, “pandapower: An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems”, *IEEE Transactions on Power Systems*, ปี 33, ฉบับที่ 6, น. 6510–6521, 2018, doi: 10.1109/TPWRS.2018.2829021.
- [34] S. Joshi, “Feasibility of Proof of Authority as a Consensus Protocol Model”. 2021. doi: 10.48550/arXiv.2109.02480.
- [35] Solana Foundation, “Tokens on Solana”. 2026.
- [36] Solana Foundation, “Solana Documentation”. 2026.